

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 13.09.2026, 16:58:35
Файлов исходного кода: 94
Суммарный объём кода: 1.39 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракса (KMP) [id: kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасик [id: aho-corasick]

24. src/data/vizStepBus.ts
25. src/data/vizSync.ts

Билеты (учебный контент)

26. src/data/tickets/ticket1_5.ts
27. src/data/tickets/ticket14_20.ts
28. src/data/tickets/ticket21_24.ts
29. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

30. src/components/ArticulationPointsViz.tsx
31. src/components/BellmanFordViz.tsx
32. src/components/BfsViz.tsx
33. src/components/ChapterImage.tsx
34. src/components/ChapterNav.tsx
35. src/components/ChapterView.tsx
36. src/components/ColoringViz.tsx
37. src/components/ComplexityClassesViz.tsx
38. src/components/ComponentsViz.tsx
39. src/components/DfsBridgesSimulator.tsx
40. src/components/DijkstraViz.tsx
41. src/components/DPVisualizer.tsx
42. src/components/EulerSimulator.tsx
43. src/components/ExpressQuiz.tsx
44. src/components/FloydViz.tsx
45. src/components/GraphTraversalViz.tsx
46. src/components/HeapViz.tsx
47. src/components/hints/demoKit.tsx
48. src/components/hints/demosExtra.tsx
49. src/components/hints/demosGraph.tsx
50. src/components/hints/demosStruct.tsx
51. src/components/hints/MiniDemo.tsx
52. src/components/hints/SimulatorModal.tsx
53. src/components/hints/TermPopover.tsx
54. src/components/JohnsonViz.tsx
55. src/components/KosarajuViz.tsx
56. src/components/KruskalSimulator.tsx
57. src/components/MnemonicCards.tsx

```
88. scripts/export/common.mjs
89. scripts/export/guide-pdf-entry.tsx
90. scripts/export/render-pages.mjs
91. scripts/selftest/bundle.ts
92. scripts/selftest/lazy.ts
```

CI / автоматизация

```
93. .github/workflows/deploy-pages.yml
94. .github/workflows/rebuild-pdf.yml
```

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/94: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```

` .github/workflows/deploy-pages.yml `

Полное содержимое после изменений

- name: Build for the repository subpath
env:
 VITE_BASE: /algv0/
run: npm run build
- name: Make GitHub Pages fallback
run: cp dist/index.html dist/404.html
- name: Upload Pages artifact
uses: actions/upload-pages-artifact@v5
with:
 path: ./dist

deploy:

```
name: Publish to GitHub Pages
needs: build
runs-on: ubuntu-latest
environment:
  name: github-pages
  url: ${ steps.deployment.outputs.page_url }
steps:
  - name: Deploy
    id: deployment
    uses: actions/deploy-pages@v5
```

....

Patch относительно `main`

```diff

diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml

new file mode 100644

index 00000000..17418e3

--- /dev/null

+++ b/.github/workflows/deploy-pages.yml

@@ -0,0 +1,60 @@

+name: Deploy app to GitHub Pages

+

+on:



```
uses: actions/setup-node@v7
```

with:

node-version: "24"

cache: npm

- ```
- name: Install ImageMagick + DejaVu fonts
```

run: 1

```
sudo apt-get update -qq
```

```
sudo apt-get install -y --no-install-recommen
```

На Ubuntu политика ImageMagick иногда запре

```
# разрешаем то, что нужно пайплайну (text ->
```

```
for policy in /etc/ImageMagick-6/policy.xml /
```

```
if [ -f "$policy" ]; then
```

```
sudo sed -i 's/rights="none" pattern="\{P
```

fi

done

```
convert -version
```

- ```
- name: Install dependencies
```

```
run: npm ci --no-audit --no-fund
```

- name: Step 1 – код → txt

```
run: npm run export:txt
```

- name: Step 2 – txt → png

env:

```
EXPORT DENSITY: ${github.event.inputs.density}
```

```
run: npm run export:png
```

- name: Step 3 – png → pdf

```
run: npm run export:pdf
```

- name: Type-check приложения

```
run: npx tsc --noEmit
```

```
continue-on-error: true
```

- name: Summary

run: 1

```
.....
git push origin "HEAD:${GITHUB_REF_NAME}"
....

Patch относительно `main`

````diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

  steps:
    - name: Checkout
      uses: actions/checkout@v4
+    uses: actions/checkout@v6
      with:
        fetch-depth: 0
        persist-credentials: true

    - name: Setup Node.js
      uses: actions/setup-node@v4
+    uses: actions/setup-node@v7
      with:
        node-version: "22"
+        node-version: "24"
        cache: npm

    - name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
      } >> "$GITHUB_STEP_SUMMARY"

    - name: Upload artifacts (PDF + TXT)
      uses: actions/upload-artifact@v4
+    uses: actions/upload-artifact@v7
      with:
        name: full-code-export
        path: |
```

Универсальное пособие • полный исходный код

@@ -3,6 +3,7 @@

```
    <head>
      <meta charset="UTF-8" />
      <meta name="viewport" content="width=device-width,
+    <link rel="icon" type="image/svg+xml" href="%BASE_
      <title> Дискретка для тупых – Интерактивный гид по
    </head>
    <body class="bg-slate-950 text-slate-100 min-h-screen
....
```

`public/favicon.svg`

Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64">
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
````
```

Patch относительно `main`

````diff

```
diff --git a/public/favicon.svg b/public/favicon.svg
new file mode 100644
index 00000000..3bbbba0
```

--- /dev/null

+++ b/public/favicon.svg

@@ -0,0 +1,5 @@

```
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64">
+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>
+ <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
+ <circle cx="47" cy="21" r="4" fill="#34d399"/>
+</svg>
````
```



```

    if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target)) return;
    if (e.metaKey || e.ctrlKey || e.altKey) return;
    if (e.key === "ArrowRight" && next) goTo(next.id);
    if (e.key === "ArrowLeft" && prev) goTo(prev.id);
  };
  window.addEventListener("keydown", onKey);
  return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

```

```

const progress = useMemo(() => ((index + 1) / chapters.length) * 100, [index, chapters]);

```

```

return (
  <div className="min-h-screen bg-slate-950 text-slate-50">
    <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
      {activeTab === "guide" ? (
        <>
          <MobileToc
            selectedChapterId={activeChapterId}
            setSelectedChapterId={goTo}
            currentTitle={activeChapter.title}
            index={index}
            total={chapters.length}
          />
          <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
            {/* Сайдбар только на десктопе – на мобильном скрывается */}
            <div className="hidden lg:block lg:w-80 shrink-0" >
              <Sidebar selectedChapterId={activeChapterId} />
            </div>
            <main id="main-content" className="flex-1 min-h-screen" >
              {/* Шапка главы с быстрыми переходами */}
              <div className="flex items-center justify-between" >
                <div className="min-w-0">
                  <div className="text-[10px] uppercase" >
                    {activeChapter.category || "Универсальное пособие"}
                  </div>

```

```

        <PythonCompiler />
    }}

    <SimulatorModal vizId={modalVizId} onClose={() =>

    <footer className="p-8 text-center text-slate-600"
        © 2026 Universal Educational Guide. Интерактивны
    </footer>
</div>
);
}
....

```

Patch относительно `main`

```

````diff
diff --git a/src/App.tsx b/src/App.tsx
index 2091f33..4583756 100644
--- a/src/App.tsx
+++ b/src/App.tsx
@@ -7,9 +7,10 @@ import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { SimulatorHub } from "../components/SimulatorHub";
+import { PythonCompiler } from "../components/PythonCompiler";

export default function App() {
- const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
+ const [activeTab, setActiveTab] = useState<"guide" | "simulator"> "guide";
 const [activeChapterId, setActiveChapterId] = useState<string>("");
 const [modalVizId, setModalVizId] = useState<string>("");

@@ -95,7 +96,7 @@ export default function App() {
 </main>
 </div>
 </>
-) : (
+) : activeTab === "simulator" ? (

```

```

 }
 };

 return (
 <header className="bg-slate-900 border-b border-slate-800">
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
 <div className="flex items-center gap-3 w-full">
 <div className="bg-gradient-to-tr from-indigo-500 to-indigo-600 w-6 h-6">
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-white">
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-medium">
 Подготовка к экзаменам: Алгоритмы, Структуры
 </p>
 </div>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-auto">
 <button
 id="tab-guide-btn"
 onClick={() => setActiveTab('guide')}
 className={`flex-1 lg:flex-none flex items-center justify-center text-white font-medium transition-colors duration-200 whitespace-nowrap ${
 activeTab === 'guide'
 ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500/50'
 : 'text-slate-400 hover:text-white'
 }`}
 >
 <BookOpen className="w-4 h-4" /> Учебное пособие
 </button>
 <button
 id="tab-simulator-btn"

```

```
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
 :bg-sky-600/20 border border-sky-500/30 tra
 title="Скачать полный TXT файл со всем кодом
 >
 <FileText className="w-4 h-4" /> TXT

<button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
 er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
</button>
</div>
</div>
</header>
);
};
...

```

### Patch относительно `main`

```
````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
  import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
  import { chapters } from '../data/content';
  import { downloadBookHtml } from '../utils/exportHtml'
```

```
        download="full_code_all_pages.txt"
        target="_blank"
        rel="noreferrer"
    ....

## `src/components/PythonCompiler.tsx`

### Полное содержимое после изменений

```tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play } from "lucide-react";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
```

```

runtime.setStderr({ batched: (message) => stderr.p
runtime.setStdin({ stdin: () => (inputLines.length
await runtime.runPythonAsync(code);

const text = [...stdout, ...stderr].join("");
setOutput(text || "Готово: программа ничего не вы
} catch (error) {
const captured = [...stdout, ...stderr].join("");
setOutput(`${captured}${captured && !captured.end
setRuntimeMessage("Не удалось выполнить код – про
} finally {
setRunning(false);
}
}, [code, stdin, running, runtimeReady]));

const reset = () => {
setCode(STARTER_CODE);
setStdin("");
setOutput("Нажмите «Запустить», чтобы увидеть резул
});

return (
<main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
<div className="max-w-6xl mx-auto">
<div className="flex flex-col sm:flex-row sm:items-center">
<div>
<div className="inline-flex items-center gap-2">
<Terminal className="w-4 h-4" /> Боковая панель
</div>
<h2 className="text-2xl sm:text-3xl font-extrabold">
<p className="text-slate-400 mt-2 max-w-2xl">
Пишите небольшой код и запускайте его прямо в браузере,
который переводит CPython в WebAssembly.
</p>
</div>
<a
href="https://pyodide.org/"
target=" blank"

```

```

 value={code}
 onChange={({event}) => setCode(event.target.value)}
 onKeyDown={({event}) => {
 if ((event.ctrlKey || event.metaKey) && event.defaultPrevented) {
 event.preventDefault();
 void runCode();
 }
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[360px] resize-y bg-transparent focus:ring-2 focus:ring-inset focus:ring-slate-400"
 placeholder="Напишите Python-код..."
 />

 <div className="border-t border-slate-800 border-b border-slate-800 border-l border-slate-800 border-r border-slate-800" >
 <label className="block px-4 pt-3 text-slate-400" >
 Ввод для input() • по одной строке на клавишу
 </label>
 <textarea
 id="python-stdin"
 value={stdin}
 onChange={({event}) => setStdin(event.target.value)}
 spellCheck={false}
 rows={2}
 placeholder={'Например:\nАлиса'}
 className="block w-full resize-y bg-transparent border border-slate-800 border-t border-slate-800 border-l border-slate-800 border-r border-slate-800 border-b border-slate-800"
 />
 </div>
</section>

<section className="rounded-2xl border border-slate-800 border-t border-slate-800 border-l border-slate-800 border-r border-slate-800 border-b border-slate-800" >
 <div className="flex items-center justify-between" >
 <div className="flex items-center gap-2 text-slate-400" >
 <Terminal className="w-4 h-4 text-emerald-500" />
 </div>


```

```

--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python
+name = "алгоритмы"
+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>;
+ setStdout: (options: { batched: (message: string) => void }) => void;
+ setStderr: (options: { batched: (message: string) => void }) => void;
+ setStdin: (options: { stdin: () => string | number | null }) => void;
+};
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = null;
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL)
+ .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL }));
+ }
+ return runtimePromise;
+}

```



```

+ const captured = [...stdout, ...stderr].join("")
+ setOutput(`${captured}${captured && !captured.endsWith("\n") ? "\n" : ""}`);
+ setRuntimeMessage("Не удалось выполнить код – попробуйте снова.");
+ } finally {
+ setRunning(false);
+ }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть результат.");
+ };
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:items-center">
+ <div>
+ <div className="inline-flex items-center gap-2">
+ <Terminal className="w-4 h-4" /> Боковая панель
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-extrabold">
+ <p className="text-slate-400 mt-2 max-w-2xl">
+ Пишите небольшой код и запускайте его прямо в браузере,
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.5"
+ >
+ Как это работает <ExternalLink className="text-slate-400" />
+
+ </div>
+ </div>
+ </div>
+ </main>
+);

```

```

+ }}
+ spellCheck={false}
+ aria-label="Код Python"
+ className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+ placeholder="Напишите Python-код..."
+ />
+
+ <div className="border-t border-slate-800"
+ <label className="block px-4 pt-3 text-[
+ Ввод для input() • по одной строке на
+ </label>
+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.ta
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-tr
ceholder:text-slate-700"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border borde
+ <div className="flex items-center justify-l
+ <div className="flex items-center gap-2
+ <Terminal className="w-4 h-4 text-emera
+ </div>
+ <span className={`inline-flex items-cent
+ {runtimeReady && <CheckCircle2 classNa
+ {runtimeReady ? "готов" : "не загружен
+
+ </div>
+ <pre className="min-h-[360px] max-h-[560px
px] leading-6 text-slate-300">
+ {output}

```

```

const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algov0/, локально
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
});
\`\`\`
```

### Patch относительно `main`

```
\`\`\`diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__filename);
```

Универсальное пособие • полный исходный код

-----  
Ты – AI-ассистент, работающий в Gemini App Canvas (или ...  
При запросе пользователя создать или сгенерировать новую ...  
\*\*строго\*\* придерживаться готовых шаблонов разметки,

## Правила для малых страниц (Билетов / Глав)

1. **\*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML
2. **\*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна ...  
 1"`, `content: ...`) для вставки в `src/data/content.ts`.
3. **\*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, `bg-indigo-100`)
  - Определение/правило в центре (`border-blue-500` или ...)
  - Обязательная сетка аналогий (2 колонки: неформальный ...)
  - "Душные" детали (код, формулы, сложные доказательства ...)
4. **\*\*Не ломай верстку:\*\*** Не придумывай свои CSS-классы ...  
 ерах `src/data/content.ts`. Не используй чистый markdown

-----  
ФАЙЛ 4/94: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 14 | РАЗМЕР: 600

-----  
<!doctype html>  
<html lang="ru" class="dark bg-slate-950 text-slate-100">  
 <head>  
 <meta charset="UTF-8" />  
 <meta name="viewport" content="width=device-width, ...>  
 <link rel="icon" type="image/svg+xml" href="%BASE\_URL%/icon.svg">  
 <title> Универсальное пособие по алгоритмам – билет ...</title>  
 </head>  
 <body class="bg-slate-950 text-slate-100 min-h-screen">  
 <div id="root"></div>  
 <script type="module" src="/src/main.tsx"></script>  
 </body>  
</html>

```

 mp/lazy-selftest.cjs --jsx=automatic --loader:.cs
},
"dependencies": {
 "clsx": "2.1.1",
 "lucide-react": "^1.21.0",
 "motion": "^12.40.0",
 "pdfkit": "^0.20.1",
 "react": "19.2.6",
 "react-dom": "19.2.6",
 "svg-to-pdfkit": "^0.1.8",
 "tailwind-merge": "3.4.0"
},
"devDependencies": {
 "@tailwindcss/vite": "4.1.17",
 "@types/node": "22.19.17",
 "@types/pdfkit": "^0.17.6",
 "@types/react": "19.2.7",
 "@types/react-dom": "19.2.3",
 "@vitejs/plugin-react": "5.1.1",
 "esbuild": "^0.24.0",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3",
 "vite": "7.3.2",
 "vite-plugin-singlefile": "2.3.0"
},
"description": "Интерактивное пособие по алгоритмам и
}
```

-----  
ФАЙЛ 7/94: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 125 | РАЗМЕР: 1000

-----  
# algv0 – Универсальное пособие по алгоритмам и структурам

Интерактивная web-читалка (React + Vite + Tailwind) с синтаксисом  
и автоматическим экспортом всего исходного кода в PDF.

Универсальное пособие • полный исходный код

-----  
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док  
и коммитит обновлённые `public/export/full\_code\_all\_page`  
(с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code`

## ## Аудит контента

```
```bash
node scripts/audit-coverage.mjs          # какие темы без
node scripts/audit-coverage.mjs --md     # markdown-табли
```
```

Актуальный отчёт: [`TOPICS\_AUDIT.md`](./TOPICS\_AUDIT.md)

## ## Как устроен интерфейс

- \* **Темы 1–24** — канонический список билетов; одна стр.  
(`src/data/content.ts`, таблица `CANON` задаёт точные
- \* **Python-компилятор** — боковая панель (кнопка «Python  
отладчик. Список «переменных темы» для каждой страниц  
`src/data/vizSync.ts`; значения из выполняемого Python  
объекты демонстрации (матрицы, графы, деревья) напрям  
эталонный код, инициализация подставляет заранее зада
- \* **Содержание** — на десктопе боковая панель с поиском  
на телефоне — липкая строка «Содержание · N из M» и в
- \* **Переходы между темами** — кнопки «Назад / Вперёд» в  
«Предыдущая / Следующая тема» под текстом; на клавиату
- \* **Всплывающие подсказки по терминам** — текст главы а  
известные термины (BFS, куча, бор, суффиксная ссылка,  
подчёркиваются, а по наведению/тапу показывается карт  
мини-анимацией и кнопкой «Показать симулятор» (симуля  
Словарь — `src/data/glossary.ts`, мини-анимации — `src
- \* **Реестр интерактивов** — `src/components/vizRegistry`  
и для панели под главой, и для модалки из подсказки.

## ## Структура

## ## Списки

\* Стандартные списки оформлены через `<ul class="list-n`

## ## Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`

2. Элементы `.analogy-badge` получают выделение маркера

3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация]`.

4. Элементы списка `<li>` получают маркер `-`.

5. Результат извлекается в виде текста и чистится от лишнего

---

ФАЙЛ 8/94: TOPICS\_AUDIT.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 106 | РАЗМЕР: 1.5 КБ

---

# Аудит покрытия тем: где нет «объяснения на пальцах» и

> Сгенерировано скриптом `node scripts/audit-coverage.m`

> Критерии:

> \* **Аналогия** — в HTML главы есть блок «Аналогия ...»

> \* **2D** — к главе привязан интерактивный визуализатор

>

> Всего глав в пособии: **25**.

---

## Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| Глава | id | Что делать |
|-------|----|------------|
|-------|----|------------|

|     |     |     |
|-----|-----|-----|
| --- | --- | --- |
|-----|-----|-----|

|                    |                      |                       |
|--------------------|----------------------|-----------------------|
| Алгоритмы на карте | <code>alg-map</code> | Страница-заглушка (8) |
|--------------------|----------------------|-----------------------|

---

> Замечание: «5. Двумерная матрица префиксных сумм» полна

| Глава                                | id                 | Чего не хватает     |
|--------------------------------------|--------------------|---------------------|
| 3. Декартово дерево (Trear)          | `trear`            | Нет ни визуализации |
| 8. Графы. Компоненты связности       | `graph-components` |                     |
| 24. Классы сложности, сведение задач | `complexity-class` |                     |

## Сводная таблица по всем главам

| #  | Глава                                           | Аналогий | 2D-виз. | SVG | Картинка |
|----|-------------------------------------------------|----------|---------|-----|----------|
| 2  | 2. Двумерная разреженная матрица (Sparse Table) |          |         |     |          |
| 3  | 3. Декартово дерево (Trear)                     | 2        | —       | —   | —        |
| 4  | 4. Splay-дерево                                 | 2        | —       | —   |          |
| 5  | 5. Двумерная матрица префиксных сумм            | 1        |         |     | —        |
| —  | Планарность: K5, K3,3 и формула Эйлера          | —        |         |     | —        |
| 6  | 6. Графы. Представление, DFS, BFS               | 4        |         | —   | —        |
| 8  | 8. Графы. Компоненты связности                  | 1        | —       | —   | —        |
| 9  | 9. Графы. Топологическая сортировка             | 1        |         |     | —        |
| 10 | 10. Графы. Компоненты сильной связности (SCC)   |          |         |     | —        |
| 12 | 12. Графы. Точки сочленения                     | 1        |         | —   | —        |
| 14 | 14. Графы. Кратчайший путь. Дейкстра            | 3        |         |     | —        |
| 15 | 15. Графы. Кратчайший путь. Форд–Беллман        | 1        |         |     | —        |
| 16 | 16. Графы. Кратчайший путь. Флойд               | —        |         |     | —        |
| 17 | 17. Графы. Алгоритм Джонсона                    | —        |         | —   | —        |
| 18 | 18. Графы. Остовное дерево. Краскал             | 1        |         |     | —        |
| 19 | 19. Графы. Остовное дерево. Прима               | 1        |         |     | —        |
| 20 | 20. Графы. Остовное дерево. Борувка             | 1        |         |     | —        |
| 21 | 21. Префикс-функция (π), КМП                    | 4        |         | —   | —        |
| 22 | 22. Z-функция                                   | 4        |         | —   | —        |
| 23 | 23. Алгоритм Ахо–Корасик                        | 2        |         | —   | —        |
| 24 | 24. Классы сложности, сведение задач            | 4        | —       |     | —        |
| —  | Обзор: Кратчайшие пути и Графы                  | —        |         | —   | —        |



```
 /* Path mapping */
 "baseUrl": ".",
 "paths": {
 "@/*": ["src/*"]
 },

 /* Linting */
 "strict": true,
 "noUnusedLocals": true,
 "noUnusedParameters": true,
 "noFallthroughCasesInSwitch": true
 },
 "include": ["src", "vite.config.ts"]
}
```

---

ФАЙЛ 10/94: vite.config.ts

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 36 | РАЗМЕР: 1

---

```
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локально
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
```

---

```
import { PythonCompiler } from "../components/PythonComp
```

```
export default function App() {
 const [activeChapterId, setActiveChapterId] = useState<string>()
 const [modalVizId, setModalVizId] = useState<string>()
 /** Боковая (на мобильном – нижняя) панель компилятора */
 const [compilerOpen, setCompilerOpen] = useState(false)
 /** Ширина панели компилятора (desktop), растягивается */
 const [compilerWidth, setCompilerWidth] = useState<number>()
 const cached = typeof window !== "undefined" ? Number(window.localStorage.getItem("compilerWidth")) : null
 return Number.isFinite(cached) && cached >= 380 ? cached : null
});
useEffect(() => {
 window.localStorage.setItem("compilerWidth", String(compilerWidth));
}, [compilerWidth]);
/** Высота терминала: null оставляет адаптивную высоту */
const [compilerHeight, setCompilerHeight] = useState<number>()
const cached = typeof window !== "undefined" ? Number(window.localStorage.getItem("compilerHeight")) : null
return Number.isFinite(cached) && cached >= 260 ? cached : null
});
useEffect(() => {
 if (compilerHeight !== null) window.localStorage.setItem("compilerHeight", String(compilerHeight));
}, [compilerHeight]);
/** Содержание свернуто (по умолчанию сворачиваем сам компилятор) */
const [sidebarCollapsed, setSidebarCollapsed] = useState<boolean>()
const cached = typeof window !== "undefined" ? window.localStorage.getItem("sidebarCollapsed") : null
if (cached !== null) return cached === "1";
return typeof window !== "undefined" && window.innerWidth < 380;
});
const toggleSidebar = useCallback(() => {
 setSidebarCollapsed((c) => {
 window.localStorage.setItem("sidebarCollapsed", c ? "1" : null);
 return !c;
 });
}, []);

// Desktopный отступ контента = ширине панели компилятора
const contentRef = useRef<HTMLDivElement>(null);
```

```

 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target)) return;
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id);
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, [goTo, next, prev]);

```

```

const progress = useMemo(() => ((index + 1) / chapters.length) * 100, [index, chapters]);

```

```

return (
 <div className="min-h-screen bg-slate-950 text-slate-50">
 <Navbar
 compilerOpen={compilerOpen}
 onToggleCompiler={() => setCompilerOpen((o) => !o)}
 />

```

```

 {/* При открытой панели контент сдвигается: снизу */}
 <div ref={contentRef} className={compilerOpen ? "p-4" : "p-4 pt-16"}>

```

```

 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

```

```

 <div className="flex flex-col lg:flex-row min-h-screen">
 {/* Сайдбар только на десктопе – на мобильных скрывается */}
 <div
 className={`hidden lg:block shrink-0 border-r border-slate-800 p-4 transition-all duration-200 ${sidebarCollapsed ? "hidden" : ""}`}
 >
 {!sidebarCollapsed && <Sidebar selectedChapterId={activeChapterId} />}
 </div>
 {/* Вкладка-переключатель содержания у левого сайдбара */}

```

```

 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 onOpenCompiler={openCompilerWithViz}
 />
 </main>
 </div>
</>

<footer className="p-8 text-center text-slate-600">
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>

<SimulatorModal vizId={modalVizId} onClose={() =>
 {
 compilerOpen && (
 <PythonCompiler
 chapterId={activeChapter.id}
 chapterTitle={activeChapter.title}
 width={compilerWidth}
 height={compilerHeight}
 onWidthChange={setCompilerWidth}
 onHeightChange={setCompilerHeight}
 onOpenGuide={() =>
 document.getElementById("chapter-viz")?.scrollTo(0,0)
 }
 onClose={() => setCompilerOpen(false)}
 />
)
 }
 </div>
);
}

```

```

const perTerm = new Map<string, number>();
const perSurface = new Map<string, number>();
const walker = document.createTreeWalker(root, NodeFilter.
 acceptNode(node) {
 const parent = (node as Text).parentElement;
 if (!parent) return NodeFilter.FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return NodeFilter.FILTER_REJECT;
 if (!node.nodeValue || node.nodeValue.trim().length === 0)
 return NodeFilter.FILTER_REJECT;
 return NodeFilter.FILTER_ACCEPT;
 },
 true);

const targets: Text[] = [];
let current = walker.firstChild();
while (current) {
 targets.push(current as Text);
 current = walker.nextSibling();
}

for (const textNode of targets) {
 const text = textNode.nodeValue ?? "";
 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE)
 continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);
 frag.appendChild(document.createTextNode(m[0]));
 }
 root.appendChild(frag);
}

```

```
// при смене главы
// eslint-disable-next-line react-hooks/exhaustive-deps
useEffect(run, deps);

// страховка: контент перерисовали, а подсказок в нём
useEffect(() => {
 const el = ref.current;
 if (!el) return;
 if (!el.querySelector(".term-hint")) run();
});
}
```

---

ФАЙЛ 13/94: src/index.css

КАТЕГОРИЯ: Ядро приложения | СТРОК: 175 | РАЗМЕР: 3.3 КБ

---

```
@import "tailwindcss";

@layer utilities {
 .neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
 }
 .neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
 }
 .neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
 }
 .neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
 }
 .no-scrollbar::-webkit-scrollbar {
 display: none;
 }
 .no-scrollbar {
```

```
 transform: translateY(0);
 }
}

.term-popover {
 animation: hintIn 0.14s ease-out;
}

/* Интерактивные термины в тексте главы (превью как в Википедии)
.term-hint {
 cursor: help;
 border-bottom: 1px dashed rgba(129, 140, 248, 0.75);
 color: #c7d2fe;
 border-radius: 3px;
 padding: 0 1px;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.term-hint:hover,
.term-hint:focus-visible {
 background: rgba(99, 102, 241, 0.22);
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Контент главы приходит готовым HTML — чиним его под себя
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}
```

```
 line-height: 1.35 !important;
 }
 .chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
 }
 .chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
 }
 .chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
 }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци
@keyframes demoProgress {
 from {
 width: 0%;
 }
 to {
```



-----  
ФАЙЛ 16/94: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

-----  
/// <reference types="vite/client" />

```
declare module '*.jpg' {
 const src: string;
 export default src;
}
declare module '*.jpg?url' {
 const src: string;
 export default src;
}
declare module '*.jpeg' {
 const src: string;
 export default src;
}
declare module '*.png' {
 const src: string;
 export default src;
}
```

=====

РАЗДЕЛ: КОНТЕНТ И ДАННЫЕ ПОСОБИЯ

=====

-----  
ФАЙЛ 17/94: src/data/additionalTickets.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 817 | РАЗМЕР: 10 000 В

-----  
import { Chapter } from "../types";

```
export const additionalTickets: Chapter[] = [
 {
 id: "sparse-table",
```

```

<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300"
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <pre class="bg-slate-950 p-4 rounded"
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);</pre>
 </div>
 </details>
 </div>
</div>
</section>`,
 }, {
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приоритету",
 category: "Продвинутые структуры",
 content: `
<section id="treap" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Treap
 <h2 class="text-2xl sm:text-3xl font-bold text-white">Декартово дерево
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400">Описание
 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-blue-300 italic">Декартово дерево — это бинарное дерево поиска, в котором узлы также имеют приоритет, и оно удовлетворяет свойству кучи.
 <p class="text-xl font-mono text-white">Пара (ключ, приоритет) называется парой. Пара — точка на декартовой плоскости.
 </div>
 </div>
 </div>

```

</div>

```
<div class="bg-slate-900/40 rounded-lg border-1" style="border-color: #444; border-radius: 10px; padding: 10px; margin: 10px 0;">
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
 быстрее $n \cdot \log n$.</p>
 <p class="text-slate-400 text-sm mb-1">
 <p class="text-slate-400 text-sm">• «Уб
аются merge за $O(h)$.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b class="text-slate-300 text-sm">
 "bg-slate-950 px-2 py-0.5 rounded text-xs font-mono text-rose-300 border border-1" style="border-color: #444; border-radius: 5px; padding: 2px 5px; margin: 5px 0;">
 оение — <code class="bg-slate-950 px-2 py-0.5 rounded text-xs font-mono text-rose-300 border border-1" style="border-color: #444; border-radius: 5px; padding: 2px 5px; margin: 5px 0;">
code> (сортировка + n спусков). Худший случ
 text-xs font-mono text-rose-300 border border-1" style="border-color: #444; border-radius: 5px; padding: 2px 5px; margin: 5px 0;">
ed text-xs font-mono text-rose-300 border border-1" style="border-color: #444; border-radius: 5px; padding: 2px 5px; margin: 5px 0;">
b
```

```
<p class="text-slate-500 text-xs mt-3 leading-tight">
и останется набор букв, как treap без y оста
но кричат по «!»: корень самый громкий, род
; равные x — идут влево по уставу.</p>
```

```
<details class="bg-slate-900/40 rounded-lg border-1" style="border-color: #444; border-radius: 10px; padding: 10px; margin: 10px 0;">
 <summary class="p-4 font-bold text-slate-300 border-bottom border-slate-700/50">
```

Код: split (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
```

```
pair<Node*, Node*> split(Node* t, int x) {
```

```
 if (!t) return {nullptr, nullptr};
```

```
 if (t->val <= x) {
```

```
 auto [L, R] = split(t->right, x);
```

```
 t->right = L;
```

```
 return {t, R};
```

```
 } else {
```

```
 auto [L, R] = split(t->left, x);
```

```

 </details>
 </div>
 </div>
 </section>`,
 },
 {
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Самонастраивающееся дерево: поднимаем",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-violet-600">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 ротов поднимает x в корень. Единственный ин
 </div>
 <div class="grid grid-cols-1 md:grid-cols-3">
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">
 <p class="text-white font-bold">мож
 <p class="text-slate-400 text-xs mt-1">
 </div>
 <div class="bg-slate-900 rounded-lg p-4">
 <p class="text-xs uppercase tracking-wider">
 <p class="text-emerald-300 font-bold">
 <p class="text-slate-400 text-xs mt-1">
 </div>
 <div class="bg-slate-900 rounded-lg p-4">

```

```

 x γ -----> α ρ
 / \ / \ / \
 α β ρ выше β γ</pre>

```

<p class="text-slate-300 text-sm mt-3">Обхо  
омать дерево поиска поворотами невозможно в  
омент «отстёжки», а врезка «Механика одного

<h4 class="text-lg font-bold text-white mt-3">

<p class="text-slate-300 text-sm mb-3">Смотр

<div class="overflow-x-auto -mx-2 px-2">

<table class="w-full text-xs sm:text-sm">

<thead>

<tr class="text-left text-slate">

<th class="py-2 pr-3 font-b">

<th class="py-2 pr-3 font-b">

<th class="py-2 pr-3 font-b">

<th class="py-2 font-bold">

</tr>

</thead>

<tbody class="text-slate-300">

<tr class="border-b border-slate">

<td class="py-3 pr-3 font-b">

<td class="py-3 pr-3">деда

<td class="py-3 pr-3">сам

<td class="py-3">х — корень

</tr>

<tr class="border-b border-slate">

<td class="py-3 pr-3 font-b">

<td class="py-3 pr-3">х и

<td class="py-3 pr-3"><span

<td class="py-3">ветка укор

</tr>

<tr>

<td class="py-3 pr-3 font-b">

<td class="py-3 pr-3">х и

<td class="py-3 pr-3"><span

<td class="py-3">р и г стал

</tr>

```
>O(n)</code>; n вставок подряд – амортизированно
-300 border border-slate-800">O(n log n)</code>
```

```
<p class="text-slate-300 text-sm mt-4">Все
lay последнего на пути); insert – BST
муна левой половины не будет правого сына –
а под «горячие» ключи, дешёвые split/merge.
во меняется даже при чтении (плохо для много
```

```
<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">
```

Код: rotate + splay + find (Скрыто)

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
```

```
узел: { key, left, right, parent }
```

```
def rotate_up(x): # x встаёт на место
 p = x.parent
 g = p.parent
 if p.left is x: # правый поворот
 p.left = x.right
 if x.right: x.right.parent = p
 x.right = p
 else: # левый (зеркально)
 p.right = x.left
 if x.left: x.left.parent = p
 x.left = p
 p.parent = x
 x.parent = g
 if g:
 if g.left is p: g.left = x
 else: g.right = x
```

```
def splay(x): # поднять x в корень
 while x.parent:
 p, g = x.parent, x.parent.parent
```

```

<div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-emerald-400
 <div class="bg-slate-900 p-4 rounded-lg mb-4
 <p class="text-lg text-emerald-300 italic
 <p class="text-xl font-mono text-white"
 , y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
</div>
<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 ≈ Аналогия 1: Вырезание прямоу
 </div>
 <p class="text-slate-300 text-sm mb
 з него маленький целевой прямоугольник в цел
 ый верхний угол отрезается дважды! Поэтому
 </div>
</div>
</div>
</div>
</section>`,
 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m
 <div class="flex items-center mb-6 flex-wrap gap-3"
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-r
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400
 <div class="bg-slate-900 p-4 rounded-lg mb-

```

```

 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2"
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb
еская сортировка – это расписание, которое
p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb
получить опыт, нужна работа". Алгоритм выявл
 </div>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SC
 type: "html",
 description: "Разбиение орграфа на макро-вершины. А
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord

```



```

 <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
</div>
<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb
этот мост, экономикой обоих кусков придет
 </div>
 </div>
</div>
</div>
</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает свя
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-rose-400 r
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-rose-300 italic r
 <p class="text-xl font-mono text-white">
тей > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">

```

```

 <p class="text-slate-300 text-sm mb-6">
 е сходится нечетное число линий, значит, из
 а везде должно быть четное число (зашел-вышел)
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "pathfinding-accel",
 title: "17. Графы. Ускорения поиска",
 type: "html",
 description: "",
 category: "Графы. Пути",
 content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 d-длина.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 rder border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под
 </div>
 <p class="text-slate-300 text-sm mb-6">
 ней земли. Но если начать копать одновременно
 в 2 РАЗА меньше работы (геометрически: площ

```

```

 </div>
</section>`,
 },
 {
 id: "mst-prima",
 title: "19. Графы. Остовное дерево. Прима",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },

```

```

 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 i]), который одновременно является её суффиксом.
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс и суффикс)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 идишь "АБРА". Если ты ошибешься при поиске
 концовка "АБРА" совпадает с началом "АБРА"
 моя перепроверок!</p>
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
vector<int> pi(s.length());

```

```

 <p class="text-slate-300 text-sm mb-6">
 в тексте кричит: "Эй! Начиная с этой буквы
 т "окно" [L, R] самой дальней найденной копии
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded-lg">
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description: "",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">

```

```
export interface Chapter {
 id: string;
 title: string;
 type: "html";
 content: string;
}

export const chapters: Chapter[] = [
 {
 id: "euler-path-vs-cycle",
 title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
 type: "html",
 content: `<section id="euler-path-vs-cycle" class="
<div class="flex items-center mb-6">
 <span class="bg-indigo-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Эйлер
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400
 <div class="bg-slate-900 p-4 rounded-lg mb-1
 <p class="text-lg text-blue-300 italic
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
 <p><strong class="text-white">Эйлер
 <div class="p-3 bg-slate-950 rounde
 <p class="text-sm text-blue-300
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </div>
 </div>
 </div>
 </div>
 </div>
```

```

 <details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-400
 order-slate-700/50">
 Душный режим: Формальное доказательство
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <p class="text-blue-400">Теорема Эйлера
 <p>
 Связный граф содержит эйлеров цикл $\Leftrightarrow$
 Эйлеров путь (не цикл) $\Rightarrow$ ровно 2 вершины с нечётной
 </p>
 <p class="text-rose-400">Почему Гамильтон?
 <p>
 Задача коммивояжёра (TSP) – это взрыв сложности.
 TSP – NP-трудная: полиномиальный алгоритм не найден.
 В общем, не парься. Просто запомни!
 </p>
 </div>
 </details>
 </div>
 </section>`,
 },
 {
 id: "bridges-code",
 title: "Мосты в графах: код + визуализация",
 type: "html",
 content: `<section id="bridges-code" class="mb-20 space-y-10">
 <div class="flex items-center mb-6">
 Код
 <h2 class="text-3xl font-bold text-white">Мосты
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">Задача коммивояжёра
 </div>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">

```

Граф — это кровеносная система. Ребра (артериалы) — это не мост, живём.

Если же перерезать единственную артерию, то наступит катастрофа.

Алгоритм DFS ищет такие «критические» ребра.

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">  
 <summary class="p-4 font-bold text-slate-400">  
 order-slate-700/50">

Полный код на C++ (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300 space-y-2">

<p>Классическая реализация — <code class="text-slate-400">DFS по рёбрам охотов.</p>

<div class="bg-slate-950 p-4 rounded-lg">

<pre class="text-xs font-mono text-slate-400">

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int n; // число вершин
```

```
vector<vector<int>>> adj; // список смежности
```

```
vector<bool> visited;
```

```
vector<int> tin, low;
```

```
int timer;
```

```
void dfs(int v, int p = -1) {
```

```
 visited[v] = true;
```

```
 tin[v] = low[v] = timer++; // устанавливаем время
```

```
 for (int to : adj[v]) {
```

```
 if (to == p) continue; // не идём назад
```

```
 if (visited[to]) {
```



```

 <h2 class="text-3xl font-bold text-white">Планар
</div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-3xl font-mono text-white">
 <div class="text-sm text-slate-400 space-y-2">
 <p>
 <p>
 <p>
 </div>
 </div>
```

```
 <p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше – граф гарантированно
 </p>
```

```
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border"
 <div class="absolute -top-3 left-4 bg-slate-900
order-green-500 shadow-md">
```

K5: Полный граф на 5 вершинах

```
</div>
<p class="text-slate-300 text-sm mb-4">
```

У K5: `class="text-white">V=5, E=10 > 9</code> – БАХ, непланарен!  
Все 5 вершин соединены со всеми. На K5  
Теорема Куратовского: K5 – эталон непланарности.  
Если внутри графа спрятан K5 – забудь про планарность.`

```
</p>
```

```
<h3 class="text-xl font-bold text-fuchsia-400">
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
 <p class="text-lg text-fuchsia-300 italic">
```

```
 <p class="text-xl font-mono text-white">
```

```
аут</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
```

```
 <div class="bg-slate-800 p-6 rounded-lg">
```

```
 <div class="absolute -top-3 left-4 l
```

```
 rder border-fuchsia-500 shadow-md">
```

Аналогия: карта на стене

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

Красим карту регионов: соседние

Регион = вершина, общая граница

Для плоской карты (планарный граф)

теорема о четырёх красках (Аппеля)

```
 </p>
```

```
</div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
```

```
 <div class="absolute -top-3 left-4 l
```

```
 er border-amber-500 shadow-md">
```

Что нужно знать на экзамен

```
 </div>
```

```
 <ul class="text-slate-300 text-sm sp
```

```
 <strong class="text-white">
```

вободный у соседей. Дал  $\leq \Delta + 1$  цветов ( $\Delta$  — макс

```
 <strong class="text-white">
```

тит 2 цветов, хоть он и непланарен.</li>

```
 <strong class="text-white">
```

$= n$ ,  $\chi(\text{цикла чётной длины}) = 2$ ,  $\chi(\text{цикла неч$

```
 <strong class="text-white">
```

й и не нечётный цикл,  $\chi(G) \leq \Delta$ .</li>

```

```

```
</div>
```

```
<p class="text-slate-300 text-sm">
 Важно: Это работает то
 епятся к точкам сочленения. То есть,
 ег - это тупик со степенью 1).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">
 <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
 Аналогия: Главный перекресток
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь город, где два района соединены
 её перекроют (удалят вершину) – районы будут
 хрупкость системы.
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border border-slate-800">
 <div class="absolute -top-3 left-4 bg-emerald-500 border border-emerald-500 shadow-md">
 Телеком: Центральный свитч
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У тебя несколько компьютеров в одной
 абелем (мостом). Сами свитчи – это точки со
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-800">
 <summary class="p-4 font-bold text-slate-400">
 Душный мод: Код и корень DFS (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">
```

```
 <strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
 <strong class="text-emerald-400">SCC
ависимые "конгломераты".
 Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S

 </div>
</div>
</section>`,
 },
];
```

---

ФАЙЛ 19/94: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 86 | РАЗМЕР: 1.5 КБ

---

```
import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets1to5 } from "../tickets/ticket1_5";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { chapters as extraChapters } from "../content_tem

/**
 * Канонический порядок тем (билеты 1–24) и «правильные
 * одна страница может раскрывать одну тему или несколь
 * поэтому названия выверяются по единому списку ниже.
 */
const CANON: Record<string, { num: number; title: string }> =
 "segment-trees": { num: 1, title: "1. Дерево отрезков
```

```
// graph-euler → euler-path-vs-cycle, pathfinding-accel
const stale = new Set([
 "graph-planar-colors",
 "graph-bridges",
 "graph-euler",
 "pathfinding-accel",
]);

// Дубликаты: берём последнюю (самую свежую) версию стр.
const dedup = new Map<string, Chapter>();
merged.forEach((c) => {
 if (c.id === "alg-map") return; // страница-заглушка
 if (stale.has(c.id)) return;
 dedup.set(c.id, c);
});

// Нормализуем названия по каноническому списку тем.
const withCanon = Array.from(dedup.values()).map((c) => {
 const canon = CANON[c.id];
 if (canon) return { ...c, title: canon.title, category: canon.category };
 return c;
});

// Сначала вводная страница, затем билеты 1–24 по порядку
const numbered = withCanon.filter((c) => CANON[c.id]);
numbered.sort((a, b) => CANON[a.id].num - CANON[b.id].num);
const extras = withCanon
 .filter((c) => !CANON[c.id])
 .sort((a, b) => a.title.localeCompare(b.title, "ru"));

export const chapters: Chapter[] = [...extras, ...numbered];
```

## Универсальное пособие • полный исходный код

```

* Строит самодостаточный python-скрипт: выполняет userSrc
* и возвращает JSON с шагами (строка + func + locals)
* последнего выражения (его вернёт runPythonAsync).
*/
```

```
export function buildDebugRunner(userSrc: string): string {
 const srcLiteral = JSON.stringify(userSrc); // JSON-сериализация
 return `import sys as __sys, json as __json
```

```
__src = ${srcLiteral}
__steps = []
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False
```

```
class __TraceLimit(Exception):
 pass
```

```
def __safe(v):
 try:
 r = repr(v)
 except Exception:
 r = "<...>"
 return r if len(r) <= 80 else r[:77] + "..."
```

```
def __snapshot(v, depth=0):
 """Небольшой JSON-снимок: UI использует его без парсинга
 if v is None or isinstance(v, (bool, int, str)):
 return v
 if isinstance(v, float):
 return v if v == v and v not in (float("inf"), float("-inf")) else None
 if depth >= 4:
 return __safe(v)
 limit = 80 if depth < 2 else 32
 if isinstance(v, (list, tuple)):
 return [__snapshot(x, depth + 1) for x in list(v)[:limit]]
 if isinstance(v, (set, frozenset)):
 return [__snapshot(x, depth + 1) for x in list(v)[:limit]]
```

```

 if len(__steps) >= __CAP:
 # Нельзя просто отключить trace: бесконечный цикл
 # тогда продолжит работать вечно. Мягко прерываем
 __was_truncated = True
 raise __TraceLimit()
Финальные локалы программы (после последней строки)
if event == "return" and frame.f_code.co_filename == __FILE__:
 loc, values = __capture(frame)
 __final.update(loc)
 __final_values.update(values)
return __tracer

__err = ""
try:
 __sys.settrace(__tracer)
 exec(compile(__src, "<user-code>", "exec"), {"__name__": "__main__"})
except __TraceLimit:
 pass
except Exception:
 import traceback as __tb
 __err = __tb.format_exc(limit=3)
finally:
 __sys.settrace(None)

__OUT = __json.dumps({
 "steps": __steps,
 "finalLocals": __final,
 "finalValues": __final_values,
 "error": __err,
 "truncated": __was_truncated,
})
__OUT = __OUT.replace("\n", "\\n")
}

/**
 * Какие переменные изменились на этом шаге относительно предыдущего
 * они подсвечиваются (как amber-подсветка изменений в коде)

```

-----  
import type { DemoKind } from "../components/hints/Mini"

/\*\*

\* Глоссарий «википедийных» подсказок.

\*

\* Любое слово из `labels`, встреченное в тексте главы,

\* в подчёркнутый термин: при наведении (или тапе на мо

\* с объяснением на пальцах и мини-визуализацией.

\*/

export interface GlossaryEntry {

id: string;

/\*\* Варианты написания в тексте (регистр не важен). \*/

labels: string[];

title: string;

/\*\* Объяснение «на пальцах» – 1–2 фразы. \*/

short: string;

/\*\* Строгая формулировка / что важно не забыть. \*/

formal?: string;

complexity?: string;

/\*\* Какая мини-анимация показывается в карточке. \*/

demo?: DemoKind;

/\*\* id полного симулятора (см. vizRegistry) – кнопка

simulator?: string;

}

export const GLOSSARY: GlossaryEntry[] = [

{

id: "bfs",

labels: ["BFS", "обход в ширину", "поиск в ширину",

title: "BFS – обход в ширину",

short:

"Волна от источника: сначала все соседи, потом со

formal:

"Очередь (FIFO). Кладём старт, достаём вершину –

у рёбер.",

complexity: "O(V + E)",

demo: "bfs",

simulator: "queue-bfs",



```
{
 id: "stack",
 labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
 title: "Стек (LIFO)",
 short: "Стопка тарелок: кладёшь сверху и берёшь сверху",
 formal: "push / pop / top за $O(1)$. На стеке живёт рекурсия",
 demo: "stack",
 simulator: "stack-dfs",
},
{
 id: "queue",
 labels: ["очередь", "очереди", "очередью", "FIFO"],
 title: "Очередь (FIFO)",
 short: "Очередь в столовой: кто первым встал, тот первым уходит",
 formal: "push_back / pop_front за $O(1)$. Основа BFS",
 demo: "queue",
 simulator: "queue-bfs",
},
{
 id: "dsu",
 labels: ["DSU", "CHM", "система непересекающихся множеств"],
 title: "DSU — система непересекающихся множеств",
 short:
 "«Кто твой староста?» Каждый элемент знает своего старосту",
 formal:
 "find со сжатием путей + union по рангу/размеру. Реализация на 10 строк",
 complexity: "почти $O(1)$ — обратная функция Аккермана",
 demo: "dsu",
 simulator: "mst-kruskal",
},
{
 id: "trie",
 labels: ["бор", "бора", "боре", "бору", "бором", "trie"],
 title: "Бор (trie, префиксное дерево)",
 short:
 "Дерево, где путь от корня по буквам и есть слово",
 formal: "Из корня по символам; в вершине хранится ф. слово",
 complexity: "поиск слова — $O(|\text{слова}|)$ ",
}
```

```

{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топосорт"],
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая с
 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, л
 complexity: "O(V + E)",
 demo: "toposort",
 simulator: "top-sort",
},
{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь
 ,
 formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "префикс"],
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набежало от н
 formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r]-P[l-1]
 complexity: "препроцессинг O(n), запрос O(1)",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
},
{
 id: "sparse-table",
 labels: [

```

```

 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
 formal: "Для не-корня: v — точка сочленения. Это ребро",
 demo: "articulation",
 simulator: "graph-articulation",
},
{
 id: "mst",
 labels: [
 "остовное дерево", "остовного дерева", "остовное",
 "минимальное остовное дерево", "MST",
],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная",
 formal: " $V-1$ ребро, никаких циклов. Краскал (жадно и",
 "но).",
 demo: "mst",
 simulator: "mst-kruskal",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизи",
 "амортизированное", "амортизированного", "амортизи",
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер",
 formal: "Метод потенциалов: дорогая операция оплачи",
 "ассив.",
 demo: "amortized",
},
{
 id: "np",
 labels: [
 "NP-полная", "NP-полнота", "NP-трудная", "NP-полн"
]
}
```

```

 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начался
 от кусок.",
 formal:
 " $\pi[i]$ — длина наибольшего собственного префикса s
 возвращая указатель по тексту.",
 complexity: "O(n + m)",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "z-функция"],
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начинающихся
 с каждой буквы.",
 formal:
 " $z[i]$ = длина наибольшего общего префикса s и $s[i..n]$ ",
 complexity: "O(n)",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "динамическое",
 "мемоизация", "мемоизацию", "подзадач", "подзадачи",
],
 title: "Динамическое программирование",
 short:
 "Решаем задачу через ответы на её меньшие копии и их комбинации.",
 formal:
 "Нужны: оптимальная подструктура + перекрывающиеся подзадачи.",
 demo: "dp",
 simulator: "dynamic-programming",
},

```

```
{
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова"],
 title: "Эйлеров путь и цикл",
 short:
 "Пройти по каждому ребру ровно один раз, не отрыв",
 formal:
 "Связный граф: эйлеров цикл все степени чётные;
 simulator: "euler-path-vs-cycle",
},
{
 id: "planarity",
 labels: ["планарный", "планарность", "планарного",
 title: "Планарность",
 short:
 "Граф планарен, если его можно нарисовать на листе",
 formal:
 " $V - E + F = 2$ для связного планарного графа. Кур",
 simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [
 "splay", "splay-дерево", "AVL", "AVL-дерево", "вс",
],
 title: "Splay и повороты",
 short:
 "Каждый раз, когда к узлу обратились, его «вытягив",
 formal:
 "Три случая: zig (родитель – корень), zig-zig (уз",
 "рацию.",
 demo: "amortized",
 simulator: "splay-tree",
},
{
 id: "adjacency",
```

```

 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд–Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл – это не «шаг», а «какие пересадки
 шить пересадку в k?",
 formal:
 "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Поряд
 ами.",
 complexity: "O(V³) времени, O(V²) памяти",
 demo: "floyd",
 simulator: "floyd",
},
{
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на кажд
 formal:
 "Множество посещённых + куча рёбер на границе. До
 усок.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-prim",
},
{
 id: "boruvka",
 labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
 title: "Алгоритм Борувки",
 short:
 "Все компоненты одновременно шлют «гонца» к ближай
 formal:
 "Фаза: для каждой компоненты ищем минимальное исх
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst-boruvka",
},

```

```

 "G = (V, E). Ребро бывает ориентированным (улица
demo: "graph",
simulator: "graph-dfs-bfs",
},
{
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "с"],
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам ну
 рядковым номером.",
 formal:
 "sort + unique + binary search: значение → индекс
 ,
 complexity: "O(n log n)",
 demo: "coord-compress",
},
{
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова де
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу
 ча. Случайность и держит его сбалансированным."
 formal:
 "Базис — split(t, x) (разрезать по ключу) и merge
 я через них. Ожидаемая высота O(log n).",
 complexity: "ожидаемо O(log n)",
 demo: "heap",
},
{
 id: "bst",
 labels: ["бинарное дерево поиска", "дерево поиска",
 title: "Бинарное дерево поиска",
 short:
 "Слева всё меньше корня, справа — всё больше. Пои
 formal:
 "Без балансировки дерево может вырождаться в списо

```

```

 "Суммы — префиксные суммы $O(1)$. Минимум без измен
demo: "prefix-sum",
simulator: "prefix-sums-2d",
},
{
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "с
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: г
 (Z-функция) и где встречаются слова словаря (бо
 formal:
 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функци
demo: "prefix-function",
simulator: "string-kmp",
},
{
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "поворо
 title: "Поворот (rotation) в дереве",
 short:
 "Локальная перевеска двух узлов: ребёнок встаёт н
 дившемуся месту. Порядок ключей при этом не лом
 formal:
 "Правый поворот вокруг p: $x = p.left; p.left = x.$
 g и Zig-Zag.",
 complexity: " $O(1)$ ",
 demo: "zig",
 simulator: "splay-rotations",
},
{
 id: "zig",
 labels: ["Zig", "Zag", "зиг"],
 title: "Zig — одиночный поворот",
 short:
 "Самый простой случай: родитель x уже сидит в корн
 канчивается, если глубина была нечётной.",
 formal: "Применяется ровно один раз за всю операцию

```



ФАЙЛ 22/94: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 275 | РАЗМ

```
import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Обзор: Кратчайшие пути и Графы",
 type: "html",
 category: "Графы",
 content: `<section id="intro-content" class="mb-12">
<div class="flex items-center mb-6">

 <h2 class="text-3xl font-bold text-white">Обзор
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <p class="text-slate-300 text-sm mb-4">Пред
 оезда). Алгоритмы поиска кратчайшего пути п
 <div class="mt-8 mb-4">
 <div id="slot-mnemonic-cards"></div>
 </div>
 </div>
</div>
</section>`
 },
 {
 id: "dijkstra",
 title: "Билет 14. Дейкстра",
 type: "html",
 category: "Графы",
 content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Дейкс
```

```

 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p>Алгоритм использует приоритетную очередь V). Гарантирует корректность только для неориентированных графов.
 </div>
 </details>

 <div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{
 id: "bellman-ford",
 title: "Билет 15. Форд-Беллман",
 type: "html",
 category: "Графы",
 content: `<section id="bellman-ford-content" class="flex items-center mb-6">
 <div class="bg-blue-600 text-white px-4 py-1 rounded">
 <h2 class="text-3xl font-bold text-white">Форд-Беллман
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-emerald-500 shadow-md">
 <h3 class="text-xl font-bold text-rose-400">Описание задачи
 </div>

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">Дана взвешенная ориентированная графа G(V, E) с n вершинами и m ребрами. Найти кратчайшие пути от заданной вершины s к каждой вершине t ∈ V.
 <p class="text-xl font-mono text-white">Гарантируется отсутствие отрицательных циклов.
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 </div>
 </div>
 </div>
 </div>

```

```

 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Автомагистрали
 </div>
 <p class="text-slate-300 text-sm mb-2">
 Сколько времени долететь из Москвы в Париж, если мы сидим в самолете?
 </p>
 <div id="slot-chapter-image-floyd" class="h-40">
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Время работы
 </div>
 <p class="text-slate-300 text-sm mb-2">
 Сколько времени потребуется на выполнение
 три вложенных цикла. Сложность $O(V^3)$. Если
 </p>
 </div>
 </div>

 <div id="slot-floyd-viz" class="mt-8"></div>
 </div>
</div>
</section>`
},
{
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `<section id="aho-corasick" class="mb-12">

```

```

 p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50"
 <summary class="p-4 font-bold text-slate-400 outline-1
 pen:border-slate-700/50">
 Внутренняя структура автомата и код BFS (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-4">
 <p>Каждая вершина имеет таблицу переходов <code>term_link</code>
 ссылку <code>term_link</code> (ближайшая терминальная вершина)
 <div class="bg-black/50 p-4 rounded-lg font-mono">
struct Node {
 map<char, int> go; // Предвычисленные переходы
 int link = 0; // Суффиксная ссылка: куда перейти по пре-
 int term_link = 0; // Терминальная ссылка: куда перейти по пре-
 bool is_terminal = false;
 vector<string> words;
};

// Расчет ссылок идет через BFS (level-order traversal)
// так как для вычисления link вершины v на глубине d,
// её предки на глубине d-1 уже должны иметь вычисленные ссылки
void buildLinks() {
 queue<int> q;
 // Корни и их дети
 for (auto const& [ch, child] : nodes[0].trie) {
 nodes[child].link = 0;
 q.push(child);
 }

 while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка – это переход по суффиксной ссылке
 nodes[child].link = nodes[nodes[v].link].go[ch];
 }
 }
}

```

```
 }
 };
}
```

ФАЙЛ 23/94: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 106 | РАЗМЕР: 1.1 КБ

```
export interface QuizQuestion {
 question: string;
 options: string[];
 correctIndex: number;
 explanation: string;
}

export const quizzes: Record<string, QuizQuestion[]> =
 "euler-path-vs-cycle": [
 {
 question: "В связном графе ровно 2 вершины с нечётной степенью",
 options: [
 "Есть эйлеров цикл (вернись домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)",
 "Ни пути, ни цикла нет — это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 — это эйлеров путь. Если нечётных больше 2 — эйлеров цикла нет, эйлеров путь тоже не будет. Если нечётных 0, то эйлеров цикл есть, эйлеров путь тоже есть." ,
 },
 {
 question: "Почему гамильтонов цикл — это NP-полная задача?",
 options: [
 "Потому что Гамильтон — болезнь, и лечить её тяжело",
 "Потому что для эйлерова цикла есть простой критерий",
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критерий нечётности степеней вершин. Для гамильтонова цикла — нет. Проверка наличия гамильтонова цикла — это NP-полная задача." ,
 },
],
 "hamiltonian-cycle": [
 {
 question: "Почему гамильтонов цикл — это NP-полная задача?",
 options: [
 "Потому что Гамильтон — болезнь, и лечить её тяжело",
 "Потому что для эйлерова цикла есть простой критерий",
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критерий нечётности степеней вершин. Для гамильтонова цикла — нет. Проверка наличия гамильтонова цикла — это NP-полная задача." ,
 },
],
};
```

```

 options: [
 "0(V^2) – ищем мосты в квадрате",
 "0($V + E$) – каждый элемент один раз",
 "0(2^V) – полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое
}
],
"planarity-euler-formula": [
{
 question: "Планарный граф имеет $V=6$, $E=12$. Возможн
 options: [
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство $E \leq 3V - 6$ ",
 "Да, если нет треугольных граней"
],
 correctIndex: 1,
 explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$ – на грани пр
 Так что такой граф непланарен."
 },
 {
 question: "Сколько граней у планарного графа с $V=7$
 options: [
 "3 грани",
 "5 граней",
 "10 граней"
],
 correctIndex: 1,
 explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$. Не
 },
 {
 question: "Почему K_5 непланарен?",
 options: [
 "Потому что $10 > 3*5 - 6 = 9$ – нарушение",
 "Потому что Эйлер был против",
 "Потому что 5 вершин – несчастливое число"
],
 },

```

```

/** Последний снимок хранится, чтобы поздно смонтированные компоненты могли получить последний снимок
const latestByChapter = new Map<string, VizStateEventDetail>()

/** Транслировать реальные переменные визуализациям текстов
export function emitVizState(
 chapterId: string,
 snapshot: Omit<VizStateEventDetail, "chapterId">
) {
 if (typeof window === "undefined") return;
 const detail: VizStateEventDetail = { chapterId, ...snapshot };
 latestByChapter.set(chapterId, detail);
 window.dispatchEvent(new window.CustomEvent<VizStateEventDetail>("viz-state", { detail }));
}

/** Вернуть демонстрацию в ручной режим при закрытии/перезагрузке
export function clearVizState(chapterId: string) {
 if (typeof window === "undefined") return;
 latestByChapter.delete(chapterId);
 window.dispatchEvent(new window.CustomEvent<VizStateEventDetail>("viz-clear", { chapterId }));
}

/** Текущий снимок компилятора для прямой отрисовки масива
export function useVizRuntime(): VizStateEventDetail | null {
 const chapterId = useContext(VizChapterContext);
 const [runtime, setRuntime] = useState<VizStateEventDetail>();
 return chapterId ? latestByChapter.get(chapterId) ?? null : null;
}

useEffect(() => {
 setRuntime(chapterId ? latestByChapter.get(chapterId) : null);
 if (!chapterId) return;
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizStateEventDetail>).detail;
 if (detail?.chapterId === chapterId) {
 setRuntime(detail.variables || detail.line !== null ? detail : null);
 }
 };
 window.addEventListener("viz-state", handler);
 return () => {
 window.removeEventListener("viz-state", handler);
 };
});

```

```

 const handler = (e: Event) => apply((e as CustomEvent) as any,
 window.addEventListener(CHANNEL, handler);
 return () => window.removeEventListener(CHANNEL, handler);
 }, [chapterId, currentStep, maxStep, goToStep, selectStep]
);
}

```

```

/** Утилиты без небезопасных cast-ов для визуализаторов
export const vizNumber = (value: DebugValue | undefined) =>
 typeof value === "number" && Number.isFinite(value) ? value

```

```

export const vizString = (value: DebugValue | undefined) =>
 typeof value === "string" ? value : null;

```

```

export const vizArray = (value: DebugValue | undefined) =>
 Array.isArray(value) ? value : null;

```

```

export const vizRecord = (value: DebugValue | undefined) =>
 value !== null && typeof value === "object" && !Array.isArray(value)
 ? (value as Record<string, DebugValue>)
 : null;

```

```

/* — Выбор демонстрации внутри страницы (вкладки ID / название)

```

```

const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

```

```

export interface VizDemoEventDetail {
 chapterId: string;
 /** base-запись страницы – demoId пустой; подписанные демо
 demoId: string;
}

```

```

/** Демонстрация сообщает, какая её вкладка сейчас открыта
export function emitVizDemo(chapterId: string, demoId: string) {
 if (typeof window === "undefined") return;
 latestDemoByChapter.set(chapterId, demoId);
 window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>(
 "VIZ_DEMO", { detail: { chapterId, demoId } }
));
}

```



```

/** Характерный диапазон/значение на демонстрации. */
range?: string;
}

export interface PageSync {
 /** Заголовок демонстрации на странице (если интерактив)
 vizTitle?: string;
 /** Чему соответствует один шаг визуализации (для тул
 stepNote?: string;
 /** Переменные синхронизации. Пусто – на странице нет
 variables: SyncVariable[];
 /** Полный референсный Python-код, который кнопка-гла
 code?: string;
}

/** Справочные значения, указанные внутри самих симулято
export const PAGE_SYNC: Record<string, PageSync> = {
 "segment-trees": {
 vizTitle: "Дерево отрезков: build → query → массовое
 stepNote: "Шаг = спуск по вершине v: целиком внутри
 lazy детям 2·v и 2·v+1.",
 code: `n = 6 # длина массива
A = [5, 8, 3, 12, 7, 2]
tree = [0] * (4 * n)
lazy = [0] * (4 * n) # lazy[v] – невыполненное «
v, l, r = 1, 0, n - 1 # вершина и её отрезок
m = (l + r) // 2 # граница детей 2v, 2v+1
ul, ur, add = 1, 3, 10 # массовое обновление: A[ul

--- тело алгоритма ---
def build(v, l, r):
 if l == r:
 tree[v] = A[l]
 return
 m = (l + r) // 2
 build(2 * v, l, m)
 build(2 * v + 1, m + 1, r)
 tree[v] = tree[2 * v] + tree[2 * v + 1]

```

```

i = 0 # кадр анимации: 0 ... (число
case = "zig-zig" # вкладка «Случай поворота»

print(f"splay({key}): подъем в корень")`,
 variables: [
 { name: "keys", role: "демо-дерево страницы (BST)",
 { name: "key", role: "ключ, который splay поднимает",
 },
 { name: "i", role: "номер кадра анимации", range: 0,
 { name: "case", role: "случай поворота для вкладки",
],
},
"splay-rotations": {
 vizTitle: "Splay: случаи поворота",
 stepNote: "Шаг = один кадр разбора случая: zig (1 кадр),
 code: `case = "zig-zig" # "zig" / "zig-zig"
i = 0 # номер кадра внутри случая
x, p, g = 20, 40, 60 # узел, родитель, дед – как

print(f"{case}: кадр {i}, крутим ребро {p}-{g if case == 'zig-zig' else 'zig'}`",
 variables: [
 { name: "case", role: "какой случай крутить: zig или z",
 { name: "i", role: "номер кадра в разборе случая",
 { name: "x, p, g", role: "тройка узлов: поднимаемый узел,
],
},
"sparse-table": {
 vizTitle: "Разрезанная таблица (1D и 2D)",
 stepNote: "Шаг = одна ячейка st[i][j] = min(st[i][j-1], st[i-1][j])",
 code: `a = [2, 3, 5, 62, 3, 21, 1, 4]
n, LOG = 8, 4
i, j = 0, 0
st = [] # сюда складываем строки
for row in a:
 st.append([row] + [None] * (LOG - 1))

for j in range(1, LOG):
 for i in range(n - (1 << j) + 1):

```

```

 r2 = fib(k - 2)
 memo[k] = r1 + r2
 return memo[k]

print("итог:", fib(n))`,
 variables: [
 { name: "n", role: "целевой номер числа Фибоначчи"},
 { name: "k", role: "аргумент текущего рекурсивного вызова"},
 { name: "memo", role: "кэш уже посчитанных значений"},
 { name: "r1, r2", role: "ответы двух рекурсивных вызовов"},
],
},
"heap-beam-search": {
 vizTitle: "Куча (priority queue)",
 stepNote: "Шаг = просеивание: вставленный элемент в кучу",
 code: `a = [7, 3, 5, 1] # куча-массивом
n = len(a)
i = n - 1 # всплываемый элемент

parent = (i - 1) // 2 # родитель
kids = (2 * i + 1, 2 * i + 2) # дети
print(f"i={i} parent={parent} kids={kids}")`,
 variables: [
 { name: "n", role: "текущее число элементов в куче"},
 { name: "i", role: "индекс элемента, который сейчас просеиваем"},
 { name: "(i-1)//2", role: "индекс родителя вершины"},
 { name: "2*i+1, 2*i+2", role: "индексы левого и правого ребенка"},
],
},
"stack-dfs": {
 vizTitle: "Стек и обход в глубину",
 stepNote: "Шаг = push новой тарелки на стек или pop тарелки",
 code: `st = [] # стек = рекурсионная таблица
st.append("A") # push — зашли в вершину
st.append("B")
top = st.pop() # pop — возврат на уровень выше
print("pop →", top, "стек:", st)`,

```

## Универсальное пособие • полный исходный код

```

visited, order, q = set(), [], ["A"] # ---- BFS ----
while q:
 v = q.pop(0)
 if v in visited:
 continue
 visited.add(v)
 order.append(v)
 q += [u for u in adj[v] if u not in visited]
print("BFS:", order)\,
variables: [
 { name: "adj", role: "список смежности демо-графа"},
 { name: "v", role: "текущая вершина обхода – подс"},
 { name: "stack", role: "стек DFS: последний зашёл"},
 { name: "q", role: "очередь BFS: первый зашёл – п"},
 { name: "visited / order", role: "множество посещ"},
],
},
"graph-components": {
 vizTitle: "Компоненты связности: BFS-разметка",
 stepNote: "Шаг = вершина извлечена из очереди или с",
 code: `from collections import deque

демо-граф страницы: три «острова» A-D, E-G, H-J
adj = {"A": ["B", "C", "D"], "B": ["A", "C"], "C": ["A",
 "E": ["F", "G"], "F": ["E", "G"], "G": ["E", "F"]
comp = {} # comp[v] – номер компонен
count = 0 # сколько раз запускали об
v, q = None, [] # текущая вершина и очередь

--- тело алгоритма ---
for s in adj:
 if s in comp:
 continue
 count += 1
 q = deque([s])
 comp[s] = count - 1
 while q:
 v = q.popleft()
```

```

 vizTitle: "Компоненты сильной связности (Косарайю)"
 stepNote: "Шаг = вершина из order (в обратном поряд
 code: `adj = {0: [1], 1: [2], 2: [0, 3], 3: [4], 4:
order, used = [], set())

def dfs1(v): # проход 1: порядок выхода
 used.add(v)
 for to in adj[v]:
 if to not in used:
 dfs1(to)
 order.append(v)

--- тело алгоритма ---
for v in adj:
 if v not in used:
 dfs1(v)

adjT = {v: [] for v in adj} # транспонированный граф
for v in adj:
 for to in adj[v]:
 adjT[to].append(v)

used = set()
comp = {}
num = 0
for v in reversed(order): # проход 2: по order задом
 if v in used:
 continue
 num += 1
 stack = [v]
 used.add(v)
 while stack:
 u = stack.pop()
 comp[u] = num
 for to in adjT[u]:
 if to not in used:
 used.add(to)
 stack.append(to)

```

```

v2, to2 = "C", "D"
is_bridge2 = low[to2] > tin[v2] # 4 > 3 → C-D тоже мост
print(f"{v}-{to}: мост? {is_bridge}; {v2}-{to2}: мост? {is_bridge2}")
 variables: [
 { name: "tin", role: "время захода DFS в вершину",
 { name: "low", role: "минимальный tin, достижимый из to",
 { name: "v, to", role: "ребро дерева DFS, для которого is_bridge",
 { name: "is_bridge", role: " $\text{low}[\text{to}] > \text{tin}[\text{v}] \Rightarrow \text{из } v \text{ в } to \text{ нет пути, не используя ребро } (v, to)$ "
],
},
"euler-path-vs-cycle": {
 vizTitle: "Эйлеров путь и цикл",
 stepNote: "Шаг = один клик по следующей вершине маршрута",
 code: `edges = [{"A","B"}, {"A","C"}, {"B","C"}, {"B","D"}, {"C","D"}]
path = ["A", "B", "C"] # маршрут: рисуется на графе
deg = {v: 0 for v in "ABCDE"}
for u, v in edges: # степени вершин
 deg[u] = deg.get(u, 0) + 1
 deg[v] = deg.get(v, 0) + 1

odd = []
for v in deg:
 if deg[v] % 2:
 odd.append(v)
print(deg, "нечётных:", len(odd))
0 нечётных → цикл · 2 → путь · иначе — нельзя`
 variables: [
 { name: "path", role: "маршрут списком меток вершин",
 { name: "deg", role: "степень вершины: чётная — всю вершину можно обойти",
],
},
"planarity-euler-formula": {
 vizTitle: "Планарность: K5/K3,3 + раскраска",
 stepNote: "Шаг = проверка формулы Эйлера для K5/K3,3",
 code: `V, E = 5, 10 # K5 (K3,3: V=6, E=9)
F = 2 - V + E # формула: V - E + F = 2
not planar = E > 3 * V - 6 # 10 > 9 → K5 непланарен

```

```

print(f"обрабатываем {u} (dist={du})")
for v, w in g[u]:
 if v in done:
 continue
 print(f"смотрим ребро {u}→{v} (w={w})")
 if du + w < dist.get(v, float("inf")):
 dist[v] = du + w
 prev[v] = u
 heapq.heappush(pq, (dist[v], v))
print("итог:", dist)`,
variables: [
 { name: "u", role: "вершина с минимальным dist среди",
 { name: "v", role: "сосед вершины u, до которого",
 { name: "w", role: "вес ребра (u, v)", range: "по",
 { name: "dist[v]", role: "лучшее известное расстояние"},
],
},
"bellman-ford": {
 vizTitle: "Форд–Беллман",
 stepNote: "Шаг = одна релаксация ребра (u, v, w) в графе",
 code: `# рёбра демо-графа страницы (включая коварные отрицательные)
E = [("S", "A", 4), ("S", "B", 5), ("B", "C", -2), ("A", "C", 3),
 ("C", "D", 3), ("C", "E", 4), ("D", "F", 2), ("E", "F", 1)]
n = 7 # вершин: S, A, B, C, D, E, F
dist = {"S": 0}
u, v, w = E[0] # шаг: очередное ребро (u, v, w)
i = 1 # номер итерации

--- тело алгоритма ---
for i in range(1, n): # n-1 прохода по всем рёбрам
 for u, v, w in E:
 if dist.get(u, float("inf")) + w < dist.get(v, float("inf")):
 dist[v] = dist[u] + w # релаксация
print(dist)
n-й проход улучшил бы dist ⇒ отрицательный цикл (на э
variables: [
 { name: "E", role: "список рёбер (u, v, w) – демо-граф",
 { name: "n", role: "число вершин: после n-1 итерации"},
]

```

## Универсальное пособие • полный исходный код

```

w_new = w + h[u] - h[v] # $w' = w + h[u] - h[v] = -$
print(f"w'({u}->{v}) = {w_new}")
остальные: B->C: 1 + (-2) - (-1) = 0 · C->A: 2 + (-1)
после Дейкстры ответ = d[v] - h[start] + h[v]`,
 variables: [
 { name: "h", role: "потенциалы: кратчайшие пути о",
 { name: "u, v, w", role: "ребро (u, v) перевзвешив",
 { name: "w_new", role: "новый вес — всегда ≥ 0 , к",
 { name: "d[v]", role: "расстояние Дейкстры в новы",
],
},
"mst-kruskal": {
 vizTitle: "Краскал и DSU",
 stepNote: "Шаг = следующее ребро из отсортированных",
 code: `# демо-граф страницы: 9 вершин A..I, 12 рёбер
E = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10,
 (12, "H", "I"), (13, "F", "I")]
E.sort() # сортируем по весу
parent = {v: v for v in "ABCDEFGH I"} # DSU: каждый сам
w, u, v = E[0] # шаг: очередное ребро
i = 0

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[x] # сжатие пути
 x = parent[x]
 return x

mst = []
for i, (w, u, v) in enumerate(E):
 if find(u) != find(v): # разные компоненты → берём
 parent[find(u)] = find(v)
 mst.append((w, u, v))
print("остов:", mst, "вес:", sum(w for w, _, _ in mst))
 variables: [

```



```

 { name: "taken", role: "вершины уже в дереве — «звезда»",
 { name: "mst", role: "рёбра остова, вес 51", gang: "красная"
 },
},
"mst-boruvka": {
 vizTitle: "Борувка",
 stepNote: "Шаг = каждая компонента одновременно выбирает минимальное ребро",
 code: `# демо-граф страницы: 9 вершин A..I, 12 рёбер
E = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "A", "C"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "A", "F"),
 (12, "H", "I"), (13, "F", "I")]
comp = {v: v for v in "ABCDEFGHI"} # каждая вершина — отдельная компонента
cheapest = {} # компонента → её самое дешёвое ребро
u, v, w = "A", "B", 2 # шаг: ребро-кандидат
phase = 1

--- тело алгоритма ---
def find(x):
 while comp[x] != x:
 comp[x] = comp[comp[x]]
 x = comp[x]
 return x

while len({find(v) for v in comp}) > 1:
 cheapest = {}
 for w, u, v in E: # каждая компонента выбирает минимальное ребро
 ru, rv = find(u), find(v)
 if ru == rv:
 continue
 if ru not in cheapest or cheapest[ru][0] > w:
 cheapest[ru] = (w, u, v)
 if rv not in cheapest or cheapest[rv][0] > w:
 cheapest[rv] = (w, u, v)
 for ru, (w, u, v) in cheapest.items(): # все слияния
 comp[find(u)] = find(v)
 print(f"фаза {phase}: осталось компонент", len({find(v) for v in comp}))
 phase += 1
print("готово за", phase - 1, "фазы (на демо — 2)"),`

```

```

stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]"
code: `pattern = "aba"
text = "abacaba"
s = pattern + "#" + text # демо-страница ищет pattern
i, l, r = 1, 0, 0 # правый Z-блок [l, r]
z = [None] * len(s) # пустые клетки: значения по умолчанию
z[0] = 0

--- тело алгоритма ---
for i in range(1, len(s)): # шаг: вычисляем z[i]
 if i <= r:
 z[i] = min(r - i + 1, z[i - l]) # из блока
 else:
 z[i] = 0 # свежая клетка: с нуля
 while i + z[i] < len(s) and s[z[i]] == s[i + z[i]]:
 z[i] += 1 # досчёт в лоб
 if i + z[i] - 1 > r:
 l, r = i, i + z[i] - 1
print(z)`
variables: [
 { name: "pattern", role: "образец: питон подставл." },
 { name: "text", role: "текст, в котором ищем", range: 1, len(s) },
 { name: "s", role: "склейка pattern + \"#\" + text", range: 1, len(s) },
 { name: "i", role: "позиция, для которой считаем z[i]", range: 1, len(s) },
 { name: "l, r", role: "правый Z-блок: отрезок, соотв. [l, r]", range: 1, len(s) },
 { name: "z", role: "z[i] – длина общего префикса s[i:] и pattern", range: 1, len(s) }
],
},
"aho-corasick": {
 vizTitle: "Ахо–Корасик",
 stepNote: "Шаг = построение бора по образцам, затем поиск",
 code: `s = "USHERS" # текст, в котором ищем
i, v = 0, 0 # позиция в тексте и вершина бора
trie = {"": {}} # бор: вершина → переходы
state, c = "", "u" # шаг: автомат идёт по символу
nxt = trie.get(state, {}).get(c, "") # go(state, c) с помощью бора
print(f"шаг {i + 1}: v={v}, символ {c!r} → {nxt!r}")`,

```

```

1, 42, 85, 14]]
r, c, k = 0, 0, 0
st2 = {} # пустой каркас виден до

--- тело алгоритма ---
База не отдельный уровень вычислений: блок 1x1 – это
Записываем её одним шагом, а не проигрываем 64 одноклеточных
Генератор остаётся отдельным фреймом и не конфликтует
st2.update(dict(({r, c, 0, 0}, A[r][c]) for r in range(9) for c in range(9)))

for k in range(1, 4):
 print(f"уровень k={k}: блоки {1 << k}x{1 << k}")
 size = 9 - (1 << k)
 half = 1 << (k - 1)
 for r in range(size):
 for c in range(size):
 st2[(r, c, k, k)] = min(st2[(r, c, k - 1, k - 1),
 (r + half, c, k - 1, k - 1)],
 (r, c + half, k - 1, k - 1)),
 (r + half, c + half, k - 1, k - 1))`
 variables: [
 { name: "r, c", role: "левый верхний угол блока, координаты"},
 { name: "k", role: "уровень: блоки 2^k x 2^k (шаг 2^k)"},
 { name: "st2[(r,c,k,k)]", role: "минимум квадратичной формы"}
],
},
"sparse-table#2d-query": {
 vizTitle: "Разреженная таблица 2D: запрос",
 stepNote: "Демонстрация кликабельна вручную: выбери",
 code: `# st2[(r, c, kx, ky)] из вкладки «построение»
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({kx}, {ky})")
variables: [
 { name: "r1, c1, r2, c2", role: "углы запрашиваемого"},
 { name: "kx, ky", role: "уровень самого крупного блока"}
],

```

```
* Визуализация читает из его трассы знакомые имена (i,
* но точно так же примет пользовательский код с теми же
*/
export function buildSyncTemplate(chapterId: string, chapterTitle: string) {
 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle} .`
 : `# «${chapterTitle}»\n# демо на странице нет – св

 if (!sync.code) return head;
 return `${head}\n${sync.code}\n`;
}

/**
* Инициализация скелета: только объявление демо-данных
* (без for/while/def/if/print). Именно это показывается
* умолчанию – дальше пользователь пишет свой вариант,
* заменяет редактор полным референсным кодом.
*/
function extractInit(code: string): string {
 const keep: string[] = [];
 for (const line of code.split("\n")) {
 const t = line.trim();
 if (t === "# --- тело алгоритма ---" || /^(for|while)/.test(t)) {
 keep.push(line);
 }
 }
 // убрать пустые строки с конца
 while (keep.length > 0 && keep[keep.length - 1].trim() === "") {
 keep.pop();
 }
 return keep.join("\n");
}

/**
* Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
* переменные демо (без всего кода алгоритма).
*/
export function buildInitTemplate(chapterId: string, chapterTitle: string) {
 const sync = getPageSync(chapterId, demoId);
```

```
<h3 class="text-xl font-bold text-blue-400">
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
 <p class="text-lg text-blue-300 italic">
```

```
 <p class="text-xl font-mono text-white">
```

```
 promise (lazy).</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
```

```
 <div class="bg-slate-800 p-6 rounded-lg">
```

```
 <div class="absolute -top-3 left-4">
```

```
 <div class="border-emerald-500 shadow-md">
```

```
 Аналогия 1: Корпоративная пре
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 ы рассылать 10 000 писем, он пишет одно пис
```

```
 счетах сотрудников только тогда, когда сот
```

```
 женное обновление).</p>
```

```
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
```

```
 <div class="absolute -top-3 left-4">
```

```
 border-rose-500 shadow-md">
```

```
 Аналогия 2: Матрешки-коробки
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 ше, и так далее. Если нам нужно покрасить п
```

```
 Внутри всё синее!" на большую коробку. Опус
```

```
 </div>
```

```
</div>
```

```
<details class="bg-slate-900/40 rounded-lg">
```

```
 <summary class="p-4 font-bold text-slate-300">
```

```
 -b border-slate-700/50">
```

```
 Строгое доказательство / Код (Скр
```

```
 </summary>
```

```
 <div class="p-5 text-sm text-slate-300">
```

```
 <pre class="bg-slate-950 p-4 rounded">
```

```

 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Школьные линейки
 </div>
 <p class="text-slate-300 text-sm mb
зок длины 7. Ты берешь две заготовленные за
ь отрезок длины 7 (перекрывая друг друга по
ничего складывать, просто пересекаем куски.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Ступени
 </div>
 <p class="text-slate-300 text-sm mb
ужно покрыть отрезок, мы берем две самые бо.
 </div>
 </div>
 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
int k = log2(R - L + 1);
int minimum = min(st[L][k], st[R - (1 << k) + 1][k]);</pre>
 </div>
 </details>
</div>
</div>
</section>`
},
{
 id: "treap",
 title: "3. Декартово дерево (Treap)",
 type: "html",
 description: "Дерево поиска по ключу и Куча по приор

```

```

 </summary>
 <div class="p-5 text-sm text-slate-300" style="border: 1px solid #ccc; border-radius: 10px; padding: 10px; margin-top: 10px;">
 <pre class="bg-slate-950 p-4 rounded" style="margin: 0;">
pair<Node*, Node*> split(Node* t, int x) {
 if (!t) return {nullptr, nullptr};
 if (t->val <= x) {
 auto [L, R] = split(t->right, x);
 t->right = L;
 return {t, R};
 } else {
 auto [L, R] = split(t->left, x);
 t->left = R;
 return {L, t};
 }
}</pre>
 </div>
 </details>
 </div>
</div>
</section>`
 },
 {
 id: "splay-tree",
 title: "4. Splay-дерево",
 type: "html",
 description: "Дерево поиска, которое самобалансируется",
 category: "Продвинутые структуры",
 content: `
<section id="splay-tree" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 4
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">Splay-дерево</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">Описание</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Дерево поиска, которое самобалансируется, то есть поддерживает сложность O(log n) для всех операций. Оно основано на операции splay, которая перемещает выбранный узел в корень дерева, что приводит к перестройке всего дерева. Это обеспечивает балансировку дерева и предотвращает деградацию в линейное время.
 </p>
 <p class="text-lg text-blue-300 italic">В отличие от AVL и B-деревьев, splay-дерево не требует хранения информации о балансе (например, фактора баланса или высоты) в узлах. Вместо этого оно использует локальные операции вращения (left-rotate, right-rotate) для перестройки дерева.
 </p>
 <p class="text-lg text-blue-300 italic">Операция splay выполняется за O(log n) времени в среднем, хотя в худшем случае может достигать O(n). Однако, благодаря эффекту асимптотического摊还 (amortized), среднее время на операцию остается O(log n).
 </p>
 <p class="text-lg text-blue-300 italic">Splay-дерево часто используется в алгоритмах, требующих частых операций поиска, вставки и удаления, особенно когда данные имеют локальную кластерность (например, в кэше, в системах управления памятью).
 </p>
 </div>
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">Пример кода</h3>
 <pre class="text-sm text-slate-300">
// Функция splay перемещает узел t в корень дерева.
// Она использует вращения: сначала вращение вокруг родителя,
// затем вокруг дедушки.
void splay(Node* t) {
 if (!t) return;
 Node* p = t->parent;
 if (!p) return; // t уже корень
 Node* g = p->parent;
 if (!g) return; // p уже корень
 // Вращение вокруг родителя
 if (p->left == t) {
 p->left = t->right;
 t->right = p;
 } else {
 p->right = t->left;
 t->left = p;
 }
 t->parent = g;
 // Вращение вокруг дедушки
 if (g->left == p) {
 g->left = t;
 } else {
 g->right = t;
 }
 t->parent = g;
}

// Вставка нового узла
Node* insert(Node* root, int val) {
 if (!root) return new Node(val);
 Node* cur = root;
 while (true) {
 if (cur->val == val) return cur;
 if (cur->val < val) {
 if (!cur->right) cur->right = new Node(val);
 else cur = cur->right;
 } else {
 if (!cur->left) cur->left = new Node(val);
 else cur = cur->left;
 }
 }
 return root;
}

// Поиск узла
Node* find(Node* root, int val) {
 if (!root) return nullptr;
 Node* cur = root;
 while (cur) {
 if (cur->val == val) return cur;
 if (cur->val < val) cur = cur->right;
 else cur = cur->left;
 }
 return nullptr;
}

// Удаление узла
Node* erase(Node* root, int val) {
 if (!root) return nullptr;
 Node* cur = root;
 while (true) {
 if (cur->val == val) {
 // Удаляем узел, ссылаясь на его детей
 Node* left = cur->left;
 Node* right = cur->right;
 delete cur;
 // Соединяем детей
 if (!left) return right;
 if (!right) return left;
 // Вставляем в дерево
 Node* newRoot = insert(left, val);
 if (newRoot->right) newRoot->right = right;
 return newRoot;
 }
 if (cur->val < val) cur = cur->right;
 else cur = cur->left;
 }
 return root;
}

// Вывод дерева (префиксная запись)
void print(Node* root) {
 if (!root) return;
 cout << root->val << " ";
 print(root->left);
 print(root->right);
}

// Пример использования
int main() {
 Node* root = nullptr;
 root = insert(root, 10);
 root = insert(root, 20);
 root = insert(root, 30);
 root = insert(root, 40);
 root = insert(root, 50);
 root = insert(root, 60);
 root = insert(root, 70);
 root = insert(root, 80);
 root = insert(root, 90);
 root = insert(root, 100);

 Node* found = find(root, 50);
 if (found) cout << "Найдено: " << found->val << endl;

 root = erase(root, 50);
 cout << "Удалено: " << 50 << endl;

 print(root);
 return 0;
}
 </pre>
 </div>
 </div>
`
 }
}

```

```

x.left = p;
}
}</pre>

</div>
</details>

<div class="bg-indigo-950/60 p-6 rounded-xl">
 <h4 class="text-lg font-bold text-indigo">

 <div class="space-y-4 text-sm text-slate">
 <div>
 <strong class="text-white block">
 корень?
 <p>Оно поднимет узел, на котором
 ска. За счет сдвига этого листа в корень, д
 /p>
 </div>

 <div>
 <strong class="text-white block">
 <p>Если агент постоянно переключ
 находящимися на разных концах дерева, его
 ащая дерево в вытянутый бамбук для другого.
 . Постоянное свинг-переключение превратит пр
 </div>

 <div>
 <strong class="text-white block">
 <p>Хотя один поиск может стоять
 емах обладают прекрасным свойством: они не
 по пути. "Палочное" дерево прессуется в сб
 осов.</p>
 </div>
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`

```



---

ФАЙЛ 27/94: src/data/tickets/ticket14\_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 248 | РАЗМ

---

```
import { Chapter } from '../..types';
```

```
export const tickets14to20: Chapter[] = [
```

```
{
```

```
 id: "dijkstra",
```

```
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра
```

```
 type: "html",
```

```
 category: "Графы. Пути",
```

```
 content: `
```

```
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
```

```
 <div class="flex items-center mb-6 flex-wrap gap-3">
```

```

```

```
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
```

```
 </div>
```

```
 <div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
```

```
 <h3 class="text-xl font-bold text-emerald-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
 <p class="text-lg text-emerald-300 italic">
```

```
 <p class="text-xl font-mono text-white">
```

```
 </div>
```

```
 <div class="grid grid-cols-1 xl:grid-cols-2">
```

```
 <!-- Аналогия 1 -->
```

```
 <div class="bg-slate-800 p-6 rounded-lg">
```

```
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
```

```
 Аналогия 1: Позитивное планирование
```

```
 </div>
```

```
 <p class="text-slate-300 text-sm mb-4">
```

```
 (нельзя поехать и заработать). Он всегда в
```

```
 </div>
```

```
 <!-- Аналогия 2 -->
```

```

 <p class="text-slate-300 text-sm mb-4">
 еряет ВСЕ возможные дороги V-1 раз подряд.
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb-4">
 опали во временную петлю.</p>
 </div>
 </div>
 <div id="slot-bellman-viz" class="mt-8"></div>
 </div>
 </div>
 </section>`
 },
 {
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg border">
 <div class="absolute -top-3 left-4 bg-slate-900 p-2 rounded">

```

```

 <p class="text-xl font-mono text-white">
 </div>
 <div class="grid grid-cols-1 gap-6 mb-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
er border-amber-500 shadow-md">
 ⚡ Суть перевзвешивания
 </div>
 <p class="text-slate-300 text-sm mb
 Дейкстра (Билет 14) быстрая, но
 ает очень медленно. Мы хотим сделать Джон
 е пути остались теми же.
 </p>
 <ol class="text-sm text-slate-300 s
 Добавляем фиктивную верш
 Запускаем от неё медленно
 «потенциалы» <code class="text-pink-4
 Меняем старые веса: новое р
 Магия! Все веса тепер
 ренные потенциалы сокращаются).
 Теперь смело запускаем б

 </div>
 </div>
 <div id="slot-johnson-viz" class="mt-8"></d
</div>
</div>
</section>`
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r

```

```

<h3 class="text-xl font-bold text-emerald-400">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 которое торчит из посещенных в непосещенных
</div>
<div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </p>
 </div>
</div>
</div>
</div>
</section>`
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">

```

```

</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-blue-300 italic"
 <p class="text-xl font-mono text-white"
сом, оканчивающимся в позиции i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Поиск без отка
 </div>
 <p class="text-slate-300 text-sm mb"
 Мы идём по строке слева направо. К
строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слова
х</code>. Тупой алгоритм начнет искать заново
</code>, а это начало нашего слова! Не надо
 </p>
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: LLM стриминг
 </div>
 <p class="text-slate-300 text-sm mb"
 Для LLM, которая стримит токены
каждом шаге. Если мы частично совпали с зап
акого места этого слова мы можем продолжить
 </p>
 </div>

```

```

 pi[i] = j;
 }</pre>
 </div>
 </details>
</div>
</div>
</section>`
 },
 {
 id: "string-z-func",
 title: "22. Z-функция — Сдвиг и наложение",
 type: "html",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ca 0) и её суффиксом, начинающимся с i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Линейка-шаблон
 </div>
 <p class="text-slate-300 text-sm mb">
 Тут мы берём всю строку и «прик
 я начну читать строку отсюда, насколько дли

```

```

 <details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded

int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
 </details>
 </div>
</div>
</section>`
 },
 {
 id: "aho-corasick",
 title: "23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 round
 <h2 class="text-2xl sm:text-3xl font-bold text-white
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-blue-400 mb-4">1

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0с
 <p class="text-lg font-mono text-white">Бор (Tr
```

```

 return t[v].link;
 }

 int go(int v, char c) {
 if (t[v].go[c] == -1) {
 if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
 else t[v].go[c] = v == 0 ? 0 : go(get_link(v), c);
 }
 return t[v].go[c];
 }
}

</div>
</details>
</div>
</section>`
},
{
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-purple-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-purple-300 italic">
 <p class="text-xl font-mono text-white">
 Сведение (Reduction) $A \leq_p B$: вход задачи A и
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4">

```



```

category: "Графы. База",
content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-300">
 </div>
 <div class="space-y-8">
 {/* База представлений */}
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-slate-300">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-slate-300 italic">
 <p class="text-xl font-mono text-white">
 (V + E).</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия: Матрица = Таблица
 </div>
 <p class="text-slate-300 text-sm mb-4">
 глупо, но проверка "знает ли Вася Петю" мгно
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb-4">
 т. Оптимально для почти всех задач.</p>
 </div>
 </div>
 </div>
 </div>
 {/* DFS vs BFS */}
 <div class="bg-slate-700/50 p-6 rounded-xl border

```

<li>Двунаправленный поиск для у

</ul>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg l

<summary class="p-4 font-bold text-slate

-b border-slate-700/50">

Код обходов C++ (Основа)

</summary>

<div class="grid grid-cols-1 md:grid-co

<pre class="bg-slate-950 p-3 rounde

// DFS (Рекурсия)

vector<bool> vis;

vector<vector<int>>> adj;

void dfs(int v) {

vis[v] = true;

for (int u : adj[v]) {

if (!vis[u]) dfs(u);

}

}</pre>

<pre class="bg-slate-950 p-3 rounde

// BFS (Очередь)

queue<int> q;

q.push(start\_node);

vis[start\_node] = true;

while(!q.empty()) {

int v = q.front(); q.pop();

for (int u : adj[v]) {

if (!vis[u]) {

vis[u] = true;

q.push(u);

}

}

}</pre>

</div>

```

</section>`,
 },
 {
 id: "graph-components",
 title: "8. Графы. Компоненты связности",
 type: "html",
 description: "Нахождение кусков графа",
 category: "Графы. База",
 content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">8. Графы. Компоненты связности
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">8.1. Поиск компонент связности
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Представьте себе карту мира, где страны — это узлы, а океаны — это ребра. Если вы хотите найти все страны, которые можно посетить, не выходя из страны, то вы ищете компоненту связности.
 <p class="text-xl font-mono text-white">Вот как это выглядит в коде:
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Острова
 </div>
 <p class="text-slate-300 text-sm mb-4">Если вы видите на карте только зеленые (исследованные) страны, то вы можете сказать, что все страны, которые можно посетить, — это те, которые вы уже посетили.
 </div>
 <div class="bg-slate-800 p-6 rounded-lg">
 <p class="text-slate-300 text-sm mb-4">Если вы видите на карте только серые (неисследованные) страны, то вы можете сказать, что все страны, которые можно посетить, — это те, которые вы еще не посетили.
 </div>
 </div>
 <p class="text-slate-300 text-sm mb-4">Вот как это выглядит в коде:
 </div>
 </div>
</section>`,
 },
 {

```

```

 <details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300"
 -b border-slate-700/50">
 Душный мод: Код на C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300"
 <p class="text-indigo-400 font-bold"
 <div class="bg-slate-950 p-4 rounded"
 <pre class="text-xs font-mono text-slate-300"
vector<int> adj[MAXN];
bool visited[MAXN];
vector<int> ans;

void dfs(int v) {
 visited[v] = true;
 for (int to : adj[v]) {
 if (!visited[to]) {
 dfs(to);
 }
 }
 ans.push_back(v); // Добавляем в момент ВЫХОДА (pos
}

void topological_sort(int n) {
 for (int i = 0; i < n; ++i) {
 if (!visited[i]) dfs(i);
 }
 reverse(ans.begin(), ans.end()); // Разворачиваем с
}</pre>
 </div>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {

```

## Аналогия: Интриги в офисе

```
</div>
<p class="text-slate-300 text-sm mb-4">
 Сильная связность – это "клуб" сотрудников, в котором все сидит
 ни в одной SCC. Добавить курьера Гошу, который может выйти из клуба
 йти наружу (к директору), но обратно в клуб не может.
</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 border border-rose-500 shadow-md">
 Связь с Билетом 12
 </div>
 <p class="text-slate-300 text-sm mb-4">
 SCC (Билет 10) склеивает все вершины, которые можно достучаться

 Точки сочленения (Билет 12) – это вершины, удаление которых
 ость монолита. Вырвешь точку сочленения – получишь две SCC.
 </p>
</div>
</div>
```

```
<details class="bg-slate-900/40 rounded-lg">
 <summary class="p-4 font-bold text-slate-300">
 Душный мод: Код Алгоритм Косарайю
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <p class="text-emerald-400 font-bold">Алгоритм Косарайю</p>
 <ol class="list-decimal list-inside">
 Запускаем DFS на исходном графе, чтобы найти SCC.

 Разворачиваем этот список (то есть делаем reverse-topological sort).

 Переворачиваем все ребра в графе.

 Идем по <code>order</code>: для каждой вершины v в порядке убывания
 !

```

```

</section>`,
 },
 {
 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых убивает связность",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">12. Графы. Точки сочленения
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-rose-400">Точки сочленения
 </div>

 <div class="bg-slate-900 p-4 rounded-lg mb-6">
 <p class="text-lg text-rose-300 italic">Вершины, удаление которых увеличивает число компонент связности, называются точками сочленения.
 <div class="text-left font-mono text-sm">
 <p><strong class="text-white">Точка сочленения - это вершина, удаление которой увеличивает число компонент связности.
 <div class="p-3 bg-slate-950 rounded">
 Определение
 <p class="text-xl font-bold text-rose-400">Вершина, удаление которой увеличивает число компонент связности, называется точкой сочленения.
 <p class="text-xs text-slate-400">Пример: вершина 2 на рисунке ниже является точкой сочленения.
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Важно: Это работает только для вершин, имеющих степень больше 1. Вершины с степенью 1 не являются точками сочленения. То есть, точка сочленения - это тупик со степенью 1).
 </p>
 </div>
`
 }
]

```

```

<div class="bg-slate-950 p-4 rounded-lg" style="background-color: #1a2b3c; color: white; padding: 10px;">
 <pre class="text-xs font-mono text-white" style="font-family: monospace; font-size: 0.8em;">
void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++;
 int children = 0;

 for (int to : adj[v]) {
 if (to == p) continue;
 if (visited[to]) {
 low[v] = min(low[v], tin[to]);
 } else {
 dfs(to, v);
 low[v] = min(low[v], low[to]);
 if (low[to] >= tin[v] && p != -1)
 IS_CUTPOINT(v);
 ++children;
 }
 }
 if (p == -1 && children > 1)
 IS_CUTPOINT(v);
}
 </pre>
 </div>
</div>
</details>

```

```

<div class="bg-indigo-950/60 p-6 rounded-xl border" style="background-color: #2c3e50; color: white; padding: 10px; border: 1px solid #34495e;">
 <h4 class="text-lg font-bold text-indigo-400" style="margin: 0;">
 <p class="text-slate-300 text-sm mb-4" style="margin: 0;">
 Это глубокий дуальный мост между ориентированными графами и неориентированными графами.
 </p>
 <ul class="list-disc list-inside text-sm text-white" style="margin: 0;">
 <strong class="text-rose-400">Точкиarticulation points и показывают физическую уязвимость связности графа.
 <strong class="text-emerald-400">SCCs или "сильно связные компоненты" являются фундаментальными зависимыми "конгломератами".
 Как точка сочленения рушит связность? Косарайю, свернув каждую SCC в одну супер-вершину, мы можем увидеть, как удаление одной вершины может разрушить всю структуру.


```

```
 </div>
 </div>
</section>`,
 },
];
```

---

## РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

---

---

ФАЙЛ 30/94: src/components/ArticulationPointsViz.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizNumber } from '../data/vizS
import { Play, Pause, RotateCcw, Info } from "lucide-re
```

```
interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}
```

```
interface Edge {
 u: number;
 v: number;
}
```

```
const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
```



```

 const v = vizNumber(vars.v);
 if (v === null) return null;
 return steps.map((step, index) => ({ step, index }));
 });
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const newSteps: StepInfo[] = [];
 let timer = 0;
 const visited = new Set<number>();
 const ap = new Set<number>();
 const currentTin = Array(nodes.length).fill(-1);
 const currentLow = Array(nodes.length).fill(-1);

 const recordStep = (desc: string, u: number | null) => {
 newSteps.push({
 desc,
 visited: new Set(visited),
 ap: new Set(ap),
 currentNode: u,
 tin: [...currentTin],
 low: [...currentLow],
 });
 };

 recordStep("Начало DFS (Тарьян) для поиска точек со

 const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${c

 for (const to of adj[v]) {
 if (to === p) continue;

 if (visited.has(to)) {

```

```

 for (let i = 0; i < nodes.length; i++) {
 if (!visited.has(i)) {
 dfs(i);
 }
 }

 recordStep("Алгоритм завершен. Найдены все точки со
 setSteps(newSteps);
 }, []);

const currentStep = steps[currentStepIndex] || {
 desc: "",
 visited: new Set(),
 ap: new Set(),
 currentNode: null,
 tin: [],
 low: [],
};

useEffect(() => {
 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length -
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
}, [isPlaying, currentStepIndex, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
};

```

```

const v = nodes.find((n) => n.id === edge.v);

const isBridge =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);

return (
 <line
 key={"edge-" + i}
 x1={u.x}
 y1={u.y}
 x2={v.x}
 y2={v.y}
 stroke={isBridge ? "#f43f5e" : "#475568"}
 strokeWidth={isBridge ? "4" : "2"}
 strokeDasharray={isBridge ? "8,4" : "0"}
 />
);
}}}

{/* Nodes */}
{nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id;
 const isVisited = currentStep.visited.has(node.id);
 const isAp = currentStep.ap.has(node.id);

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"
 : "#1e293b";
 const stroke = isCurrent
 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited

```

```

 {currentStep.tin[node.id] >= 0
 ? currentStep.tin[node.id]
 : "-"}
 </text>
 <text
 x={node.x}
 y={node.y + r + 20}
 textAnchor="middle"
 fill="#94a3b8"
 fontSize="10px"
 >
 low:{" "}
 {currentStep.low[node.id] >= 0
 ? currentStep.low[node.id]
 : "-"}
 </text>
 </>
 })
</g>
);
}}
</svg>
</div>

```

```

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-4
 Статус алгоритма
 </h4>

```

```

<div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-
 {currentStep.desc}
 </p>
</div>

```

```

<div className="space-y-4 flex-1 overflow-y-a
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 fon

```

```
 <div className="mt-4 shrink-0">
 <div className="w-full bg-slate-800 h-2 rounded">
 <div
 className="bg-rose-500 h-full transition"
 style={{
 width: `${(currentStepIndex / Math.max(
 steps.length, 1
)) * 100}%`
 }}
 />
 </div>
 </div>
 </div>
</div>
</div>
);
};
```

---

ФАЙЛ 31/94: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord, vizString } from 'viz';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 iteration: number;
 checkingEdge: { u: string; v: string; w: number } | null;
 distances: { [key: string]: number };
 previous: { [key: string]: string | null };
 log: string;
 hasNegativeCycle?: boolean;
 activeCodeLine: number;
}

export default function BellmanFordViz() {
```

```

nodes.forEach(n => (list[n.id] = []));
ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
return list;
};
const adjList = getAdjList(edges);

const codeLines = [
 { line: 1, text: 'function bellman_ford(graph, start'
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0'
 { line: 3, text: ' for i from 1 to V - 1: // |V| - 1 iterations'
 { line: 4, text: ' for (u, v, weight) in E: // E = edges'
 { line: 5, text: ' if dist[u] + weight < dist[v]'
 { line: 6, text: ' dist[v] = dist[u] + weight'
 { line: 7, text: ' for (u, v, weight) in E: // Проверка на отрицательный цикл'
 { line: 8, text: ' if dist[u] + weight < dist[v]'
 { line: 9, text: ' return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ'"
 { line: 10, text: ' return dist' },
];

useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 999999999 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фура заведена. Начинаем с Базы (S). Расстояние до А: 999999999'
 activeCodeLine: 2,
 });

 const vCount = nodes.length;
 for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

```

```

 });
 break;
 }
}

if (!cycleFound) {
 simulationSteps.push({
 iteration: vCount, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev }
 log: ' Проверка завершена. Ни одно ребро не
 activeCodeLine: 10,
 });
}
}

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [hasCycle]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length -
 timer = setTimeout(() => setCurrentStepIndex((p)
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Зарплата...</div>;
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveW = vizNumber(vars?.w);
const liveDist = vizRecord(vars?.dist);

```





```

 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">
 </div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950">
 <h4 className="text-lg font-bold text-emerald-400">
 <div className="space-y-2 text-sm font-mono">
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap gap-2">

 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingEdge(u, edge.v);
 return (
 <span key={idx} className={`px-2 py-1 ${
 edge.w < 0 ? 'bg-rose-900/80 text-rose-300' : 'bg-emerald-900/80 text-emerald-300'
 }`}>
 {edge.v} ({edge.w})

)
 })}
 </div>
 </div>
 })}
 </div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div>
 <h4 className="text-lg font-bold text-rose-400">
 <div className="grid grid-cols-4 gap-2 text-sm">
 {nodes.map(n => {
 const d = currentStep.distances[n.id];
 const isTgt = currentStep.checkingEdge(n.id, currentStep.targetId);
 return (

```

```

import { useState, useEffect, useCallback } from 'react'
import { Play, Pause, RotateCcw, StepForward } from 'lu

const NODES = [
 { id: 0, label: '0', x: 300, y: 50 },
 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];

const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];

const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
};

interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
 currentNode: number | null;
 logs: string;
}

const generateBfsSteps = (): Step[] => {
 const steps: Step[] = [];
 const queue: number[] = [];
 const visited = new Set<number>();

 // Init
 steps.push({
```

```

 if (!visited.has(n)) {
 steps.push({
 activeLine: 7,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не
 });

 visited.add(n);
 queue.push(n);

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добавляем в очередь.
 });
 }
 } else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Соседей нет или все уже посещены.`
 });
 }
}

steps.push({
 activeLine: 12,
 queue: [],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
});

```



```

 {isCurrent && {
 <circle cx={n.x} cy={n.y} r="30" fill={fill} />
 }}
 <circle
 cx={n.x} cy={n.y}
 r="20"
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-colors duration-300"
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold text-sm"
 >
 {n.label}
 </text>
 </g>
);
}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
 <div className="bg-slate-900 border border-slate-800 p-4">
 <div className="bg-slate-800 text-slate-400 p-4">
 bfs(graph, start)
 var {step}
 </div>
 <div className="p-4">
 {[
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); vi

```

```
 ms-center justify-center">
 "{step.logs}"
 </div>
 </div>
</div>
</div>
</div>
});
}
```

---

ФАЙЛ 33/94: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string; }> = {
 dijkstra: {
 src: dijkstraImg,
 alt: 'Добрый Дейкстра',
 caption: 'Добрый — только позитив!',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',
 },
 'bellman-ford': {
 src: bellmanImg,
 alt: 'Фура по болоту',
 caption: 'Тяжёлая, но ей пофиг на ямы!',
 borderColor: 'border-rose-500/30',
 captionColor: 'text-rose-400',
 },
 floyd: {
 src: floydImg,
 alt: 'Все ко всем Флойд',
 },
}
```

```

/**
 * Переходы «предыдущая / следующая тема» — как на Code
 * Компактный вариант живёт в шапке главы, крупный — по
 */
export const ChapterNav: React.FC<Props> = ({ prev, next,
 if (compact) {
 return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 <ChevronLeft className="w-4 h-4" />
 Назад
 </button>

 {index + 1} / {total}

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 Вперёд
 <ChevronRight className="w-4 h-4" />
 </button>
 </div>
);
 }

 return (
 <nav className="grid grid-cols-1 sm:grid-cols-2 gap-2">

```





```

const handlePointerOut = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (el) scheduleHide();
};

const handleClick = (e: React.MouseEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (!el) return;
 e.preventDefault();
 if (anchor && anchor.termId === el.dataset.termId)
 else open(el);
};

const handleFocus = (e: React.FocusEvent) => {
 const el = (e.target as HTMLElement).closest<HTMLElement>
 if (el) open(el);
};

const viz = getViz(chapter.id);

return (
 <>
 <article className="chapter-body">
 {/* панель загрузки: сохранить именно эту страницу */}
 <div className="flex flex-wrap items-center gap-2">
 <button
 onClick={saveHtml}
 disabled={saving === "work"}
 title="Сохранить эту тему одним автономным файлом"
 className="flex items-center gap-1.5 text-xs text-white border-1px border-emerald-500"
 style={300 hover:text-white hover:border-emerald-500}
 >
 {saving === "work" ? (
 <Loader2 className="w-3.5 h-3.5 animate-spin" />
) : saving === "done" ? (
 <Check className="w-3.5 h-3.5 text-emerald-500" />
) : null}
 </div>
 </article>
 </>
)

```

```

 <h3 className="flex items-center gap-2 text-indigo-400">
 <Sparkles className="w-4 h-4 text-indigo-400">
 Демонстрация: {viz.title}
 </h3>
 <div className="flex items-center gap-2">
 <button
 onClick={() => onOpenSimulator(chapter)}
 className="flex items-center gap-1.5 text-white border-indigo-400
 slate-300 hover:text-white hover:border-indigo-400"
 >
 <Maximize2 className="w-3.5 h-3.5" />
 </button>
 {onOpenCompiler && (
 <button
 onClick={onOpenCompiler}
 title="Открыть Python: код выполняе
рации"
 className="flex items-center gap-1.5 text-white border-indigo-400
 slate-300 hover:text-white hover:border-indigo-400"
 >
 <Terminal className="w-3.5 h-3.5" />
 </button>
)}
 </div>
 </div>
 {viz.hint && <p className="text-xs text-slate-400">
 <VizChapterContext.Provider value={chapter}>
 <viz.Component />
 </VizChapterContext.Provider>
 </section>
)}

 <ChapterNav prev={prev} next={next} index={index} />
</article>

<TermPopover
 anchor={anchor}
 onClose={() => setAnchor(null)}
 />

```

```

 { id: 2, label: "C", x: 320, y: 250 },
 { id: 3, label: "D", x: 110, y: 260 },
 { id: 4, label: "E", x: 70, y: 110 },
 { id: 5, label: "F", x: 210, y: 150 },
 { id: 6, label: "G", x: 205, y: 285 },
];

const EDGES: [number, number][] = [
 [0, 1], [1, 2], [2, 3], [3, 4], [4, 0],
 [0, 5], [1, 5], [2, 5], [3, 5], [4, 5],
 [2, 6], [3, 6], [5, 6],
];

const ADJ: number[][] = (() => {
 const adj: number[][] = NODES.map(() => []);
 EDGES.forEach(([u, v]) => {
 adj[u].push(v);
 adj[v].push(u);
 });
 return adj;
})();

const COLORS = [
 { name: "красный", hex: "#ef4444" },
 { name: "синий", hex: "#3b82f6" },
 { name: "зелёный", hex: "#22c55e" },
 { name: "жёлтый", hex: "#eab308" },
];

const CONFLICT = "#e11d48";

/** Жадная раскраска по возрастанию номеров – честный у
function greedy(): number[] {
 const color: number[] = NODES.map(() => -1);
 for (let v = 0; v < NODES.length; v++) {
 const used = new Set(ADJ[v].map((u) => color[u])).fi
 let c = 0;
 while (used.has(c)) c++;
 }
}
```

```
})();
```

```
const compilerLinked = liveColors !== null || liveVLabel !== null;
const shownColors = liveColors ?? colors;
```

```
const liveCurrent = (() => {
 if (liveVLabel) {
 const i = NODES.findIndex((n) => n.label === liveVLabel);
 if (i >= 0) return i;
 }
 if (liveVNum !== null && liveVNum >= 0 && liveVNum < colors.length) {
 return colors[liveVNum];
 }
 return null;
})();
```

```
const conflicts = useMemo(() => {
 const bad = new Set<number>();
 EDGES.forEach(([u, v]) => {
 const cu = shownColors[u];
 const cv = shownColors[v];
 if (cu >= 0 && cu === cv) {
 bad.add(u);
 bad.add(v);
 }
 });
 return bad;
}, [shownColors]);
```

```
const usedColors = useMemo(
 () => new Set(shownColors.filter((c) => c >= 0)).size,
 [shownColors]
);
```

```
const paint = (id: number) => {
 const next = [...colors];
 next[id] = next[id] === brush ? -1 : brush;
 setColors(next);
};
```

```

 return (
 <g key={n.id} className="cursor-pointer"
 {isCurrent && <circle cx={n.x} cy={n.y} r={14} fill={c}
 />}
 <circle cx={n.x} cy={n.y} r={14} fill={c}
 >
 <text x={n.x} y={n.y + 4} textAnchor="center"
 {n.label}
 </text>
 {c >= 0 && (
 <text x={n.x} y={n.y - 20} textAnchor="center"
 {n.label}
 </text>
)}
 </g>
);
 }
 }
 </svg>
 {compilerLinked && (
 <div className="absolute top-2 right-2 bg-emerald-300 z-20">
 Python красит: {liveVLabel ?? ({liveVNum: 0})}
 </div>
)}
 {conflicts.size > 0 && (
 <div className="absolute bottom-2 left-1/2 -[11px] text-rose-300 z-20">
 ⚠ Конфликт: смежные вершины одного цвета
 </div>
)}
</div>

<div className="space-y-3">
 <div className="bg-slate-950 rounded-xl border">
 <div className="text-[10px] uppercase tracking-wider">
 <Palette className="w-3 h-3" /> Кисть

```

```
import { useMemo, useState } from "react";
import { RotateCcw } from "lucide-react";
import { useVizRuntime, vizArray, vizString } from "../viz";

/**
 * Тема 24 «Классы сложности, сведение задач».
 * Диаграмма вложенности классов $P \subseteq NP \subseteq PSPACE \subseteq EXPTIME$.
 * полиномиальных сведений $A \leq_p B$: если B решается легко
 */

type ClassId = "P" | "NP" | "PSPACE" | "EXPTIME";

interface Problem {
 id: string;
 name: string;
 cls: ClassId;
 x: number;
 y: number;
 npComplete?: boolean;
}

const PROBLEMS: Problem[] = [
 { id: "sort", name: "сортировка\n $O(n \log n)$ ", cls: "P", x: 15, y: 15, npComplete: false },
 { id: "mst", name: "остов (Краскал)", cls: "P", x: 15, y: 185, npComplete: false },
 { id: "2sat", name: "2-SAT", cls: "P", x: 115, y: 185, npComplete: false },
 { id: "shortpath", name: "Дейкстра / Флойд", cls: "P", x: 15, y: 215, npComplete: false },
 { id: "sat", name: "3-SAT", cls: "NP", x: 330, y: 120, npComplete: true },
 { id: "vc", name: "Vertex Cover", cls: "NP", x: 385, y: 185, npComplete: true },
 { id: "ham", name: "Гамильтонов цикл", cls: "NP", x: 385, y: 215, npComplete: true },
 { id: "knapsack", name: "Рюкзак (точный)", cls: "NP", x: 385, y: 245, npComplete: true },
 { id: "circuit", name: "QBF / игра на формуле", cls: "PSPACE", x: 385, y: 275, npComplete: false },
 { id: "chess", name: "обобщённые шахматы", cls: "EXPTIME", x: 385, y: 305, npComplete: false },
];

const REDUCTIONS: [string, string, string][] = [
 ["sat", "vc", "3-SAT $\leq_p$ VC"],
];
```

```

const liveRed = vizArray(vars?.reductions);
const compilerLinked = liveCls !== null || liveTask != null;

const liveRedPair = useMemo(() => {
 if (!liveRed || liveRed.length < 2) return null;
 return [String(liveRed[0]), String(liveRed[1])] as
}, [liveRed]);

const activeCls: ClassId | null =
 compilerLinked && liveCls
 ? (Object.keys(CLASS_META).find((k) => k.toLowerCase() === selected)
 as ClassId)
 : null;

const problemHit = (p: Problem) => {
 if (compilerLinked && liveTask) return p.name.toLowerCase() === selectedProblem;
 return selectedProblem === p.id;
};

const infoText = (() => {
 if (compilerLinked && liveTask) {
 const p = PROBLEMS.find((q) => q.name.toLowerCase() === selectedProblem);
 if (p) return `${p.name.replace("\n", " ")} ∈ ${p.cls}`;
 }
 if (compilerLinked && liveCls) {
 const cls = (activeCls ?? "NP") as ClassId;
 return CLASS_INFO[cls];
 }
 if (selectedProblem) {
 const p = PROBLEMS.find((q) => q.id === selectedProblem);
 return `${p.name.replace("\n", " ")} ∈ ${p.cls}${p.id}`;
 }
 if (activeCls) return CLASS_INFO[activeCls as ClassId];
 return "Кликни на класс или задачу. «A ≤p B» = «A не решается быстрее, чем B»";
})();

const reset = () => {
 setSelected(null);
};

```

```

 });
 }}}

 { /* стрелки сведений */
 { REDUCTIONS.map([from, to, label]) => {
 const a = PROBLEMS.find((p) => p.id === from);
 const b = PROBLEMS.find((p) => p.id === to);
 const hot = compilerLinked && liveRedPair
 ? (liveRedPair[0] && a.name.toLowerCase() === b.name.toLowerCase()
 ? liveRedPair[1] && b.name.toLowerCase()
 : selectedProblem === from || selectedProblem === to)
 : false;
 const mx = (a.x + b.x) / 2;
 const my = (a.y + b.y) / 2 - 4;
 return (
 <g key={` ${from} - ${to}`} opacity={hot ? 0.5 : 0.2}>
 <line x1={a.x} y1={a.y} x2={b.x} y2={b.y} />
 <marker id="arrowhead" markerWidth="7" markerHeight="5"
 viewBox="0 0 7 5" />
 <text x={mx} y={my} textAnchor="middle">{label}</text>
 </g>
);
 }}
 </defs>
 <marker id="arrowhead" markerWidth="7" markerHeight="5"
 viewBox="0 0 7 5" />
 </defs>

 { PROBLEMS.map((p) => {
 const dim = activeCls !== null && activeCls === p.class
 ? p.class : null;
 const hit = problemHit(p);
 return (
 <g key={p.id} onClick={() => setSelectedProblem(p.id)}>
 <rect
 x={p.x - 46} y={p.y - 13} width={92} height={26}
 fill={hit ? "#c2410c" : p.npCompleted ? "#f9f9f9" : "#f9f9f9"}
 stroke={hit ? "#fb923c" : CLASS_META[p.class] ? "#fb923c" : "#ccc"}
 strokeWidth={hit ? 2.5 : 1.2}
 />
 </g>
);
 })
}

```



```

 k="${liveTask}"` : ""}</div>
 })
 {infoText}
 </div>
 <div className="bg-amber-950/30 border border-
 Главный вопрос – $P = NP$? Все верят,
 Несведение ($P \neq NP$) доказать никто не смог
 ою задачу к 3-SAT, и она автоматически сложн
 </div>
 </div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 38/94: src/components/ComponentsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useEffect, useMemo, useState } from "react";
import { Pause, Play, RotateCcw, SkipForward } from "lucide-react";
import { useVizRuntime, vizArray, vizNumber, vizRecord, vizText } from "viz";

/**
 * Тема 8 «Графы. Компоненты связности».
 * Обход в ширину помечает каждую вершину номером её координат
 * сколько раз пришлось запускать обход – столько и компонент
 */

interface V {
 id: number;
 label: string;
 x: number;
 y: number;
}

```

```

}

/** Полный прогон BFS-разметки: кадры строим заранее (ч
function buildFrames(): StepFrame[] {
 const frames: StepFrame[] = [];
 const comp: (number | null)[] = NODES.map(() => null);
 let count = 0;
 const push = (f: Omit<StepFrame, "comp" | "count"> &
 frames.push({ comp: [...comp], count, ...f }));

 push({ queue: [], current: null, scanning: null, log:
 ной вершины." }));

 for (let s = 0; s < NODES.length; s++) {
 if (comp[s] !== null) continue;
 count += 1;
 const queue = [s];
 comp[s] = count - 1;
 push({ queue: [...queue], current: s, scanning: null,
 арует с неё.` }));
 while (queue.length) {
 const v = queue.shift()!;
 push({ queue: [...queue], current: v, scanning: null,
 й.` }));
 for (const to of ADJ[v]) {
 if (comp[to] === null) {
 comp[to] = comp[v];
 queue.push(to);
 push({ queue: [...queue], current: v, scanning: null,
 {comp[v]}, в очередь.` }));
 } else {
 push({ queue: [...queue], current: v, scanning: null,
 });
 }
 }
 }
 }
 push({ queue: [], current: null, scanning: null, log:
}

```

```

const frame = FRAMES[Math.min(step, FRAMES.length - 1)
const compilerLinked =
 liveVLabel !== null || liveVNum !== null || !!liveC

const liveIndexOf = () => {
 if (liveVLabel) {
 const idx = NODES.findIndex((n) => n.label === liveVLabel)
 return idx >= 0 ? idx : null
 }
 return liveVNum !== null && liveVNum >= 0 && liveVNum <= 1000 ? liveVNum : null
};
const liveCurrent = liveIndexOf();

const liveCompOf = (n: V): number | null => {
 if (liveCompByLabel && n.label in liveCompByLabel)
 const c = vizNumber(liveCompByLabel[n.label]);
 if (c !== null) return c;
 if (liveCompByLabel && String(n.id) in liveCompByLabel)
 const c = vizNumber(liveCompByLabel[String(n.id)]);
 if (c !== null) return c;
 return frame.comp[n.id];
};
const liveQueueLabels = (liveQ
 ? liveQ.map((x) => (typeof x === "string" ? x : vizNumber(x)))
 : null
)?.filter(Boolean);
const displayCount = liveCount ?? frame.count;
const displayLog = compilerLinked
 ? `Компилятор: v=${liveVLabel ?? liveVNum ?? "-"}
 ES[q].label)).join(", ")`
 : frame.log;

const shownCurrent = compilerLinked ? liveCurrent : frame.current;
const shownScanning = compilerLinked ? null : frame.scanning;
const shownQueue: number[] = liveQueueLabels
 ? liveQueueLabels

```

```

 className="flex items-center gap-1 bg-indigo-500 text-white p-2 rounded-md"
 t-xs font-medium"
 >
 <SkipForward className="w-3.5 h-3.5" /> ⏭
 </button>
 </div>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-[1fr 1fr] gap-4" >
 <div className="bg-slate-950 rounded-xl border border-slate-800 p-4" >
 <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,black_1px)]" >
 <svg className="w-full h-[320px] relative z-10" >
 {EDGES.map(([u, v], i) => {
 const a = NODES[u];
 const b = NODES[v];
 const sameComp = liveCompOf(a) !== null && liveCompOf(b) !== null && liveCompOf(a) === liveCompOf(b);
 return (
 <line
 key={i}
 x1={a.x} y1={a.y} x2={b.x} y2={b.y}
 stroke={sameComp ? (COMP_COLORS[liveCompOf(a)] || 'black') : 'black'}
 strokeWidth={sameComp ? 3.5 : 2}
 opacity={sameComp ? 0.9 : 0.6}
 />
);
 })}
 </svg>
 </div>
 <div>
 {NODES.map((n) => {
 const c = liveCompOf(n);
 const isCurrent = shownCurrent === n.id;
 const isScanned = shownScanning === n.id;
 const inQueue = shownQueue.includes(n.id);
 const color = c === null ? "#334155" : COMP_COLORS[c] || "black";
 return (
 <g key={n.id}>
 {(isCurrent || isScanned) && (
 <circle cx={n.x} cy={n.y} r={18} fill={color} stroke={color} stroke-width={2}
)}
 {isCurrent && <circle cx={n.x} cy={n.y} r={18} fill="white" stroke={color} stroke-width={2}
 </g>
);
 })}
 </div>
 </div>
 </div>

```

```

 <div className="grid grid-cols-5 gap-1">
 {NODES.map((n) => {
 const c = liveCompOf(n);
 return (
 <div key={n.id} className={`text-center
0 text-slate-500" : "border-transparent text
 style={c === null ? {} : { background
LORS.length] }>
 <div className="text-slate-400">{n.
 <div>{c === null ? "-" : c}</div>
 </div>
);
 })}
 </div>
 </div>
 <div className="bg-slate-900/60 rounded-xl bo
 {displayLog}
 </div>
</div>
</div>
</div>
);
}

```

ФАЙЛ 39/94: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import { useState, useEffect, useRef } from "react";
import { useVizStepSync, vizNumber, vizString } from "."
import { Undo, Play, Pause, SkipForward, RefreshCw } from "lucide-react";

```

```

interface CodeLine {
 line: string;
 highlight: string;
}

```

```

 { id: 1, label: "B", x: 260, y: 120 },
 { id: 2, label: "C", x: 180, y: 200 },
 { id: 3, label: "D", x: 180, y: 300 },
 { id: 4, label: "E", x: 100, y: 370 },
 { id: 5, label: "F", x: 260, y: 370 },
 { id: 6, label: "G", x: 340, y: 200 }
];

const GRAPH_EDGES = [
 { u: 0, v: 1, key: "0-1" },
 { u: 1, v: 2, key: "1-2" },
 { u: 2, v: 0, key: "0-2" },
 { u: 2, v: 3, key: "2-3" },
 { u: 3, v: 4, key: "3-4" },
 { u: 4, v: 5, key: "4-5" },
 { u: 5, v: 3, key: "3-5" },
 { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,
 log: "Входим в A. tin[A]=1, low[A]=1.",
 activeCodeLine: [0,1,2]
 },
 {
 currentNode: 1, visitedIndices: [0,1],
 tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
 activeEdge: [0,1], backEdge: null,
 log: "Идём (A,B). B впервые. tin[B]=2, low[B]=2.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 6, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: [1,6], backEdge: null,

```

```

{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: [4,5], backEdge: null,
 log: "E→F. tin[F]=7, low[F]=7.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: [5,3],
 log: "Из F видим D (посещён). Обратное ребро (F,D).",
 activeCodeLine: [7,8,9]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо",
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мо",
 activeCodeLine: [11,13,16,17]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3},
 activeEdge: null, backEdge: null,
 log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) –",
 activeCodeLine: [13,14,15,16,17]
},
{
 currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3},

```

```

const [dfsAuto, setDfsAuto] = useState(false);
const dfsTimer = useRef<NodeJS.Timeout | null>(null);

useEffect(() => {
 if (dfsAuto) {
 dfsTimer.current = setInterval(() => {
 setDfsIdx(prev => {
 if (prev >= DFS_STEPS.length - 1) { setDfsAuto(false);
 return prev + 1;
 });
 }, 2500);
 }
 return () => { if (dfsTimer.current) clearInterval(dfsTimer.current);
}, [dfsAuto]);

const step = DFS_STEPS[dfsIdx];

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">
 <div className="flex items-center gap-1 bg-slate-800 p-1">
 <button onClick={() => { setDfsAuto(false); setDfsIdx(0);
 disabled={dfsIdx === 0}
 className="p-1.5 hover:bg-slate-700 rounded text-white">
 <Undo className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => setDfsAuto(!dfsAuto)}
 className={`px-2.5 py-1 rounded text-[10px] ${
 dfsAuto ? "bg-rose-600 text-white" : "bg-slate-700 text-white"
 }`} >
 {dfsAuto ? <><Pause className="h-3 w-3" /> : <><Play className="h-3 w-3" />
 </button>
 <button onClick={() => { setDfsAuto(false); setDfsIdx(DFS_STEPS.length-1);
 disabled={dfsIdx === DFS_STEPS.length-1}
 className="p-1.5 hover:bg-slate-700 rounded text-white">
 <SkipForward className="h-3.5 w-3.5" />
 </button>
 </div>
 </div>
 </div>
 </div>
);

```





```

 </div>

 { /* Log */
 <div className="bg-indigo-950/10 border borde
 <p className="text-[11px] text-slate-300 le
 <div className="mt-2 pt-2 border-t border-s
 Мосты:</
 <span className="font-mono font-bold text
 {step.bridges.length > 0 ? step.bridges

 </div>
 </div>
 </div>
 </div>
 </div>
);
}

```

ФАЙЛ 40/94: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber, vizRecord,

interface Node { id: string; x: number; y: number; name
interface Edge { u: string; v: string; w: number; }
interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
 visited: { [key: string]: boolean };
 previous: { [key: string]: string | null };
 log: string;
 activeCodeLine: number;
}

```

```

 { line: 6, text: ' if d > dist[u]: continue'
 { line: 7, text: ' for v, weight in graph.ne
 { line: 8, text: ' if dist[u] + weight <
 { line: 9, text: ' dist[v] = dist[u]
 { line: 10, text: ' pq.push((dist[v]
 { line: 11, text: ' return dist // Все кратчайши
];

```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState(false);

```

```

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999
 const visited: Record<string, boolean> = { A: false
 const prev: Record<string, string | null> = { A: nu

```

```

simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из Пи
 activeCodeLine: 2,
});

```

```

for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }
}

```

```

 if (!u) break;

```

```

 log: ` Дейкстра всё доставил! Все возможные кратчайшие пути
 activeCodeLine: 11,
 });

```

```

 setSteps(simulationSteps);
 }, []);

```

```

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

```

```

const baseStep = steps[currentStepIndex] || null;

```

```

if (!baseStep) return <div className="text-white">Зарегистрируйтесь, чтобы продолжить!</div>;
const vars = runtime?.variables;
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveDist = vizRecord(vars?.dist);
const livePrev = vizRecord(vars?.prev);
const liveDone = vizArray(vars?.done) ?? vizArray(vars?.done);
const compilerLinked = liveU !== null || liveV !== null;
const currentStep: Step = compilerLinked
 ? {

```

```

 ...baseStep,
 currentNode: liveU,
 checkingEdge: liveU && liveV ? { u: liveU, v: liveV } : null,
 distances: Object.fromEntries(nodes.map((node) => [node.id, 0])),
 previous: Object.fromEntries(nodes.map((node) => [node.id, null])),
 visited: Object.fromEntries(nodes.map((node) => [node.id, false])),
 log: `Компилятор: u=${liveU ?? "-"}, v=${liveV ?? "-"}, dist=${liveDist ?? "-"}, prev=${livePrev ?? "-"}, done=${liveDone ?? "-"}

```

```

 >
 Шаг
 </button>
 </div>
 </div>

 <div className="flex flex-col lg:flex-row gap-6 md:gap-8" >
 { /* Граф */ }
 <div className="w-full lg:w-2/3 bg-indigo-950/20 flex flex-direction-column justify-center min-h-[400px]" >
 <div className="absolute top-3 left-3 bg-indigo-950/20 text-white font-bold shadow-sm" >
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>

 <svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-dh-normal" markerWidth="10" markerHeight="10" >
 <path d="M0,0 L6,3 L0,6 z" fill="#475569" />
 </marker>
 <marker id="arrow-dh-active" markerWidth="10" markerHeight="10" >
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 </marker>
 <marker id="arrow-dh-opt" markerWidth="6" markerHeight="6" >
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 </marker>
 </defs>
 { /* Рёбра */ }
 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u)
 const vNode = nodes.find((n) => n.id === edge.v)
 const isChecking =
 currentStep.checkingEdge &&
 (currentStep.checkingEdge.u === edge.u && currentStep.checkingEdge.v === edge.v)
 const isOptimal = currentStep.previous[edge.u] === edge.v

 let marker = 'url(#arrow-dh-normal)';

```

```

 <circle
 cx={node.x} cy={node.y}
 r={isCurrent ? 22 : 18}
 fill={isCurrent ? '#4f46e5' : isVis
 stroke={isCurrent ? '#a5b4fc' : isV
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-1
 />
 <text x={node.x} y={node.y + 5} fill=
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill
"middle" className="bg-indigo-950 px-1 py-0
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 <text x={node.x} y={node.y + 35} fill
 {node.name.split(' ')[0]}
 </text>
 </g>
);
 }}}
</svg>
<div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400
 <p className="min-h-[40px] flex items-cente
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2.5 flex-1 overflow-y
 {Object.keys(adjList).map((u) => {
 const isCurrentNode = currentStep.current
 return (
 <div key={u} className={`p-3 rounded-lg
 isCurrentNode ? 'bg-emerald-900/60
 t-slate-300'

```

```

 <tr className="border-b border-rose-900">
 <th className="py-2 px-3">Вершина</th>
 <th className="py-2 px-3">Расстояние</th>
 <th className="py-2 px-3">Из (пред.)</th>
 </tr>
 </thead>
 <tbody className="text-sm">
 {nodes.map((n) => {
 const dist = currentStep.distances[n.id]
 const prev = currentStep.previous[n.id]
 const vis = currentStep.visited[n.id]
 const isCurr = currentStep.currentNode === n.id

 return (
 <tr key={n.id} className={`border-1 border-rose-900
 ${isCurr ? 'bg-indigo-950/60 font-medium' : ''}`>
 <td className="py-2.5 px-3 flex items-center">

 {n.name}

 </td>
 <td className="py-2.5 px-3 font-mono">
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text-sm">
 {prev || '-'}
 </td>
 </tr>
)
 })}
 </tbody>
 </table>
 </div>
</div>
</div>

```

```

 { /* Код алгоритма */ }
 <div className="w-full lg:w-1/2 bg-slate-900/60" style={

```

```

 desc: string;
 codeLine: number;
 memoState: { [key: number]: number };
}

const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo",
 /* 5 */ " return memo[n];",
 /* 6 */ "}"
];

export function DPVisualizer() {
 const [targetN, setTargetN] = useState<number>(5);
 const [steps, setSteps] = useState<DPStep[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState<boolean>(f
 const [speed] = useState<number>(800);
 const [activeTab, setActiveTab] = useState<'table' |

 useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibMemo(${n}). Проверяем наличие в
 codeLine: 2,
 memoState: { ...memoMap }
 });
 }
 });

```



```

 id: stepId++,
 n,
 action: 'calc',
 desc: `Сохраняем в кэш: мемо[${n}] = ${res1} + ${res2}`,
 codeLine: 5,
 memoState: { ...memoMap }
 });

 return memoMap[n];
}

const finalRes = simulateFib(targetN);

generatedSteps.push({
 id: stepId++,
 n: targetN,
 action: 'done',
 desc: `Расчет завершен! fibMemo[${targetN}] = ${finalRes}`,
 codeLine: 6,
 memoState: { ...memoMap }
});

setSteps(generatedSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [targetN]));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

```

```

<div className="flex items-center gap-2 bg-slate-900" >
 N:
 <select
 value={targetN}
 onChange={(e) => setTargetN(Number(e.target.value))}
 disabled={isPlaying}
 className="bg-transparent text-white font-mono"
 >
 <option value={4}>N = 4</option>
 <option value={5}>N = 5</option>
 <option value={6}>N = 6</option>
 <option value={7}>N = 7</option>
 </select>
</div>

```

```

<div className="flex items-center bg-slate-950" >
 <button
 onClick={() => { setIsPlaying(false); setTargetN(4); }}
 className="p-2 text-slate-400 hover:text-white"
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setIsPlaying(false); setTargetN(4); }}
 disabled={currentStepIndex === 0}
 className="p-2 text-slate-400 hover:text-white"
 title="Шаг назад"
 >
 <ChevronLeft className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1.5 px-2 py-2
 ${isPlaying
 ? 'bg-amber-500 text-slate-950 hover:bg-amber-600'
 : 'bg-emerald-600 text-white hover:bg-emerald-700'}
 `}
 >
 {isPlaying ? 'Пауза' : 'Воспроизвести'}
 </button>
</div>

```

```

 {isCached ? val : '0'}

 {num <= 2 && <span className="text-[9px]
 </div>
);
 }}}
</div>

{currentStep && (
 <div className="mt-6 pt-4 border-t border-slate-400" >
 <div className="text-sm font-medium text-slate-600" >

 {currentStep.desc}
 </div>
 <div className="text-xs font-mono bg-slate-50 p-2" >
 Текущий N: <strong className="text-emerald-500" /> {currentStep.n}
 </div>
 </div>
)}
</div>

{/* Tabs */}
<div className="flex items-center gap-2 border-b border-slate-200" >
 <button
 onClick={() => setActiveTab('table')}
 className={`flex items-center gap-2 px-6 py-3
 ${activeTab === 'table'
 ? 'border-emerald-500 text-emerald-400 bg-emerald-50'
 : 'border-transparent text-slate-400 hover:text-slate-600'}
 `}
 >
 <Eye className="w-4 h-4" />
 Степ-бай-степ состояние
 </button>
 <button
 onClick={() => setActiveTab('code')}
 className={`flex items-center gap-2 px-6 py-3
 ${activeTab === 'code'

```

```

 return (
 <div
 key={lineNum}
 className={`flex items-center px-4
 isActive
 ? 'bg-emerald-600/20 border-l-4
 : 'text-slate-300 hover:bg-slate-
 `}
 >
 <span className="w-8 text-xs text-s
 {line}
 </div>
);
 })}
 </pre>
 <div className="p-6 bg-slate-900/50 border-
 <span className="p-1.5 bg-emerald-500/20
 {currentStep?.desc}
 </div>
 </div>
)}
</div>

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-8
 <div className="flex items-center justify-betwe
 Прогресс выполнения
 № {currentStepIndex + 1} из {steps.len
 </div>
 <div className="h-2 bg-slate-950 rounded-full o
 <div
 className="h-full bg-gradient-to-r from-eme
 style={{ width: `${((currentStepIndex + 1)
 ></div>
 </div>
</div>
</div>
);

```

```

const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
});

// Компилятор управляет обходом, если в коде есть path
const compilerLinked = livePathLabels !== null && live
const livePath = useMemo(() => {
 if (!compilerLinked) return null;
 const ids = livePathLabels!
 .map((lab) => C1_NODES.find((n) => n.label === lab)
 .filter((x): x is number => x !== undefined);
 return ids;
}, [compilerLinked, livePathLabels]);
const shownPath = livePath ?? c1Path;
const shownVisited = useMemo(() => {
 if (livePath) {
 const keys: string[] = [];
 for (let i = 1; i < livePath.length; i++) {
 const a = livePath[i - 1];
 const b = livePath[i];
 if (C1_EDGES.some((e) => (e.u === a && e.v === b))) {
 keys.push(`${Math.min(a, b)}-${Math.max(a, b)}`);
 }
 }
 return keys;
 }
 return c1VisitedEdges;
}, [livePath, c1VisitedEdges]);

const degrees = useMemo(
 () => C1_NODES.map((n) => ({ label: n.label, deg: C1_NODES[n.label].deg })),
 []
);

const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {

```

```

<h4 className="text-base font-bold text-white">
<button onClick={resetC1} className="flex items
 old border border-slate-700 transition-all">
 <Undo className="h-3.5 w-3.5" /> C6poc
</button>
</div>

<div className="bg-slate-950 rounded-xl p-4 borde
 -hidden">
<div className="absolute inset-0 bg-[radial-gra
<svg className="w-full h-[260px] z-10 relative"
 {C1_EDGES.map((e, i) => {
 const u = C1_NODES.find(n => n.id === e.u)!
 const v = C1_NODES.find(n => n.id === e.v)!
 const key = `${Math.min(e.u,e.v)}-${Math.ma
 const used = shownVisited.includes(key);
 return <line key={i} x1={u.x} y1={u.y} x2={
 stroke={used ? "#6366f1" : "#475569"}
 strokeWidth={used ? 4 : 2}
 className="transition-all duration-300" />
 }}}
 {C1_NODES.map(n => {
 const isLast = shownPath[shownPath.length-1
 const visited = shownPath.includes(n.id);
 return <g key={n.id} className="cursor-poin
 {isLast && <circle cx={n.x} cy={n.y} r={1
 <circle cx={n.x} cy={n.y} r={11}
 fill={isLast ? "#6366f1" : visited ? "#
 stroke={isLast ? "#fff" : visited ? "#6
 strokeWidth={2.5}
 className="transition-all duration-200
 <text x={n.x} y={n.y+4} textAnchor="middl
 bel}</text>
 </g>;
 }}}
</svg>
<div className="absolute top-2 left-2 bg-slate-
 Pe6ëp: {shownVisited.length}/{C1_EDGES.length

```

```
const quizQuestions: Question[] = [
 {
 id: 1,
 question: 'Что мы делаем в алгоритме Краскала, если
ребро (красный) цвет?',
 options: [
 'Мы радуемся и добавляем его в минимальное остовное дерево',
 'Мы перекрашиваем их в синий цвет и делим граф по
компонентам связности',
 'Мы безжалостно ВЫБРАСЫВАЕМ это ребро, иначе получим цикл',
 'Мы вызываем алгоритм Дейкстры для поиска нового ребра'
],
 correctAnswer: 2,
 explanation: 'Верно! Если вершины уже в одном клане
и добавим ребро, это нарушит структуру дерева.'
 },
 {
 id: 2,
 question: 'Почему алгоритм Дейкстры впадает в ступор?',
 options: [
 'Потому что он написан только для положительных весов',
 'Он жадный и считает, что каждый следующий шаг де-
лает его лучше, не учитывая уже закрытые вершины.',
 'Отрицательные ребра создают гравитационный коллапс',
 'Дейкстра просто не любил отрицательные балансы на
картах'
],
 correctAnswer: 1,
 explanation: 'Именно! Дейкстра уверен: если ты прие-
хал в пункт назначения, то для чего нужен алгоритм Беллмана-Форда.'
 },
 {
 id: 3,
 question: 'Какова ключевая фишка алгоритма Борувки?',
 options: [
 'Он использует меньше памяти благодаря коммунистиче-
ским принципам',
 'Он умеет работать с отрицательными весами без ци-
клов',
 'Он позволяет выполнять шаги слияния колхозов (ком-
пактизации)',
 'Он был разработан в Токио и поэтому самый быстрый'
],
 },
]
```

```

const currentQ = quizQuestions[currentQIndex];

const handleSelectOption = (index: number) => {
 if (isAnswered) return;
 setSelectedOption(index);
 setIsAnswered(true);
 if (index === currentQ.correctAnswer) {
 setScore(prev => prev + 1);
 }
};

const handleNext = () => {
 if (currentQIndex + 1 < quizQuestions.length) {
 setCurrentQIndex(prev => prev + 1);
 setSelectedOption(null);
 setIsAnswered(false);
 } else {
 setShowResults(true);
 }
};

const handleRestart = () => {
 setCurrentQIndex(0);
 setSelectedOption(null);
 setIsAnswered(false);
 setScore(0);
 setShowResults(false);
};

if (showResults) {
 return (
 <div className="bg-slate-900/90 border border-slate-800 p-12">
 <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-500 shadow-xl shadow-indigo-500/30">
 <Award className="w-10 h-10 text-white" />
 </div>
 <h3 className="text-3xl font-bold text-white mb-4">

```



```


 </div>

 <div className="mb-8">
 <h4 className="text-xl md:text-2xl font-bold text-gray-900">
 {currentQ.question}
 </h4>

 <div className="space-y-4">
 {currentQ.options.map((option, idx) => {
 const isSelected = selectedOption === idx;
 const isCorrect = idx === currentQ.correctAnswer;

 let optionStyle = "bg-slate-800/80 hover:bg-slate-800/90";
 if (isAnswered) {
 if (isCorrect) {
 optionStyle = "bg-emerald-950/80 border-emerald-500";
 } else if (isSelected) {
 optionStyle = "bg-rose-950/80 border-rose-500";
 } else {
 optionStyle = "bg-slate-800/40 border-slate-700";
 }
 }

 return (
 <div
 key={idx}
 onClick={() => handleSelectOption(idx)}
 className={"flex items-start gap-4 p-4 rounded-lg"}
 style={optionStyle}
 >
 <div className="mt-0.5 shrink-0">
 {isAnswered && isCorrect ? (
 <CheckCircle2 className="w-6 h-6 text-emerald-500" />
) : isAnswered && isSelected && !isCorrect ? (
 <XCircled className="w-6 h-6 text-rose-500" />
) : (
 <div className={"w-6 h-6 rounded-full border border-slate-700 bg-indigo-500 text-white"} />
)}
 <div className="flex-1">
 {option}
 </div>
 </div>
);
 })}
 </div>
 </div>

```

```
 </div>
 </div>
);
 };
};
```

---

ФАЙЛ 44/94: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber } from '../viz';

interface Step {
 k: number; i: number; j: number;
 matrix: (number | null)[][]; updated: boolean;
 log: string; activeCodeLine: number;
}

export default function FloydViz() {
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const runtime = useVizRuntime();
 // k/i/j/d приходят напрямую; номер строки произвольный
 const [isPlaying, setIsPlaying] = useState(false);

 const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4', 'Е 5', 'Ж 6'];
 const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];
};
```

```

 activeCodeLine: 2,
 });

 const N = 7;
 for (let k = 0; k < N; k++) {
 simulationSteps.push({
 k, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: `+ Внешний цикл. Разрешаем пути через пром
 activeCodeLine: 3,
 });

 for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {
 if (i === j || i === k || j === k) continue;

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999 &&
 dist[i][j] > viaVal) {
 dist[i][j] = viaVal;
 simulationSteps.push({
 k, i, j, matrix: dist.map(r => [...r]),
 log: ` Улучшение пути [${i}]→[${j}] чере
 ∞'}.`,
 activeCodeLine: 7,
 });
 }
 }
 }
 }

 simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: ` Все пары отработаны. Алгоритм Флойда завер
 activeCodeLine: 8,
 });

 setSteps(simulationSteps);
 setCurrentStepIndex(0);

```

```
: baseStep;
```

```
return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border"
 <div className="flex flex-wrap items-center justify"
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white"

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white"
 <p className="text-sm text-emerald-300">Полн
 </div>
 </div>

 <div className="flex items-center gap-2">
 <button onClick={() => { setIsPlaying(false);
 text-sm"> Сброс</button>
 <button onClick={() => setIsPlaying(!isPlaying
 -amber-600' : 'bg-emerald-600'}`}>{isPlaying
 <button onClick={() => { setIsPlaying(false);
 ; }} className="px-3 py-2 bg-emerald-600 te
 </div>
 </div>

 <div className="flex flex-col lg:flex-row gap-6 ml
 {/* Граф */}
 <div className="w-full lg:w-2/3 bg-indigo-950/20
 stify-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-indi
 ont-bold shadow-sm">
 {currentStep.k >= 0 && currentStep.k < 7 ?
 </div>

 <svg className="w-full h-auto max-h-[500px]"
 <defs>
 <marker id="arrow-fw-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#475569" />

```

```

s] ? 4 : 2} className="transition-colors du
 <text x={node.x} y={node.y + 5} fill=
 <text x={node.x} y={node.y + 32} fill=
 </text>
 </g>
);
 }}}
</svg>

```

```

<div className="w-full bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400 r
 <p className="min-h-[40px] flex items-cente
</div>
</div>

```

```

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map((uStr) => {
 const u = parseInt(uStr);
 const isI = currentStep.i === u;
 const isK = currentStep.k === u;
 return (
 <div key={u} className={`flex flex-wrap
 isI ? 'bg-amber-900/40 border-ambe
 isK ? 'bg-indigo-900/40 border-ind
 `}>
 <span className="font-bold text-whi
e-400">→
 {adjList[u].map((edge, idx) => {
 const isUpd = currentStep.i === u
 const isVia = currentStep.k !== -
rentStep.j === edge.v);
 return (
 <span key={idx} className={`px-1
dow' : isVia ? 'bg-indigo-500 text-white bo
 {edge.v} ({edge.w})

```

```

 bg = 'bg-indigo-950/60 text-indigo-50'
 else if (i === j) bg = 'bg-slate-900 text-slate-50'
 return (<td key={j} className={`p-2 ${bg} text-center`}
 === 999 ? '∞' : val)</td>);
 })}
 </tr>
 </tbody>
</table>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 p-4" >
 <div>
 <h4 className="text-lg font-bold text-slate-50">Алгоритм
 <div className="bg-slate-950/80 rounded-lg p-4" >
 {codeLines.map(item => {
 <div key={item.line} className={`py-1.5 px-2 ${
 item.line === 1 ? 'bg-blue-900/80 text-amber-50' : 'bg-slate-950/80 text-slate-50'
 }`} >
 {item.line}
 {item.code}
 </div>
 })}
 </div>
 </div>
</div>
</div>
</div>
);
}

```

```

 visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const stack: string[] = []; // for visualization, just for fun

 function dfs(v: string) {
 vis.add(v);
 stack.push(v);
 steps.push({ type: 'visit', node: v, desc: `Зависит от: ${visitedNodes: Array.from(vis)}` });

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 steps.push({ type: 'process', node: v, desc: `Процесс: ${u}, ${stack}, visitedNodes: Array.from(vis) });
 dfs(u);
 }
 }

 stack.pop();
 steps.push({ type: 'backtrack', node: v, desc: `Обратный шаг: ${uOrStack: [...stack]}, visitedNodes: Array.from(vis) });
 }

 dfs(start);
 return steps;
}

function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);

```

```

 .map(({step, index}) => ({ step, index })))
 .filter(({ step }) =>
 (v === null || step.node === v) &&
 (!structure || step.queueOrStack?.length ===
 (!visited || step.visitedNodes?.length === vi
));
 return candidates.at(-1)?.index ?? null;
});

useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [mode]);

useEffect(() => {
 if (autoPlay) {
 const t = setInterval(() => {
 setStepIdx(p => {
 if (p >= steps.length - 1) {
 setAutoPlay(false);
 return p;
 }
 return p + 1;
 });
 }, 1200);
 return () => clearInterval(t);
 }
}, [autoPlay, steps.length]);

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedNodes?.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
 <div className="space-y-4">
 <div className="flex bg-slate-900 border border-slate-800 p-4">
 <button onClick={() => setMode("dfs")}>DFS</button>
 </div>
 </div>
);

```



```

 let radius = 18;

 if (visited) {
 fillColor = mode === 'd' ? 'darkblue' : 'darkred';
 strokeColor = mode === 'd' ? 'darkblue' : 'darkred';
 textColor = "fill-white";
 }

 if (current && step.type !== 'd') {
 strokeColor = "stroke-amber";
 textColor = "fill-amber";
 radius = 22;
 }

 return (
 <g key={n.id} className={className} >
 <circle cx={n.x} cy={n.y} radius={radius}
 className={`stroke-${strokeColor} fill-${fillColor}`} />
 <text x={n.x} y={n.y} text={n.label}
 className={`text-${textColor}`} />
 </g>

 {
 /* Ripple for BFS */
 {current && mode === 'd' && step.type !== 'd' && !visited} && (
 <circle cx={n.x} cy={n.y} radius={radius}
 className="stroke-amber fill-amber" />
)
)
)
 }
}
</svg>
</div>

/* Structure View (Stack / Queue) */
<div className="bg-slate-900 border border-slate-800 p-10" >
 <div className={`absolute -top-3 left-3 right-3 bottom-3`} >

```

```
 <button onClick={() => {setAutoPlay
= steps.length - 1} className="p-2 text-slate-800
hover:bg-transparent">
 <SkipForward strokeWidth={3} size={24}/>
 </button>
 <button onClick={() => {setAutoPlay
g-slate-800 rounded-lg ml-2 border-1 border-slate-800
 <RefreshCw strokeWidth={3} size={24}/>
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50 border-1 border-slate-800
ow-hidden">
 <div className="text-[10px] text-slate-400">
 {steps.length}</div>
 <div className="text-sm text-slate-400">
 {step.desc}
 </div>
 </div>
</div>
</div>
);
}
```

---

ФАЙЛ 46/94: src/components/HeartViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
import { useVizRuntime, vizArray, vizNumber } from '../hooks';
```

```
interface Patient {
 id: string;
 name: string;
```

Универсальное пособие • полный исходный код

```

function heapInsert(arr: Patient[], item: Patient): Patient[] {
 return siftUp([...arr, { ...item }]);
}
```

```
// Position in tree layout
```

```
function nodePos(index: number): { x: number; y: number } {
 const level = Math.floor(Math.log2(index + 1));
 const posInLevel = index - (Math.pow(2, level) - 1);
 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}
```

```
// Priority presets for realistic ER scenarios
```

```
const PATIENT_PRESETS = [
 { name: 'Иван (Инфаркт)', symptom: ' Боли в сердце', emoji: '👤', priority: 100 },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤', priority: 100 },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈', priority: 100 },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤', priority: 100 },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤', priority: 100 },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤', priority: 100 },
];
```

```
const INITIAL_PATIENTS: Patient[] = (() => {
 let arr: Patient[] = [];
 const start = [
 { name: 'Иван (Инфаркт)', symptom: ' Боли в сердце', emoji: '👤', priority: 100 },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤', priority: 100 },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤', priority: 100 },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈', priority: 100 },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤', priority: 100 },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤', priority: 100 },
];
 start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animS: 0 });
 });
 return arr;
})
```

```

 return;
 }
 setBusy(true);

 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const finalName = name || preset.name.split(' ')[0];
 const finalEmoji = preset.emoji;
 const finalSymptom = preset.symptom;

 const newPatient: Patient = {
 id: `p${uid++}`,
 name: finalName,
 emoji: finalEmoji,
 symptom: finalSymptom,
 priority,
 animState: 'in',
 };

 const nextHeap = heapInsert(heap, newPatient).map(x => x);
 // Mark specifically this new patient as 'in' in the heap
 const target = nextHeap.find(x => x.priority === priority);
 const marked = nextHeap.map(x => x.id === (target?.id));

 setHeap(marked);
 addLog(` Поступил пациент: ${finalName} с тяжестью ${priority}`);

 setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
 }, 500);
}, [busy, heap]);

const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const randomShift = Math.floor(Math.random() * 10);
 const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
 insertPatient(preset.name, prio);
};

```

```

 // Patient gets healed in the bed
 setTimeout(() => {
 setHealing(prev => prev ? { ...prev, animStat
 addLog(` Успех! Пациент ${topPatient.name} п
 setTimeout(() => {
 setHealing(null);
 setBusy(false);
 }, 600);
 }, 800);

 }, 350);
}, 650);
}, [busy, heap]));

const reset = () => {
 setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat
 setLog(` База данных скорой помощи перезагружена.
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x =>
});

// --- Render binary tree lines ---
const edges = visualHeap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < visualHeap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%` } y1={` ${p.y + 4}%` }
 x2={` ${lp.x}%` } y2={` ${lp.y - 4}%` }
 stroke="#475569" strokeWidth="2"
 />
);
 }
});
}

```

```

 opacity-40 disabled:cursor-not-allowed text-
 ve:scale-95 transition-all cursor-pointer f
 >
 Привезти по Скорой (INSERT)
</button>

{/* Добавить кастомного */}
<form onSubmit={handleCustomInsert} className=""
 <input
 type="text"
 required
 value={customName}
 onChange={e => setCustomName(e.target.value)}
 placeholder="Имя пациента"
 className="flex-1 bg-slate-800 border border-
 er-emerald-500 transition-colors placeholder-
 />
 <div className="flex flex-col items-center min-h-100"

 <input
 type="range"
 min="10"
 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e.target.value))}
 className="w-20 accent-emerald-500 cursor-pointer"
 />
 </div>
 <button
 type="submit"
 disabled={busy || !customName.trim() || !customPriority}
 className="bg-teal-600 hover:bg-teal-500 disabled:opacity-40
 px-4 rounded-xl shadow-lg active:scale-95 transition-colors"
 >
 Везти
 </button>
 </form>

```

```

const isCompilerNode = liveI === idx;
return (
 <div
 key={patient.id}
 className={`
 absolute flex flex-col items-center
 -translate-x-1/2 -translate-y-1/2 t
 ${patient.animState === 'in' ? 'anim
 ${patient.animState === 'winner' ?
 ${patient.animState === 'out' ? 'an
 ${isCompilerNode ? 'ring-4 ring-amb
 ${isRoot ? 'bg-rose-950/80 border-r
 `}
 style={{
 left: `${pos.x}%`,
 top: `${pos.y}%`,
 width: isRoot ? 60 : 52,
 height: isRoot ? 60 : 52,
 }}
 >
 <span className="text-2xl leading-non
 <span className="text-[10px] font-mon
 {patient.priority}%

 {/* Tooltip */}
 <div className="absolute bottom-full r
 <div className="bg-slate-950 border
0 shadow-xl">
 <div className="font-bold text-wh
 <div className="text-emerald-400"
 </div>
 </div>
 </div>
</div>
);
}}
</div>

```

```
 <div className="w-full h-3 bg-gradient-to-r from-slate-400 to-slate-600">
 </div>

 </div>

 { /* ЛОГ ДЕЙСТВИЙ */ }
 <div className="bg-slate-900 border border-slate-800 p-4">
 <div className="text-[10px] font-mono text-slate-400">
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between p-2`}
 >
 {idx === 0 ? Действие : }
 {entry}
 </div>
))}
 </div>
 </div>
);
};
```

---

ФАЙЛ 47/94: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

---

```
import React, { createContext, useContext, useEffect, useState } from 'react';

/** Общие примитивы для мини-визуализаций подсказок. */
export const W = 260;
export const H = 130;

/**
 * * Общий множитель скорости. Кадры были слишком короткими,
 * * заметить, что именно переехало, и анимация читалась
```



```

const paused = useContext(DemoPauseContext);
return (
 <div className="w-full">
 <svg viewBox={`0 0 ${W} ${H}`} className="w-full h-full">
 {children}
 </svg>
 {caption && (
 <div className="text-[10px] text-slate-400 text-center">
 {caption}
 </div>
)}
 <div className="text-[9px] text-slate-600 text-center">
 {paused ? "пауза – курсор на карточке" : "наведение"}
 </div>
 </div>
);
};

export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke}>
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
)}
 </g>
);
};

export const Edge: React.FC<{
 a: [number, number];

```

```

 : step === 1
 ? "Разрешаем пересадку через k: 3 + 4 = 7"
 : "7 < 9 → D[i][j] = 7"
 }
 >
 <Edge a={pos.i} b={pos.j} color={relaxed ? C.bad
 <Edge a={pos.i} b={pos.k} color={viaOpen ? C.done
 <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done

 <text x={130} y={112} textAnchor="middle" fontSize=
 9
 </text>
 <text x={72} y={54} textAnchor="middle" fontSize=
 3
 </text>
 <text x={188} y={54} textAnchor="middle" fontSize=
 4
 </text>

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
 </Svg>
);
};

/** Компоненты связности: несколько «островов», которые
export const ComponentsDemo: React.FC = () => {
 const comps: [string, string][][] = [
 ["A", "B"], ["B", "C"],
 ["D", "E"],
 ["F", "G"], ["G", "H"], ["F", "H"],
];
 const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
 };
};

```

```

 <g key={i}>
 <Edge a={a} b={b} color={step >= 1 ? C.active : C.idle}
 {step >= 2 && {
 <text
 x={({a[0] + b[0]) / 2}
 y={({a[1] + b[1]) / 2 - 4}
 textAnchor="middle"
 fontSize={10}
 fontWeight="700"
 fill={C.warn}
 />
 }}
 </g>
);
 }
 }
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={C.idle} />
))}
 </Svg>
);
};

```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];
 const step = useLoop(3, 1200);
 const boxW = 46;
 const startX = (260 - raw.length * (boxW + 4)) / 2;

 return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты – огромные и с дырами"
 : step === 1

```

```

type Pt = [number, number];

const SplayFrames: React.FC<{
 frames: { pos: Record<string, Pt>; edges: [string, string][];
 captions: string[];
 ms?: number;
}> = ({ frames, captions, ms = 1500 }) => {
 const step = useLoop(frames.length, ms);
 const f = frames[step];
 const color = (id: string) =>
 id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "c" ? "#f1c40f" : "#f1c40f";

 return (
 <Svg caption={captions[step]}>
 {f.edges.map(([a, b], i) => (
 <line
 key={i}
 x1={f.pos[a][0]}
 y1={f.pos[a][1]}
 x2={f.pos[b][0]}
 y2={f.pos[b][1]}
 stroke={C.edge}
 strokeWidth={1.8}
 style={{ transition: MOVE }}
 />
))}
 {Object.entries(f.pos).map(([id, [x, y]]) => (
 <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
 <circle r={13} fill={color(id)} stroke={C.idl} />
 <text textAnchor="middle" dominantBaseline="bottom">
 {id}
 </text>
 </g>
))}
 </Svg>
);
};

```

```
import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

const GRAPH_POS: Record<string, [number, number]> = {
 A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
const GRAPH_EDGES: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D", "E"],
];

export const BfsDemo: React.FC = () => {
 const order = ["A", "B", "C", "D", "E", "F"];
 const step = useLoop(order.length + 1, 850);
 const visited = new Set(order.slice(0, step).flat());
 const wave = step > 0 && step <= order.length ? order[step - 1] : null;
 return (
 <Svg caption={`Уровень ${Math.max(0, Math.min(step, order.length))}`}>
 {GRAPH_EDGES.map(([u, v], i) => (
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} />
))}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k}
 fill={wave?.includes(k) ? C.active : visited.has(k) ? C.visited : C.inactive}
 stroke={wave?.includes(k) ? "#a5b4fc" : undefined} />
))}
 </Svg>
);
};

export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={`Путь ${step}`}>
 {path.map((k, i) => (
 <Node key={k} x={GRAPH_POS[k][0]} y={GRAPH_POS[k][1]} label={k}
 fill={i === visited.length - 1 ? C.active : visited.includes(k) ? C.visited : C.inactive}
 stroke={i === visited.length - 1 ? "#a5b4fc" : undefined} />
))}
 {GRAPH_EDGES.map(([u, v], i) => (
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} />
))}
 </Svg>
);
};
```

```

 <text x={n.x + 30} y={n.y - 12} textAnchor="left"
 x + 1}</text>
)}
 </g>
);
})))
<text x={224} y={62} textAnchor="middle" fontSize=16>
<text x={224} y={74} textAnchor="middle" fontSize=16>
</Svg>
);
};

```

```

export const RelaxDemo: React.FC = () => {
 const step = useLoop(3, 1100);
 const dV = [12, 12, 7][step];
 const improved = step === 2;
 return (
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Пусть" : ""}>
 <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"} />
 <text x={130} y={54} textAnchor="middle" fontSize=16>12 > 3 + 4 = ?</text>
 <Node x={60} y={70} r={20} fill={C.active} label="3" />
 <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />
 <text x={60} y={106} textAnchor="middle" fontSize=16>3</text>
 <text x={200} y={106} textAnchor="middle" fontSize=16>4</text>
 <text x={130} y={22} textAnchor="middle" fontSize=16>12</text>
 </Svg>
);
};

```

```

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
 const cut = step === 1;

```

```

 A: [40, 35], B: [130, 22], C: [220, 45], D: [70, 10]
 };
 const edges = [
 { u: "A", v: "B", w: 2 }, { u: "B", v: "C", w: 3 },
 { u: "D", v: "E", w: 4 }, { u: "B", v: "E", w: 6 },
];
 const chosen = ["A-D", "A-B", "B-C", "D-E"];
 const step = useLoop(chosen.length + 1, 800);
 const taken = new Set(chosen.slice(0, step));
 return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без ц
 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>
 <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
 <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y=
 textAnchor="middle" fontSize={9} fontWeigh
 </g>
);
 });
 }}}
 {Object.entries(pos).map(([k, [x, y]]) => (<Node
 </Svg>
);
};

```

```

export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
 };
 const edges: [string, string][] = [["A", "B"], ["B",
 const step = useLoop(2, 1400);
 const scc = ["A", "B", "C"];
 const inScc = (u: string, v: string) => step === 1 &&
 return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
 {edges.map(([u, v], i) => {
 <Edge key={i} a={pos[u]} b={pos[v]} color={inSc

```

```

const popping = step >= 3;
return (
 <Svg caption={popping ? "pop: забираем ВЕРХНИЮ tape"
 <rect x={90} y={22} width={80} height={96} rx={6}
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={94} y={100 - i * 26} width={72} height={26}
 fill={i === items.length - 1 ? (popping ? C.done : C.dim)}
 <text x={130} y={111 - i * 26} textAnchor="middle"
 </g>
)}}
 <text x={130} y={16} textAnchor="middle" fontSize={12} />
 </Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
 const states = [["A", "B"], ["A", "B", "C"], ["B", "C"], ["C"]];
 const step = useLoop(states.length, 800);
 const items = states[step];
 return (
 <Svg caption="pop_front слева, push_back справа – к концу"
 <defs>
 <marker id="mdArrow" markerWidth="6" markerHeight="6"
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХОД</text>
 <text x={206} y={44} fontSize={9} fill={C.dim}>ВХОД</text>
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} height={26}
 fill={i === items.length - 1 ? (popping ? C.done : C.dim)}
 <text x={53 + i * 54} y={71} textAnchor="middle" />
 </g>
)}}
 <line x1={24} y1={71} x2={10} y2={71} stroke={C.dim} />
 </Svg>
);
};

```



```

 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}]
 {TRIE_NODES.map((n) => (<Node key={n.id} x={n.x}
</Svg>
);
};

```

```

export const TrieBfsDemo: React.FC = () => {
 const levels = [[0], [1, 2], [3, 4, 5]];
 const step = useLoop(levels.length + 1, 950);
 const doneIds = new Set(levels.slice(0, step).flat());
 const wave = step > 0 && step <= levels.length ? levels[step - 1] : null;
 return (
 <Svg caption={`BFS по бору: уровень ${Math.max(0, M
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}]
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch}
 fill={wave.includes(n.id) ? C.active : doneId
)})
 <text x={6} y={12} fontSize={9} fill={C.dim}>очер
 </Svg>
);
};

```

```

export const SuffixLinkDemo: React.FC = () => {
 const links: [number, number][] = [[1, 0], [2, 0], [3
 const step = useLoop(links.length + 1, 900);
 return (
 <Svg caption="Суффиксная ссылка: «куда откатиться,
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}]
 {links.slice(0, step).map(([from, to], i) => {
 const a = TRIE_NODES[from];

```

```

const a = [3, 1, 4, 1, 5, 9];
const pref = a.reduce<number[]>((acc, v) => [...acc, v]);
const step = useLoop(4, 1200);
const l = 1, r = 3;
const show = step >= 1;
return (
 <Svg caption={show ? `a[${l}..${r}] = P[${r + 1}] -
 о префиксные суммы P`}>
 <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
 {a.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 39} y={24} width={34} height={12}
 fill={show && i >= l && i <= r ? C.active : C.dim}>
 <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
 </g>
))}
 <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
 {pref.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={12}
 fill={show && i === l ? C.bad : show && i <= r ? C.dim : C.idle}
 stroke={C.idleStroke} style={{ transition: "fill 0.2s" }}>
 <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
 </g>
))}
 </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы для
 {Array.from({ length: 8 }).map((_, i) => {

```

```

 <circle cx={196} cy={84} r={13} fill={step === 1
 <text x={196} y={84} textAnchor="middle" dominantBaseline="middle"
 ? "π"}</text>
 <text x={130} y={124} textAnchor="middle" font-size={12}>
 </Svг>
);
};

```

```

export const PrefixFunctionDemo: React.FC = () => {
 const s = "abacaba";
 const pi = [0, 0, 1, 0, 1, 2, 3];
 const step = useLoop(s.length + 1, 700);
 const i = Math.max(0, step - 1);
 const len = step > 0 ? pi[i] : 0;
 return (
 <Svг caption={step > 0 ? `π[${i}] = ${len}: префикс-функция` : ""}>
 {s.split("").map((ch, idx) => (
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} height={20}
 fill={step > 0 && (idx < len || (idx > i - 1)) ? C.fill : C.dim}
 style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="middle">{ch}</text>
 <text x={32 + idx * 33} y={70} textAnchor="middle">{pi[idx]}</text>
 <rect x={18 + idx * 33} y={70} width={28} height={20}
 fill={idx <= i && step > 0 ? C.warn : C.idle} />
 </g>
))}
 <text x={4} y={40} font-size={8} fill={C.dim}>s</text>
 <text x={4} y={70} font-size={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" font-size={12}>
 π[i] — длина самого длинного «префикс = суффикс»
 </text>
 </Svг>
);
};

```

```

export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;

```

```
} from "./demosStruct";
import {
 ComponentsDemo, CoordCompressDemo, FloydDemo, GraphBa
 ZigDemo, ZigZagDemo, ZigZigDemo,
} from "./demosExtra";

export type DemoKind =
 | "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
 | "suffix-link" | "toposort" | "relax" | "prefix-sum"
 | "segment-tree" | "bridge" | "articulation" | "mst"
 | "prefix-function" | "dp" | "floyd" | "components" |
 | "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
 bfs: BfsDemo,
 dfs: DfsDemo,
 heap: HeapDemo,
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
```

```

}

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
 useEffect(() => {
 if (!vizId) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 document.body.style.overflow = "hidden";
 return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
 };
 }, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
 <div className="fixed inset-0 z-[110] flex items-en
 <div className="absolute inset-0 bg-slate-950/80
 <div className="relative w-full sm:max-w-5xl max-l
 ded-2xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py
 <div className="min-w-0 flex-1">
 <h3 className="font-bold text-white text-sm
 {viz.hint && <p className="text-[11px] text
 </div>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hove
 aria-label="Заккрыть симулятор"
 >
 <X className="w-5 h-5" />
 </button>

```

```

export const TermPopover: React.FC<Props> = ({ anchor, onClose }) => {
 const cardRef = useRef<HTMLDivElement>(null);
 const [pos, setPos] = useState<{ top: number; left: number}>({ top: 0, left: 0 });

 useEffect(() => {
 if (!anchor) {
 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
 const height = cardRef.current?.offsetHeight ?? 300;

 let left = anchor.rect.left + anchor.rect.width / 2;
 left = Math.max(8, Math.min(left, vw - width - 8));

 const below = anchor.rect.top < height + GAP + 8;
 const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - height - GAP;

 setPos({ top: Math.max(8, Math.min(top, vh - height - 8)), left });

 useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onClose, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onClose, true);
 };
 }, [anchor, onClose]);

 if (!anchor) return null;
 }, [anchor, onClose]);

```

```

 {entry.formal}
 </p>
 }}

 {entry.complexity && {
 <div className="text-[11px] font-mono text-
 Сложность: {entry.complexity}
 </div>
 }}

 {viz && simulatorId && {
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-center
 unded-lg py-2 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
 }}
 </div>
</div>
</div>,
document.body
);
};

```

---

ФАЙЛ 54/94: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```

import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizSteps';

export function JohnsonViz() {
 const [step, setStep] = useState(0);

```

```

 if (e.u === 'S') {
 if (step === 0 || step === 7) return "opacity-0";
 if (step === 1) return "stroke-amber-500 stroke-width-2px";
 if (step === 2) return "stroke-pink-500 stroke-width-2px";
 return "stroke-slate-700 stroke-dasharray-4 4 stroke-width-2px";
 }

 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-width-2px";
 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-width-2px";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-width-2px";
 if (step > 6) return "stroke-emerald-500";
 }

 if (step >= 4) return "stroke-slate-600"; // отсюда и дальше
 return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
 if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
 if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
 if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
 if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
 if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";
};

```



```
};

const getDesc = () => {
 switch (step) {
 case 0: return (
 <div>
 Исходный граф. Имеется ребро
 Если мы запустим быстрый алгоритм Д
 </div>
);
 case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соед
ми.
 Это наша база для расчета потенциал
 </div>
);
 case 2: return (
 <div>
 Шаг 2: Ищем потенциалы $h(v)$.
 Запускаем медленный, но мощный алгор
ных вершин:
 <span className="text-pink-300 ml-1
 </div>
);
 case 3: return (
 <div>
 Шаг 3: Магическая формула.
 Всем ребрам графа назначаем новый в
 <div className="text-center font-mo
 Это повышает или понижает вес ребра
 </div>
);
 case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра $A \rightarrow B$.
 Старый вес: <span className="text-r
```

```

nter p-4">
 <svg width="400" height="300" class=
 <defs>
 <marker id="arrow-slate" vie
start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-slate-dim
uto-start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-amber" vie
start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-pink" vie
tart-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 <marker id="arrow-emerald"
o-start-reverse">
 <path d="M 0 0 L 10 5 L
 </marker>
 </defs>

 {/* Edges */}
 {edges.map((e, i) => {
 const u = nodes.find(n => n
 const v = nodes.find(n => n

 const isS = e.u === 'S';
 if (isS && (step === 0 || s

 const midX = (u.x + v.x) / 2;
 const midY = (u.y + v.y) / 2;

 const edgeColorClass = getE
 const textHigh = isEdgeText

```

```

 />
 <text x={n.x} y={n.y}
- white">
 {n.label}
 </text>

 {pot !== null && (
 <g transform={`
fade-in zoom-in duration-500">
 <rect x="-1
k-500 stroke-1" />
 <text x="0"
font-bold fill-pink-400">
 {pot}
 </text>
 </g>
)}
 </g>
)
 }}}}
</svg>
</div>

<div className="w-full md:w-[320px] flex
 /* Log Box */
 <div className="bg-[#0f172a] p-5 ro
l justify-center leading-relaxed text-sm te
 {getDesc()}
 </div>

 /* Controls */
 <div className="flex bg-slate-900 b
 <button onClick={() => setStep(1
 className="p-3 bg-slate-800
-slate-400 transition-colors" title="Сброси
 <RotateCcw strokeWidth={3}
 </button>
 <button onClick={() => setStep(1

```

```

 RotateCcw,
 Layers,
} from "lucide-react";

interface GNode {
 id: number;
 x: number;
 y: number;
 label: string;
}

interface GEdge {
 u: number;
 v: number;
}

const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];

interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
 desc: string;
 visited: Set<number>;
 currentNode: number | null;

```

```

 visited: new Set(visited),
 currentNode,
 order: [...order],
 transposed,
 sccMap: { ...sccMap },
 currentSccId,
 activeEdges: transposed
 ? initialEdges.map((e) => ({ u: e.v, v: e.u }
 : [...initialEdges],
));
 });
};

recordStep(
 "init",
 "Инициализация алгоритма Косарайю. Начинаем первый",
 null,
 false,
 -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение`,
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }
};

```

```

const dfs2 = (u: number, currentScc: number) => {
 visited.add(u);
 sccMap[u] = currentScc;
 recordStep(
 "dfs2",
 `DFS 2: заходим в ${u} на транспонированном графе`,
 u,
 true,
 currentScc,
);

 for (const to of adjT[u]) {
 if (!visited.has(to)) {
 dfs2(to, currentScc);
 }
 }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека`,
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в ${sccMap[u]}`,
 u,
 true,
 sccId,
);
 }
}

```

```

 }, [isPlaying, currentStepIdx, steps.length]));

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getScColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
};

return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
 </h3>

 <div className="flex items-center gap-2 bg-slat
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 d-500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}

```

```

 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#666666" />
 </marker>
</defs>

```

```

{/* Edges */}
{initialEdges.map((edge, idx) => {
 const u = initialNodes.find((n) => n.id === edge.u)
 const v = initialNodes.find((n) => n.id === edge.v)

 // If transposed, draw reversed coordinates
 const x1 = step.transposed ? v.x : u.x;
 const y1 = step.transposed ? v.y : u.y;
 const x2 = step.transposed ? u.x : v.x;
 const y2 = step.transposed ? u.y : v.y;

 const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v]
 let stroke = "#475569";
 let markerId = "arrow";

 if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f46e5";
 markerId = "arrow-transposed";
 } else {
 if (
 step.currentNode === edge.u ||
 step.currentNode === edge.v
) {
 stroke = "#10b981";
 }
 }
})}

```



```

 } else if (isVisited && step.phase === "done") {
 classes = "fill-emerald-700 stroke-emerald-700"
 }

```

```

return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r="20"
 className={`transition-all duration-300`}
 strokeWidth="3"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && (
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
)}
 </g>
)

```

```

 key={idx}
 className="bg-indigo-950 border border-
ex items-center gap-1"
 >
 {val}
 </div>
))
 }}
</div>
</div>

{/* List of actions log */}
<div>

 ЛОГ ОПЕРАЦИЙ:

 <div className="space-y-1.5 max-h-[150px]">
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map((s, idx) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded
 >
 {s.desc}
 </div>
)}}
 </div>
 </div>
</div>

<div className="mt-4 pt-3 border-t border-slate-500">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">

```

```

 color: string; // 'none', 'red', 'blue', 'purple', 'm
}

```

```

interface Edge {
 id: string;
 u: number;
 v: number;
 weight: number;
 status: 'pending' | 'inspecting' | 'included' | 'reje
 color: string;
}

```

```

interface StepLog {
 title: string;
 description: string;
 type: 'info' | 'success' | 'warning' | 'error';
}

```

// 9 Vertices layout

```

const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

```

// 12 Edges connecting the 9 vertices

```

const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },

```

```

 return edges.map((e) => (e.id === target.id && e.st
 }));
 const compilerLinked = liveI !== null || liveW !== nu

const [logs, setLogs] = useState<StepLog[]>([
 {
 title: 'Введение в раскраску (Краскал)',
 description: '«Краскал – это раскраска?». Предста
 овы начинать!',
 type: 'info',
 },
]);

// --- KRUSKAL STEPS ---
const kruskalSteps = [
 // Step 1: Inspect A-B (2)
 () => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 1: Берем самое дешевое ребро A-B (в
 description: 'Оно соединяет две бесцветные верши
 type: 'info'
 }, ...prev]));
 },
 // Step 2: Include A-B
 () => {
 setVertices(prev => prev.map(v => v.id === 0 || v
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро A-B в осто́ве',
 description: 'Вершины A and B окрашены в красны
 type: 'success'
 }, ...prev]));
 },
 // Step 3: Inspect D-E (3)
 () => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{

```

```

 title: 'Шаг 4: Берем ребро A-D (вес 5)',
 description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
 ю – мы перекрашиваем всё в один цвет!"',
 type: 'warning'
 }, ...prev]);
},
// Step 8: Include A-D (Merge to purple)
() => {
 setVertices(prev => prev.map(v => v.color === 'blue' ? 'purple' : v.color));
 setEdges(prev => prev.map(e => e.status === 'included' ? 'excluded' : e.status));
 setLogs(prev => [{
 title: 'Слияние кланов: Ребро A-D добавлено',
 description: 'Мы объединили красный и синий кланы',
 type: 'success'
 }, ...prev]);
},
// Step 9: Inspect B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {status: 'excluded'} : e));
 setLogs(prev => [{
 title: 'Шаг 5: Берем ребро B-E (вес 6)',
 description: 'НО: это ребро соединяет вершины B и E, которые уже соединены! Это создаст цикл!',
 type: 'warning'
 }, ...prev]);
},
// Step 10: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {status: 'excluded'} : e));
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро B-E. Иначе появится цикл!',
 type: 'error'
 }, ...prev]);
},
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {status: 'excluded'} : e));
 setLogs(prev => [{

```

```

 title: 'Шаг 8: Берем ребро C-F (вес 9)',
 description: 'Оно соединяет вершины C и F, кото
 type: 'warning'
 }, ...prev]));
},
// Step 16: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро C-F.',
 type: 'error'
 }, ...prev]));
},
// Step 17: Inspect G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Берем ребро G-H (вес 10)',
 description: 'Соединяет фиолетовую G и бесцветн
 type: 'info'
 }, ...prev]));
},
// Step 18: Include G-H
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро G-H в осто́ве',
 description: 'Вершина H перекрашена в фиолетовы
 type: 'success'
 }, ...prev]));
},
// Step 19: Inspect E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',

```

```

() => {
 setVertices(prev => prev.map(v => v.id === 4 ? {
 setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E-F (7)'],
 setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5']),
 setLogs(prev => [{
 title: 'Шаг 1: Плесень зарождается в Токио (Вершина E)',
 description: 'Мы выбрали вершину E как стартовую вершину. В приоритетную очередь (Heap): D-E(3), B-E(6), E-F(7)',
 type: 'info'
 }, ...prev]));
},
// Step 2: Pop D-E (3)
() => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Куча выдает самое дешевое ребро D-E (3)',
 description: 'Плесень тянется по ребру D-E к вершине D',
 type: 'info'
 }, ...prev]));
},
// Step 3: Include D-E, add D's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 ...e, status: 'in_heap' } : e));
 setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D-E (3)'],
 setLogs(prev => [{
 title: 'Успех: Плесень захватила вершину D',
 description: 'В кучу добавлены новые пути из D: A-D (5), B-E (6), E-F (7)',
 type: 'success'
 }, ...prev]));
},
// Step 4: Pop A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону вершины A'
 }

```

```

 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
 description: 'Плесень тянется к вершине C.',
 type: 'info'
 }, ...prev]));
},
// Step 9: Include B-C, add C's edges
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 eap' } : e));
 setHeapQueue(['B-E (6 - цикл)', 'E-F (7)', 'D-G (8)']);
 setLogs(prev => [{
 title: 'Успех: Вершина C захвачена',
 description: 'Ребро C-F(9) добавлено в кучу.',
 type: 'success'
 }, ...prev]));
},
// Step 10: Pop B-E (6) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает ребро B-E (вес 6)',
 description: 'ВНИМАНИЕ! Плесень проверяет вершину E.',
 type: 'warning'
 }, ...prev]));
},
// Step 11: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'B-E (6)']);
 setLogs(prev => [{
 title: 'Отказ: Игнорируем внутреннее ребро',
 description: 'Ребро B-E отброшено. Плесень не р
 type: 'error'
 }, ...prev]));
},

```



```

 description: 'В кучу добавлено G-H(10).',
 type: 'success'
 }, ...prev]);
},
// Step 16: Pop C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
 description: 'Вершины C и F уже заражены плесенью',
 type: 'warning'
 }, ...prev]));
},
// Step 17: Reject C-F
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setHeapQueue(['G-H (10)', 'E-H (11)', 'F-I (13)'])
 setLogs(prev => [{
 title: 'Отказ: Игнорируем ребро C-F',
 description: 'Выбрасываем из кучи.',
 type: 'error'
 }, ...prev]));
},
// Step 18: Pop G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
 description: 'Плесень ползет к вершине H.',
 type: 'info'
 }, ...prev]));
},
// Step 19: Include G-H, add H's edges
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I

```

```

 setHeapQueue(['F-I (13 - цикл)']);
 setLogs(prev => [{
 title: ' Успех: Плесень захватила весь граф
 description: 'Все 9 вершин поглощены. Итоговый
 иной – мы разрастались из одной точки с помо
 type: 'success'
 }, ...prev]);
 },
];

// --- BORUVKA STEPS ---
const boruvkaSteps = [
 // Step 1: Five-Year Plan 1 (Inspecting outgoing)
 () => {
 // Highlight the cheapest outgoing edges for EACH
 // Node 0(A) -> A-B(2)
 // Node 1(B) -> A-B(2)
 // Node 2(C) -> B-C(4)
 // Node 3(D) -> D-E(3)
 // Node 4(E) -> D-E(3)
 // Node 5(F) -> E-F(7)
 // Node 6(G) -> D-G(8)
 // Node 7(H) -> G-H(10)
 // Node 8(I) -> H-I(12)
 setEdges(prev => prev.map(e =>
 ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8
));
 setLogs(prev => [{
 title: 'Первая пятилетка: Каждая деревня выбира
 description: 'Каждая из 9 деревень одновременно
 А выбирает A-B(2), D выбирает D-E(3), а G вы
 !',
 type: 'info'
 }, ...prev]);
 },
 // Step 2: Concurrently merge into Kolkhozes (Compo
 () => {
 // Concurrently build chosen roads. Merge into:

```

```

 setEdges(prev => prev.map(e =>
 e.status === 'included' || e.id === '0-3' ? { .
 }));
 setLogs(prev => [{
 title: 'Объединение завершено: Один гигантский
 description: 'Оба колхоза объединились по мосту
 type: 'success'
 }, ...prev]);
 },
 // Step 5: Check remaining edges & mark as rejected
 () => {
 setEdges(prev => prev.map(e => e.status === 'pend
 setLogs(prev => [{
 title: ' Успех: MST построен Борувкой за 2 шаг
 description: 'Все лишние ребра (В-Е, Е-Н, С-Е,
 вес: 51. Благодаря параллельности, мы потрати
 type: 'success'
 }, ...prev]);
 }
];

const currentSteps =
 activeAlgo === 'kruskal' ? kruskalSteps :
 activeAlgo === 'prim' ? primSteps :
 boruvkaSteps;

const handleNextStep = () => {
 if (stepIndex < currentSteps.length) {
 currentSteps[stepIndex]();
 setStepIndex(stepIndex + 1);
 }
};

const handleReset = (algo: 'kruskal' | 'prim' | 'boru
 setActiveAlgo(algo);
 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(0);

```

```

 return 'bg-slate-700 border-emerald-500 text-slate-50';
 }
 return 'bg-slate-700 border-slate-500 text-slate-50';
}
};

```

```

const getEdgeStroke = (status: string, color: string) => {
 if (status === 'inspecting') return '#f59e0b';
 if (status === 'rejected') return '#ef4444';
 if (status === 'in_heap') return '#0284c7';
 if (status === 'included') {
 if (color === 'red') return '#ef4444';
 if (color === 'blue') return '#3b82f6';
 if (color === 'purple') return '#a855f7';
 if (color === 'mold') return '#10b981';
 }
 return '#475569';
};

```

```

return (
 <div className="space-y-12">

 {/* 3-in-1 MST Simulator Box */}
 <div className="bg-slate-900/90 border border-slate-800" >
 <div className="flex flex-col lg:flex-row justify-between" >
 <div>
 <div className="flex items-center gap-2 mb-2" >

 Супер-Интерактив 3 в 1

 <h3 className="text-2xl font-bold text-white" >
 MST Simulator
 </h3>
 </div>
 <p className="text-slate-400 text-sm" >
 Наблюдай за работой Краскала (Раскраска),
 </p>
 </div>
 </div>
 </div>
 </div>
)

```

```

 >
 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)
 </button>
 </div>

 <button
 onClick={() => handleReset(activeAlgo)}
 className="flex items-center justify-cent
-200 font-semibold text-xs rounded-xl borde
 >
 <RotateCcw className="w-3.5 h-3.5" />
 Сбросить
 </button>

 <button
 onClick={handleNextStep}
 disabled={stepIndex >= currentSteps.length}
 className={
 "flex items-center justify-center gap-2
nitial cursor-pointer " +
 {stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border
 : activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500
 : activeAlgo === 'prim'
 ? "bg-emerald-600 hover:bg-emerald-500
 : "bg-rose-600 hover:bg-rose-500 text
 }
 >
 <SkipForward className="w-4 h-4" />
 {stepIndex === 0 ? 'Начать' : stepI
 </button>
 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3
 {/* Graph Canvas */}
```



```

 }}
 </div>
)))
 </div>

 {stepIndex >= currentSteps.length && (
 <div className="absolute inset-x-6 bottom
 shadow-xl z-20">
 <p className={"font-bold text-base " +
 'ld-300' : 'text-rose-300'}>
 MST построено с помощью {activeAlgo}
 </p>
 <p className="text-slate-300 text-xs mt
 Общий вес минимального остова: 51. Вс
 </p>
 </div>
)}
</div>

{/* Logs / Heap */}
<div className="bg-slate-950 rounded-2xl border
 <div className="p-4 bg-slate-900 border-b b
 <h4 className="font-bold text-slate-200 t
 Ход выполнения
 </h4>
 </div>

 {activeAlgo === 'prim' && (
 <div className="p-3 bg-slate-900/60 borde
 <p className="text-[11px] font-bold tex
 ✂ Приоритетная очередь (Min-Heap):
 </p>
 <div className="flex flex-wrap gap-1.5":
 {heapQueue.length === 0 ? (
 <span className="text-xs text-slate
) : (
 heapQueue.map(({item, idx}) => (
 <span key={idx} className={"px-2

```

```

 /* Cheat Sheet: Complexity & Code for all 3 algo
 <div className="bg-slate-900/80 border border-sla
 <div className="flex flex-col sm:flex-row items
 <div className="flex items-center gap-2.5">
 <BookOpen className="w-5 h-5 text-indigo-400
 <h4 className="text-xl font-bold text-white
 </div>

 <div className="flex bg-slate-950 p-1 rounded
 <button
 onClick={() => setActiveCheatTab('kruskal
 className={"px-3 py-1.5 rounded text-xs f
 == 'kruskal' ? "bg-indigo-600 text-white" : "
 >
 Краскал (Билет 18)
 </button>
 <button
 onClick={() => setActiveCheatTab('prim')}
 className={"px-3 py-1.5 rounded text-xs f
 == 'prim' ? "bg-emerald-600 text-white" : "
 >
 Прима (Билет 19)
 </button>
 <button
 onClick={() => setActiveCheatTab('boruvka
 className={"px-3 py-1.5 rounded text-xs f
 == 'boruvka' ? "bg-rose-600 text-white" : "
 >
 Борувка (Билет 20)
 </button>
 </div>
 </div>

 {activeCheatTab === 'kruskal' && (
 <div className="grid grid-cols-1 lg:grid-cols
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-

```



```
return False
```

```
2. Жадный выбор ребер
```

```
edges.sort() # $O(E \log E)$
```

```
mst = []
```

```
for w, u, v in edges:
```

```
 if union(u, v):
```

```
 mst.append((u, v, w))`}
```

```
 </pre>
```

```
 </div>
```

```
 </div>
```

```
 </div>
```

```
}}
```

```
{activeCheatTab === 'prim' && {
```

```
 <div className="grid grid-cols-1 lg:grid-cols-
```

```
 <div className="lg:col-span-1 space-y-4">
```

```
 <div className="bg-slate-950 p-4 rounded-
```

```
 <p className="text-slate-400 text-xs for
```

```
 <Clock className="w-3.5 h-3.5 text-em
```

```
 </p>
```

```
 <p className="text-2xl font-black text-v
```

```
 <p className="text-slate-400 text-xs mt
```

```
 </div>
```

```
 <div className="bg-slate-950 p-4 rounded-
```

```
 <p className="text-slate-400 text-xs for
```

```
 <Award className="w-3.5 h-3.5 text-em
```

```
 </p>
```

```
 <p className="text-2xl font-black text-v
```

```
 <p className="text-slate-400 text-xs mt
```

```
 </div>
```

```
 <div className="bg-slate-950 p-4 rounded-
```

```
 <p className="text-slate-400 text-xs for
```

```
 ложении:</p>
```

```
 <p className="text-xs text-slate-300 le
```

```
 ие чистые вершины через самое дешевое досту
```

```
 </div>
```

```
 </div>
```



```

 alt: 'Фура по болоту Форд-Беллман',
 title: ' Фура по Болоту (Ф-Б)',
 desc: 'Медленный, тяжёлый — зато тащит',
 highlight: 'отрицательные',
 highlightColor: 'text-rose-400',
 rest: 'веса и детектит отрицательные циклы.',
 complexity: 'O(V · E)',
 border: 'border-rose-500/30 hover:border-rose-400/60',
 titleColor: 'text-rose-400',
 badge: 'text-rose-400/70 bg-rose-950/40',
 },
 {
 img: floydImg,
 alt: 'Все-ко-Все Флойд',
 title: ' Все ко Все (Флойд)',
 desc: 'Матрица + три цикла (',
 highlight: 'k, i, j',
 highlightColor: 'text-emerald-400',
 rest: ')). Ищет пути от всех ко всем.',
 complexity: 'O(V³)',
 border: 'border-emerald-500/30 hover:border-emerald-400/60',
 titleColor: 'text-emerald-400',
 badge: 'text-emerald-400/70 bg-emerald-950/40',
 },
];

```

```

export default function MnemonicCards() {
 return (
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {cards.map((c, i) => (
 <div key={i} className={`bg-slate-800 rounded-2xl p-4`}>
 <div className="h-48 overflow-hidden">
 <img src={c.img} alt={c.alt}
 className="w-full h-full object-cover group-hover:scale-110" />
 </div>
 <div className="p-4">
 <h3 className={`text-lg font-bold mb-1 ${c.titleColor}`}>{c.title}</h3>
 <p className="text-slate-400 text-sm">{c.desc}</p>
 <p className="text-slate-400 text-sm">{c.highlight}</p>
 <p className="text-slate-400 text-sm">{c.rest}</p>
 <p className="text-slate-400 text-sm">{c.complexity}</p>
 <p className="text-slate-400 text-sm">{c.badge}</p>
 </div>
 </div>
))}
 </div>
);
}

```

```

return () => {
 document.body.style.overflow = "";
};
}, [open]));

useEffect(() => {
 const onKey = (e: KeyboardEvent) => e.key === "Escape";
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, []);

return (
 <>
 <div className="lg:hidden sticky top-[57px] z-40">
 <button
 onClick={() => setOpen(true)}
 className="w-full flex items-center gap-2.5 p-2"
 aria-label="Открыть содержание"
 >

 <List className="w-4 h-4" />

 Содержание · {index + 1} из {total}

 {tocIn_.18s_ease-out}

 </button>
 </div>

 {open && (
 <div className="lg:hidden fixed inset-0 z-[90]">
 <div className="absolute inset-0 bg-slate-950">
 <div className="absolute inset-y-0 left-0 w-[100%] h-[100%] flex flex-direction column-reverse">
 <div className="flex items-center justify-between p-4">
 Содержание
 </div>
 <div className="flex items-center justify-between p-4">
 1 из 1
 100%
 </div>
 </div>
 </div>
 </div>
)}
 </>
)

```

```
export const Navbar: React.FC<NavbarProps> = ({ compile
 const [busy, setBusy] = useState(false);

 const saveBook = async () => {
 setBusy(true);
 try {
 await downloadBookHtml(chapters);
 } finally {
 setBusy(false);
 }
 };

 return (
 <header className="bg-slate-900 border-b border-sla
 <div className="max-w-7xl mx-auto px-4 sm:px-6 lg
 ap-3 lg:gap-0">
 <div className="flex items-center gap-3 w-full
 <div className="bg-gradient-to-tr from-indigo
 k-0">
 <Sparkles className="w-6 h-6" />
 </div>
 <div className="min-w-0 flex-1">
 <h1 className="text-xl font-extrabold text-
 Универсальное пособие
 </h1>
 <p className="text-xs text-indigo-400 font-
 Подготовка к экзаменам: Алгоритмы, Структ
 </p>
 </div>
 </div>

 <div className="flex items-center w-full lg:w-a
 >
 <button
 id="tab-guide-btn"
 onClick={() => window.scrollTo({ top: 0, be
 className="hidden sm:flex flex-1 lg:flex-no
```

```
 className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald-500
title="Скачать полный PDF сборник всех страниц"
>
 <FileDownload className="w-4 h-4" /> Скачать PDF

 <a
 id="download-txt-btn"
 href={` ${import.meta.env.BASE_URL}export/full_
download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center justify-center
 :bg-sky-600/20 border border-sky-500/30 tra
 title="Скачать полный TXT файл со всем кодом"
 >
 <FileText className="w-4 h-4" /> TXT

 <button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
 er:bg-amber-600/20 border border-amber-500/30 tra
 title="Скачать всё пособие одним автономным файлом"
 >
 {busy ? <Loader2 className="w-4 h-4 animate-spin" /> : "Скачать"}
 </button>
</div>
</div>
</header>
);
};
```

```

const crosses: number[] = [];
for(let i=0;i<edges.length;i++) {
 for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 }
}
const lines = []; const circles = [];
for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing = crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x} y1={p1.y} y2={p2.y}
 stroke={xing ? "#f43f5e" : "#4f46e5"} />);
}
const labels = ["A","B","C","D","E"];
for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" />
 <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
 </g>);
}
return <>{lines}{circles}</>;
})();
</svg>
<div className="absolute bottom-2 left-2 right-2 top-20" z-20">
 V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

{/* K3,3 */}
<div className="bg-slate-950 rounded-xl p-4 border" z-10">
 <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0" z-10">
 {(() => {

```

```

 <text x={p.x} y={p.y+3} textAnchor="middle">
 </g>);
 }
 return <>{lines}{circles}</>;
 })()}
</svg>
<div className="absolute bottom-2 left-2 right-2
 z-20">
 V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
</div>
</div>
</div>

<div className="bg-amber-950/40 border border-amber-950">
 ▲ Запомни: K5 и K3,3 — два кита непланарности.
 гомеоморфный K5 или K3,3 — он непланарен. Теорема
</div>
</div>
);
}

```

ФАЙЛ 61/94: src/components/PythonCompiler.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useCallback, useEffect, useMemo, useRef, useState } from 'react';
import {
 ArrowRightLeft,
 Check,
 CheckCircle2,
 ChevronLeft,
 ChevronRight,
 Copy,
 Eye,
 HelpCircle,
 Link2,

```



```

onWidthChange?: (w: number) => void;
onHeightChange?: (h: number) => void;
/** Перейти к странице пособия (посмотреть визуализацию) */
onOpenGuide: () => void;
/** Скрыть панель. */
onClose: () => void;
}

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_PATH)
 .then((module as PyodideModule).loadPyodide({ indexURL:
 PYODIDE_MODULE_PATH
 }));
 }
 return runtimePromise;
}

function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}

/* -----
/* Шаблон синхронизации: короткая шапка + минимальный код
/* скелет, шаг-в-шаг с подсветкой визуализации (см. data-visualizer.html)

```

```

 },
 {
 label: "while ...",
 tip: "Классический while-счётчик — как n-1 итерация",
 code: "i = 1\nwhile i < 5:\n print(f\"итерация {i}\")\n",
 },
 {
 label: "for i, x in enumerate(...)",
 tip: "Индекс и элемент одновременно: i — позиция, x — элемент",
 code: "for i, x in enumerate(['a', 'b', 'c']):\n print(f\"{i}: {x}\")\n",
 },
],
},
{
 group: "Структуры",
 items: [
 {
 label: "Стек (список)",
 tip: "LIFO, как рекурсия DFS: append = push, pop = pull",
 code: 'st = []\nst.append("A") # push\nst.pop() # pull\n',
 },
 {
 label: "Очередь (deque)",
 tip: "FIFO для BFS: popleft() за O(1). list.pop(0) за O(n)",
 code: 'from collections import deque\nq = deque()\nq.append(1)\nq.popleft()\n',
 },
 {
 label: "Куча (heapq)",
 tip: "Мин-куча: (вес, вершина) — как очередь приоритетов",
 code: 'import heapq\nh = []\nheapq.heappush(h, (1, 'a'))\nheapq.heappush(h, (2, 'b'))\nheapq.heappop(h)\n',
 },
 {
 label: "Матрица n × m",
 tip: "Таблица расстояний: dist[i][j], бесконечность — float('inf')",
 code: 'n, m = 3, 4\ndist = [[float("inf")] * m for _ in range(n)]\n',
 },
],
},

```

```

 });
 const syncVarSet = useMemo(() => new Set(syncVarBases)
 /** Полный эталон: глаз сразу вставляет его в обычный
 const template = useMemo(() => buildSyncTemplate(char
 /** Дефолт редактора: только инициализация демо; она
 const initTemplate = useMemo(() => buildInitTemplate(

 const [code, setCode] = useState(() => {
 if (!savedEditor) return initTemplate;
 if (savedEditor.code === savedEditor.template || sa
 return savedEditor.code;
 });
 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Код выполнится
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(fals
 const [runtimeMessage, setRuntimeMessage] = useState(
 /** Принудительный повтор (глаз / Ctrl+Enter), даже е
 const [traceRequest, setTraceRequest] = useState(0);
 const [copied, setCopied] = useState(false);
 const [stripTab, setStripTab] = useState<"vars" | "hi
 const [outputOpen, setOutputOpen] = useState(true);
 const editorRef = useRef<HTMLTextAreaElement>(null);

 const [desyncedFrom, setDesyncedFrom] = useState<stri
 savedEditor &&
 savedEditor.chapterId !== chapterId &&
 savedEditor.code !== savedEditor.template &&
 savedEditor.code.trim() !== "" &&
 savedEditor.code !== template &&
 savedEditor.code !== initTemplate
 ? chapterId
 : null
);
 const prevTplRef = useRef({ base: initTemplate, full:

 /** Активная трасса. input нужен, чтобы изменение std
 const [debug, setDebug] = useState<{ result: DebugRes

```

```

 }
 // eslint-disable-next-line react-hooks/exhaustive-
 }, [initTemplate, template]);

```

```

const resync = useCallback(() => {
 prevTplRef.current = { base: initTemplate, full: te
 setCode(initTemplate);
 setDesyncedFrom(null);
 setStripTab("vars");
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 setTraceRequest((n) => n + 1);
}, [initTemplate, template]);

```

```

const copyCode = useCallback(async () => {
 try {
 await navigator.clipboard.writeText(code);
 setCopied(true);
 setTimeout(() => setCopied(false), 1600);
 } catch {
 setRuntimeMessage("Не получилось скопировать – бу
 }
}, [code]);

```

/\* — Пошаговый отладчик —————

```

/** Связь больше не зависит от эталонных строк: совпа
const hasViz = sync.variables.length > 0;

```

```

/**

```

```

* Снимок ПОСЛЕ выделенной строки. Контейнеры берём и
* (там уже виден append/st[i][j]=...), но индексы цикл
* события: после st[i][j]= Python уже мог увеличить
*/

```

```

const snapshotAt = useCallback((result: DebugResult,
 const step = result.steps[idx];
 const next = result.steps[idx + 1];

```

```

 variables: snapshot.variables,
 changed: snapshot.changed,
 });
},
[chapterId, hasViz, snapshotAt]
);

// Единственная точка публикации: любое перемещение т
// визуализации согласованный снимок, без side-effect
useEffect(() => {
 if (debug) publishDebug(debug.result, debug.idx, del
}, [debug, publishDebug]);

const goDebug = useCallback(
 (nextIdx: number) => {
 setDebug((d) => {
 if (!d || d.result.steps.length === 0) return d
 const idx = Math.max(0, Math.min(d.result.steps
 return { ...d, idx };
 });
 },
 []
);

const stepBy = useCallback(
 (delta: number) => {
 setDebug((d) => {
 if (!d || d.result.steps.length === 0) return d
 const idx = Math.max(0, Math.min(d.result.steps
 return { ...d, idx };
 });
 },
 []
);

const stepTo = useCallback((idx: number) => goDebug(i

const stopDebug = useCallback(() => {

```

```

runtime.setStdout({ batched: (message) => stdout.log(message) });
runtime.setStderr({ batched: (message) => stderr.log(message) });
runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });

const raw = await runtime.runPythonAsync(buildDebugRequest({ code, stdin, traceRequest }));
const result = JSON.parse(String(raw)) as DebugResult;

setDebug({ result, idx: 0, src: source, input, result.truncated });
setPlaying(autoPlay && result.steps.length > 1);
const printed = [...stdout, ...stderr].join("\n");
setOutput(printed || "Программа ничего не вывела");
setRuntimeMessage(
 result.truncated
 ? `Выполнение остановлено после ${DEBUG_STEP_LIMIT} шагов`
 : "Трасса готова автоматически. ← → — шаги, Ctrl+Enter — продолжить",
);
} catch (error) {
 const message = errorText(error);
 setDebug({ result: { steps: [], error: message, truncated: true }, idx: 0, src: source, input });
 setOutput(`Ошибка:\n${message}`);
 setRuntimeMessage("Не удалось выполнить код — произошла ошибка");
} finally {
 setRunning(false);
}
}, [chapterId, code, stdin, traceRequest, running, runtimeReady]);

// Код выполняется сам: debounce даёт спокойно допечать результат
// изменился во время загрузки Pyodide, после завершения загрузки
useEffect(() => {
 if (running || editHold === code) return;
 if (debug?.src === code && debug.input === stdin && debug.result.steps.length > 0) {
 const timer = window.setTimeout(() => void debugRun(), 100);
 return () => window.clearTimeout(timer);
 }
}, [code, stdin, traceRequest, running, runtimeReady]);

// Стрелки ← → в режиме отладки листают ШАГИ (а не строки)
useEffect(() => {
 if (!debug) return;

```

```

// производные данные шага: что изменилось, в каком п
// Строка, которую назначает текущий шаг, подсвечивае
// локали берём со следующего шага трассы (для послед
// трейсером на возврате из фрейма). Значит на строке
const shownLocals = useMemo(() => {
 if (!debug || !curDebug?.step) return {};
 return snapshotAt(debug.result, debug.idx, debug.sr
}, [debug, curDebug, snapshotAt]);
const debugChanged = curDebug?.step ? changedVars(cu
const debugOrdered = curDebug?.step ? orderVars(shown

/** Одна и та же кнопка запуска всегда видна: из реда
 * проигрывает её, в отладчике переключает play/pause
const runOrTogglePlayback = useCallback(() => {
 if (running) return;
 if (!debug || debug.result.steps.length === 0) {
 void debugRun(true);
 return;
 }
 if (playing) {
 setPlaying(false);
 return;
 }
 if (debug.idx >= debug.result.steps.length - 1) ste
 setPlaying(true);
}, [debug, debugRun, playing, running, stepTo]);

/** Вставка сниппета из шпаргалки в позицию курсора.
const insertSnippet = useCallback(({snippet: string}) => {
 const ta = editorRef.current;
 setCode((current) => {
 if (!ta) return current + (current.endsWith("\n")
 const start = ta.selectionStart ?? current.length
 const end = ta.selectionEnd ?? start;
 const padded = (start > 0 && !current.slice(0, st
 const next = current.slice(0, start) + padded + c
 requestAnimationFrame(() => {
 ta.focus();
 });
 });

```

```

 el.removeEventListener("pointercancel", onUp as EventListener);
 };
 el.addEventListener("pointermove", onMove as EventListener);
 el.addEventListener("pointerup", onUp as EventListener);
 el.addEventListener("pointercancel", onUp as EventListener);
 }}
>
 <div className="absolute left-1/2 top-1/2 h-10 w-10
 digo-400 transition-colors" />
</div>
))

{/* Левый край меняет ширину на десктопе. */}
{onWidthChange && (
 <div
 role="separator"
 aria-orientation="vertical"
 aria-label="Потяните, чтобы изменить ширину панели"
 title="Потяните, чтобы изменить ширину панели"
 className="hidden lg:block absolute top-0 bottom-0 left-0
 onPointerDown={(e) => {
 const el = e.currentTarget;
 el.setPointerCapture(e.pointerId);
 const startX = e.clientX;
 const startW = width ?? 560;
 const onMove = (ev: PointerEvent) => {
 const next = Math.round(startW + (startX - e.clientX) * 2);
 onWidthChange(Math.max(380, Math.min(next, 1000)));
 };
 const onUp = () => {
 el.removeEventListener("pointermove", onMove as EventListener);
 el.removeEventListener("pointerup", onUp as EventListener);
 };
 el.addEventListener("pointermove", onMove as EventListener);
 el.addEventListener("pointerup", onUp as EventListener);
 }}
 >
 <div className="absolute left-1/2 top-1/2 -tr

```





```

 >
 <Variable className="w-3.5 h-3.5" /> Перемен
 </button>
 <button
 type="button"
 onClick={() => setStripTab("hints")}
 className={`inline-flex items-center gap-1.5
 stripTab === "hints" ? "bg-slate-800 text-white"
 : ""
 }`
 >
 <HelpCircle className="w-3.5 h-3.5" /> Подсказка
 </button>
 <div className="ml-auto pr-1">
 <Tooltip
 side="bottom"
 content={
 hasViz
 ? "Связь идёт по именам ниже: реальный узел
а не сравнивается."
 : "На выбранной странице нет интерактивных элементов"
 }
 >
 <span
 tabIndex={0}
 className={`cursor-help inline-flex items-center gap-1.5
 hasViz ? "text-emerald-400" : "text-slate-400"
 }`
 >
 <Link2 className="w-3 h-3" />
 {hasViz ? "синхронизировано" : "нет визуализации"}

 </Tooltip>
 </div>
 </div>

 {stripTab === "vars" ? (
 <div className="px-3 pb-2.5 space-y-2 max-h-300">
 <div className="flex items-center gap-2 min-h-300">

```

```


 {v.role}
 {v.range && {v.range}}

 }
 >
 <code
 tabIndex={0}
 className="cursor-help rounded-md font-bold text-emerald-300 hover:bg-emerald-500"
 >
 {v.name}
 </code>
 </Tooltip>
)))
{sync.stepNote && {
 <Tooltip side="bottom" content={`War`}
 <span
 tabIndex={0}
 className="cursor-help inline-flex text-[10px] text-slate-400 hover:text-white"
 >
 <ArrowRightLeft className="w-3 h-3" />
 {v.note}

 </Tooltip>
}}
</div>
)}
</div>
) : (
 <div className="px-3 pb-2.5 max-h-36 overflow-y-auto">
 {SNIPPETS.map(({gItem}) => {
 <div key={gItem.group}>
 <div className="text-[9px] font-bold uppercase">{gItem.group}</div>
 <div className="flex flex-wrap gap-1.5">
 {gItem.items.map(({s}) => {
 <Tooltip key={s.label} side="bottom"

```

```

 </button>
 </Tooltip>
 <Tooltip content="В начало трассы">
 <button type="button" onClick={() => {
md text-slate-400 hover:text-white hover:bg
 <SkipBack className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="Шаг назад (←) – автопрогн
 <button type="button" onClick={() => {
-md text-slate-400 hover:text-white hover:b
 <ChevronLeft className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="Шаг вперёд (→) – автопрогн
 <button type="button" onClick={() => {
sult.steps.length - 1} className="p-1 round
tion-colors">
 <ChevronRight className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="В конец трассы">
 <button type="button" onClick={() => {
tep || debug.idx >= debug.result.steps.leng
0 disabled:opacity-40 transition-colors">
 <SkipForward className="w-4 h-4" />
 </button>
 </Tooltip>
 <span className="ml-1.5 text-[10px] text-
 {curDebug?.step ? (
 <>
 шаг {debug.idx + 1}/{debug.result.s
 {curDebug.step.func !== "<module>" &
 </>
) : (
 "шагов нет – исправьте ошибку и начини
)}


```

```

 active ? "bg-emerald-500/15 text-emerald-500" : "bg-white text-emerald-500"
 }`}
 >
 <span
 className={`w-7 shrink-0 select-none
 active ? "text-emerald-300" : "text-emerald-500"
 }`}
 >
 {active ? "▶" : ⏪}

 {line || " "}
 </div>
);
}}}
</div>

```

```

{/* панель переменных – «Variables», как в VS Code */}
<div className="shrink-0 max-h-[34%] overflow-hidden" >
 <div className="flex items-center gap-1.5" >
 <Variable className="w-3 h-3 text-indigo-500" value={selectedVariable} />
 Переменные на этом шаге
 {selectedVariable?.value}
 </div>
 {debug.result.error && (
 <pre className="mt-1 whitespace-pre-wrap" >{debug.result.error}</pre>
)}
 {debugOrdered.length === 0 && !debug.result.error && (
 <div className="mt-1 text-[11px] text-slate-500" >
 Нет переменных
 </div>
)}
 <div className="mt-1 flex flex-wrap gap-1" >
 {debugOrdered.map(([name, value]) => {
 const changed = debugChanged[name];
 const synced = syncVarSet.has(name);
 return (
 <span
 key={name}
 className={`inline-flex items-center gap-0.5
 ${changed ? "text-emerald-500" : "text-slate-500"}
 ${synced ? "font-normal" : "font-medium"}
 >
 {name} {value}

)
 })}
 </div>

```

```

 />
 })
 <div className="shrink-0 border-t border-slate-800" >
 <label htmlFor="python-stdin" className="text-slate-400">
 stdin для input()
 </label>
 <input
 id="python-stdin"
 value={stdin}
 onChange={(event) => {
 setEditHold(null);
 setStdin(event.target.value);
 }}
 spellCheck={false}
 placeholder="строки ввода через ↵"
 className="w-full bg-transparent font-mono"
 />
 </div>
 </div>

 { /* — Вывод ————— */
 <div className="shrink-0 border-t border-slate-800" >
 <button
 type="button"
 onClick={() => setOutputOpen((o) => !o)}
 className="flex items-center justify-between text-slate-400"
 >

 <Terminal className="w-3.5 h-3.5 text-emerald-400" />

 {runtimeReady && <CheckCircle2 className="w-3.5 h-3.5 text-emerald-400" />}
 {outputOpen ? "открыт" : "закрыт"}

 </button>
 {outputOpen && (
 <pre className="flex-1 min-h-[70px] overflow-auto font-mono text-slate-600">

```

```

 { name: 'Инопланетянин', emoji: ' ', color: 'from-emerald-500/20 to-emerald-600/20 bubbleText: `q[0] из Python` },
 { name: 'Бизнесмен', emoji: ' ', color: 'from-slate-500/20 to-slate-600/20 bubbleText: `q[1] из Python` },
 { name: 'Повар', emoji: ' ', color: 'from-slate-500/20 to-slate-600/20 bubbleText: `q[2] из Python` },
];

 let counter = 3;

export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: ' ', color: 'from-emerald-500/20 to-emerald-600/20 bubbleText: `q[0] из Python` },
 { id: 'q2', name: 'Кодер Семён', emoji: ' ', color: 'from-emerald-500/20 to-emerald-600/20 bubbleText: `q[1] из Python` },
 { id: 'q3', name: 'Кот Борис', emoji: ' ', color: 'from-emerald-500/20 to-emerald-600/20 bubbleText: `q[2] из Python` },
]));

 const runtime = useVizRuntime();
 const liveQ = vizArray(runtime?.variables?.q);
 const liveV = vizString(runtime?.variables?.v);
 const displayQueue: Customer[] = liveQ
 ? liveQ.map((value, index) => ({
 id: `python-${index}`,
 name: String(value),
 emoji: " ",
 color: "from-emerald-500/20 to-emerald-600/20 bubbleText: `q[${index}] из Python`,
 animState: "idle",
 }))
 : queue;

 const [servedHistory, setServedHistory] = useState<string[]>([]);
 const [isServing, setIsServing] = useState(false);
 const [shakeEmpty, setShake] = useState(false);
 const [log, setLog] = useState<string[]>([]);

 const addLog = (msg: string) => setLog(prev => [msg, ...prev]);

```

```

 setIsServing(true);
 const headGuest = queue[0];

 addLog(`  DEQUEUE: Повар наливает суп первому — ${headGuest.name}!`);

 // Step 1: Head guest gets eating animation
 setQueue(prev => prev.map((c, idx) => idx === 0 ? {
 ...c, animState: 'eating'
 } : c));

 setTimeout(() => {
 // Step 2: Guest leaves with happy face, queue sh
 setQueue(prev => prev.map((c, idx) => idx === 0 ? {
 ...c, animState: 'happy'
 } : c));

 setServedHistory(prev => [`${headGuest.emoji} ${headGuest.name} поел`]);

 setTimeout(() => {
 setQueue(prev => prev.slice(1));
 setIsServing(false);
 addLog(`  ${headGuest.name} поел и ушел сытым.`, 400);
 }, 900);
 }, [queue, isServing]);

const reset = () => {
 counter = 3;
 setQueue([
 { id: 'r1', name: 'Студент Вася', emoji: '👨🎓', color: 'red', col: 1, animState: 'idle', full: false, fullText: 'Я не ем! Я голода после матана!', animState: 'in' },
 { id: 'r2', name: 'Кодер Семён', emoji: '👨💻', color: 'blue', col: 2, animState: 'idle', full: false, fullText: 'Я не ем! Я пишу ИИ за порцию борща!', animState: 'in' },
 { id: 'r3', name: 'Кот Борис', emoji: '🐈', color: 'green', col: 3, animState: 'idle', full: false, fullText: 'Я не ем! Я хочу котика без сметаны, плиз.', animState: 'in' },
]);
 setServedHistory([]);
 setIsServing(false);
 setLog([' Столовая сброшена. Снова голодная толпа!']);
 setTimeout(() => {
 setQueue(prev => prev.map(c => ({ ...c, animState: 'idle' })));
 }, 500);
}

```



```

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800
 r-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

 { /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
 <div className="grid grid-cols-1 md:grid-cols-12" >

 { /* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */ }
 <div className={`md:col-span-9 bg-slate-900 rounded-2xl
 -[300px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}` >
 <div className="absolute top-4 left-4 text-[1.2em] font-mono" >
 Очередь голодных гостей (Буфер FIFO / BFS)
 {liveQ && {liveQ.length}}
 </div>

 <div className="flex-1 flex flex-col justify-between" >
 <div className="flex items-end justify-between" >
 { /* ПОВАРИХА И КОТЕЛ С СУПОМ */ }
 <div className="flex flex-col items-center" >
 <div className="relative" >
 <div className="absolute -top-10 left-1/2 -translate-x-1/2" >
 <div className="w-1.5 h-6 bg-slate-400 rounded" >
 <div className="w-2 h-4 bg-slate-400 rounded" >
 </div>
 {liveQ.length}
 </div>
 <div className="text-center" >
 {liveQ.length}

 ПОВАР

 </div>
 </div>
 </div>
 </div>
 </div>
 </div>

```

```

 <div className="font-bold text-3xl" style={{
 {customer.bubbleText}
 <div className="absolute -bottom-10 left-10
it rotate-45"></div>
 </div>

 {/* Персонаж */}
 <div className={`
 w-14 h-14 rounded-2xl border-2
relative
 ${customer.color}
 ${isHead ? 'ring-4 ring-amber-500' : ''}
 `}>
 {customer.name}

 {/* Индексы HEAD/TAIL */}
 {(isHead || isTail) && (
 <span className={`
 absolute -bottom-2 text-[0.8em]
 ${isHead ? 'bg-amber-500' : 'bg-transparent'}
 `}>
 {isHead && isTail ? 'h+t' : isHead ? 'h' : 't'}

)}
 </div>
 </div>
);
))
 })
</div>

</div>
</div>

{/* Пол кухни */}
<div className="w-full h-3 bg-gradient-to-r from-amber-500 to-amber-200">
</div>

```

```
 {/* ЛОГ ДЕЙСТВИЙ */}
 <div className="bg-slate-900 border border-slate-800" >
 <div className="text-[10px] font-mono text-slate-100" >
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between`}>
 >
 {idx === 0 ?
 {entry}
 </div>
 </div>
 </div>
))}
 </div>
 </div>
);
};
```

---

ФАЙЛ 63/94: src/components/SalmonAutomatonWidget.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useEffect } from 'react';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';

interface TrieNode {
 id: number;
 char: string;
 parent: number;
 children: { [key: string]: number };
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
 depth: number;
 x?: number;
```

```

 let count = 0;

 wordList.forEach((word) => {
 let curr = 0;
 for (let i = 0; i < word.length; i++) {
 const c = word[i].toUpperCase();
 if (newNodes[curr].children[c] === undefined) {
 count++;
 newNodes[curr].children[c] = count;
 newNodes[count] = {
 id: count,
 char: c,
 parent: curr,
 children: {},
 fail: 0,
 term_link: 0,
 is_terminal: false,
 words: [],
 depth: newNodes[curr].depth + 1
 };
 }
 curr = newNodes[curr].children[c];
 }
 if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
 }
 });

 let currentX = 0;
 const paddingX = 70;
 const paddingY = 80;

 const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {

```

```

 }
 const updated = [...words, cleanWord];
 setWords(updated);
 setNewWord('');
 buildInitialTrie(updated);
 };

const removeWord = (wordToRemove: string) => {
 if (words.length <= 1) {
 setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем работать');
 return;
 }
 const updated = words.filter((w) => w !== wordToRemove);
 setWords(updated);
 buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {
 const rootChildren = Object.values(nodes[0].children);
 const queue = [...rootChildren];
 const updatedNodes = { ...nodes };
 rootChildren.forEach((childId) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на главном уровне суффикса просто нет. Жми «Следующий шаг»!'
);
};

// Один шаг BFS
const stepBFS = () => {

```

```

 logs += 'У этой вершины нет детей, просто переходим к следующей';
 }

 setNodes(updatedNodes);
 setBfsQueue(newQueue);
 setCurrentNode(r);
 setSpeechText(logs);
};

return (
 <div className="bg-slate-800/90 rounded-2xl p-6 border-2 border-slate-400">
 {/* Шапка виджета */}
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center space-x-3">

 <div>
 <h4 className="text-2xl font-bold text-white">
 Интерактивный конструктор: Говорящий Эхо-бот
 </h4>
 <p className="text-sm text-slate-400">
 Построение дерева и расчет суффиксных ссылок
 </p>
 </div>
 </div>
 <div className="flex items-center gap-2">
 <button
 onClick={() => buildInitialTrie(words)}
 className="flex items-center gap-1 bg-slate-400/50 px-4 py-2 rounded-md transition-colors"
 title="Сбросить автомат"
 >
 <RotateCcw className="w-4 h-4" /> Сбросить
 </button>
 </div>
 </div>
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {/* Верхняя панель: Говорящий лосось и диалоговое окно */}
 </div>
 </div>
);

```



```

 })
 </div>
</div>
</div>
</div>

{/* SVG Canvas for Automaton Tree */}
<div className="mb-8 bg-slate-900 border border-slate-800" >
 <h5 className="text-white font-bold mb-3 flex items-center">
 Дерево и Суффиксные (fail) ссылки
 </h5>

 {{{() => {
 let maxX = 0;
 let maxY = 0;
 Object.values(nodes).forEach(n => {
 if (n.x && n.x > maxX) maxX = n.x;
 if (n.y && n.y > maxY) maxY = n.y;
 });
 const svgWidth = Math.max(maxX + 80, 400);
 const svgHeight = Math.max(maxY + 60, 200);

 return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`} >
 <defs>
 <marker id="arrowhead" markerWidth="6" markerHeight="6" >
 <polygone points="0 0, 6 3, 0 6" fill="white" />
 </marker>
 <marker id="fail-arrow" markerWidth="6" markerHeight="6" >
 <polygone points="0 0, 6 3, 0 6" fill="white" />
 </marker>
 </defs>

 <g>
 <!-- Draw Edges -->
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x || !node.y) return null;
 const x1 = parent.x, y1 = parent.y, x2 = node.x, y2 = node.y;
 const midX = (x1 + x2) / 2, midY = (y1 + y2) / 2;
 const angle = Math.atan2(y2 - y1, x2 - x1);
 const midAngle = angle - Math.PI / 2;
 const midLength = 10;
 const midX2 = midX + Math.cos(midAngle) * midLength;
 const midY2 = midY + Math.sin(midAngle) * midLength;
 const midId = `mid-${node.id}`;
 <g>
 <line x1={x1} y1={y1} x2={x2} y2={y2} />
 <circle cx={midX2} cy={midY2} r={3} />
 <circle cx={midX2} cy={midY2} r={3} />
 <circle cx={midX2} cy={midY2} r={3} />
 </g>
 })}
 </g>
 </svg>
)
 }})}

```



```

 const isCurrent = currentNode === node..

 return (
 <path
 key={`fail-${node.id}`}
 d={`M ${node.x} ${node.y - 10} Q ${
- 16}`}
 fill="none"
 stroke={isCurrent ? '#f43f5e' : 'rgb(
 strokeWidth={isCurrent ? 2 : 1.5}
 strokeDasharray="4 4"
 markerEnd="url(#fail-arrow)"
 />
);
 }}}

 {/* Draw Nodes */}
 Object.values(nodes).map(node => {
 if (!node.x || !node.y) return null;
 const isRoot = node.id === 0;
 const isCurrent = currentNode === node..
 const isInQueue = bfsQueue.includes(node..

 let fill = '#1e293b';
 let stroke = '#475569';

 if (isCurrent) {
 fill = '#2563eb';
 stroke = '#60a5fa';
 } else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
 } else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
 }

 return (

```

```

<div className="grid grid-cols-1 lg:grid-cols-3 gap-4">
 <div className="bg-slate-900/50 p-5 rounded-xl">
 <div>
 <h5 className="text-white font-bold mb-3">
 Словарь (Паттерны)
 </h5>
 <div className="flex flex-wrap gap-2 mb-4">
 {words.map((w) => {
 <span
 key={w}
 className="bg-slate-800 border border-slate-700
 px-2 py-1 gap-1 group"
 >
 {w}
 <button
 onClick={() => removeWord(w)}
 className="text-slate-500 hover:text-white"
 title="Удалить"
 >
 ×
 </button>

)}}
 </div>
 </div>
 </div>

```

```

<form onSubmit={handleAddWord} className="flex gap-2">
 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.value)}
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-slate-700
 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 >

```

```

<tr
 key={node.id}
 className={`transition-colors ${
 isCurrent
 ? 'bg-blue-900/40 font-bold text-white'
 : isInQueue
 ? 'bg-slate-800/80'
 : 'hover:bg-slate-800/40'
 }}
>
 <td className="py-2.5 px-3 flex items-center" >
 <span
 className={`w-2 h-2 rounded-full ${
 node.id === 0
 ? 'bg-purple-500'
 : isCurrent
 ? 'bg-blue-400 animate-pulse'
 : isInQueue
 ? 'bg-amber-400'
 : 'bg-slate-600'
 }}
 >
 #{node.id}
 {node.id === 0 &&
 </td>
 <td className="py-2.5 px-3" >

 {node.char}

 </td>
 <td className="py-2.5 px-3 text-slate-400" >
 {node.id === 0 ? '-' : `#${node.id}`}
 </td>
 <td className="py-2.5 px-3" >
 {node.id === 0 ? (

 </td>


```

```
 </div>
);
};
```

---

ФАЙЛ 64/94: src/components/SegmentTreeVisualizer.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
import { useVizRuntime, vizNumber } from '../data/vizStore';
```

```
// ===== TYPES =====
```

```
interface Step {
 id: number;
 desc: string;
 codeLine: number;
 treeState: Record<number, number>; // node index
 highlightNodes: number[]; // current highlighted nodes
 mergeChildren?: [number, number]; // pair being merged
 arrayHighlight?: [number, number]; // [L, R] range
 result?: number; // partial result
 lazyState?: Record<number, number>; // node index for lazy state
 promisePushed?: number; // узел, у которого есть promise
}
```

```
// ===== PURE LOGIC: build tree =====
```

```
export function buildTreeFull(arr: number[]): Record<number, number> {
 const n = arr.length;
 const tree: Record<number, number> = {};
 function go(v: number, l: number, r: number) {
 if (l === r) { tree[v] = arr[l]; return; }
 const m = (l + r) >> 1;
 go(2 * v, l, m);
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 }
 go(1, 0, n - 1);
 return tree;
}
```

```

 { ...tree }, highlightNodes: [v], arrayHighlight:
go(2 * v, l, m);
steps.push({ id: id++, desc: `Вернулись в build(${v}
 lightNodes: [v], arrayHighlight: [l, r] });
go(2 * v + 1, m + 1, r);
tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
steps.push({ id: id++, desc: `Объединяем: tree[${v}]
 state: { ...tree }, highlightNodes: [v], mergeChild
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: ` Дерево построено! Корень
 Highlight: [0, n - 1] });
return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
 const n = arr.length;
 const tree = buildTreeFull(arr);
 const steps: Step[] = [];
 let id = 0;

 function go(v: number, l: number, r: number, ql: number, qr: number) {
 if (ql > r || qr < l) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]
 codeLine: 2, treeState: tree, highlightNodes: [v]
 return 0;
 }
 if (ql <= l && r <= qr) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]
] = ${tree[v]}`, codeLine: 4, treeState: tree, highlightNodes: [v]
 return tree[v];
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]
 codeLine: 6, treeState: tree, highlightNodes: [v]
 const left = go(2 * v, l, m, ql, qr);
 const right = go(2 * v + 1, m + 1, r, ql, qr);
 }
}

```

```

 r--;
 res += tree[r] ?? 0;
 steps.push({ id: id++, desc: `r=${r + 1} нечётный`,
 , treeState: tree, highlightNodes: [r], arrayHighlight: [qL, qR], result: res });
 highlights.push(r);
 }
 l >>= 1;
 r >>= 1;
 if (l < r) {
 steps.push({ id: id++, desc: `Поднимаемся: l = ${l}`,
 , arrayHighlight: [qL, qR], result: res });
 }
}
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}])`,
 , highlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val: number; left: VNode; right: VNode; }
```

```

function buildVisualTree(treeState: Record<number, number>) {
 if (l === r) return { v, l, r, val: treeState[v] ?? null, left: null, right: null };
 const m = (l + r) >> 1;
 return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(treeState, l, m),
 , right: buildVisualTree(treeState, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```

const BUILD_CODE = [
 "build(v, l, r) {",
 " if (l === r) { tree[v] = A[l]; return; }",
 " // -----",
 " // -----",
 " mid = (l + r) >> 1;",
 " build(2*v, l, mid);",
 " build(2*v+1, mid+1, r);",
 " tree[v] = tree[2*v] + tree[2*v+1];",
 "}"
]

```

```

" update(2v+1, m+1, r)", // 11
" tree[v] = tree[2v]+tree[2v+1]", // 12
"}", // 13
"// --- запрос sum(uL..uR):", // 14
"// спускаясь, толкаем lazy вниз," // 15
"// целиком внутри – читаем tree[v]" // 16
];

```

```

export function genLazySteps(arr: number[], uL: number,
 const n = arr.length;
 const steps: Step[] = [];
 let id = 0;
 const tree: Record<number, number> = {};
 const lazy: Record<number, number> = {};
 for (const k of Object.keys(buildTreeFull(arr))) {
 tree[Number(k)] = buildTreeFull(arr)[Number(k)];
 lazy[Number(k)] = 0;
 }
 const full = buildTreeFull(arr);

 const snap = {desc: string, codeLine: number, hl: number,
 steps.push({
 id: id++,
 desc,
 codeLine,
 treeState: { ...tree },
 highlightNodes: [...hl],
 lazyState: { ...lazy },
 ...extra,
 });
 };
 snap(`Старт: дерево построено (корень = ${full[1]}).
 `);

```

```

function pushDown(v: number, l: number, r: number) {
 if (lazy[v] !== 0) {
 const m = (l + r) >> 1;
 const d = lazy[v];

```

```

let res = 0;
function query(v: number, l: number, r: number) {
 if (r < uL || uR < l) {
 snap(`query(v=${v}, [${l}..${r}]): не пересекает
 return;
 }
 if (uL <= l && r <= uR) {
 res += tree[v] ?? 0;
 snap(`query(v=${v}, [${l}..${r}]): целиком внутри
 sult: res });
 return;
 }
 pushDown(v, l, r);
 const m = (l + r) >> 1;
 snap(`query(v=${v}, [${l}..${r}]): частичное пересе
);
 query(2 * v, l, m);
 query(2 * v + 1, m + 1, r);
}
res = 0;
snap(`Теперь запрос sum([${uL}..${uR}]) – проверим, ч
query(1, 0, n - 1);
snap(` sum([${uL}..${uR}]) = ${res} – совпадает с че
return steps;
}

```

// ===== PLAYER COMPONENT =====

```

function Player({ steps, color }: { steps: Step[]; colo
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [speed, setSpeed] = useState(700);

 useEffect(() => { setIdx(0); setPlaying(false); }, [s

 useEffect(() => {
 if (!playing || idx >= steps.length - 1) { if (idx :
 const t = setTimeout(() => setIdx(i => i + 1), speed
 return () => clearTimeout(t);
 }

```



```

return (
 <div key={nd.v} className="flex flex-col items-center justify-center" >
 <div className={`flex flex-col items-center justify-center ${cls}`} >

 [{nd.left}, {nd.right}]

 {promise !== 0 && (

 o whitespace-nowrap">
 {promise}

)}
 </div>
 {(nd.left || nd.right) && (
 <div className="flex flex-col items-center mt-2" >
 <div className="w-px h-2 bg-slate-700" />
 <div className="flex justify-around w-full" >
 {nd.left && renderNode(nd.left)}
 {nd.right && renderNode(nd.right)}
 </div>
 </div>
)}
 </div>
);
}, [step, clr, liveNode]);

return (
 <div className={`rounded-2xl border overflow-hidden ${
 ? 'border-emerald-500/30' : 'border-amber-500/30'
 }`} >
 { /* Header */ }
 <div className={`px-4 py-2.5 flex items-center justify-center ${
 'bg-emerald-950/50' : 'bg-amber-950/50'
 } border-b`} >
 <h4 className="font-bold text-sm text-white flex-1" >

 </div>

```

```

px]">
<span className={`mt-0.5 inline-block w-2 h-2 rounded`
{step.desc}
{step.result !== undefined && {
 <span className={`ml-auto shrink-0 font-mono text-
 ' : color === 'emerald' ? 'text-emerald-300' :
 = {step.result}

}}
</div>

{/* Controls */}
<div className="px-3 py-2.5 bg-slate-950 border-t
 <div className="flex items-center bg-slate-900
 <button onClick={() => { setPlaying(false); s
 unded" title="Сброс"><RotateCcw className="
 <button onClick={() => { setPlaying(false); s
 0 hover:text-white hover:bg-slate-800 rounded
 <button onClick={() => setPlaying(!playing)}
 white transition-all ${playing ? 'bg-rose-5
 {playing ? <Pause className="w-3 h-3" /> :
 </button>
 <button onClick={() => { setPlaying(false); s
 .5 text-slate-400 hover:text-white hover:bg
 utton>
 </div>
 <select value={speed} onChange={e => setSpeed(N
 text-[9px] font-bold rounded px-1.5 py-1 outl
 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Турбо</option>
 </select>
</div>

{/* Progress bar */}
<div className="h-1 bg-slate-950"><div className=
 + 1) / total) * 100 : 0}%` }} /></div>

```

```

 setArr(a);
 setCustomInput(a.join(', '));
 };

const buildSteps = useMemo(() => genBuildSteps(arr),
const queryTDSteps = useMemo(() => genQueryTopDownSteps(arr),
const queryBUSteps = useMemo(() => genQueryBottomUpSteps(arr),
// Питон умеет рулить добавкой: переменная add из кода
const effAdd = liveAdd ?? uAdd;
const lazySteps = useMemo(() => genLazySteps(arr, uL, uR), [arr, uL, uR]);

const totalSum = arr.reduce((a, b) => a + b, 0);

return (
 <div className="bg-slate-900 rounded-2xl border border-indigo-500" >
 {/* ---- TOP BAR: array editor + query range ---- */}
 <div className="mb-6 space-y-4">
 <div className="flex flex-col lg:flex-row gap-4">
 {/* Array input */}
 <form onSubmit={handleApply} className="flex flex-col">
 <div className="flex items-center justify-between">
 <label className="text-[10px] font-bold uppercase">
 Array Editor
 </label>
 <button type="button" onClick={randomize} className="text-[10px] font-medium">
 Refresh
 </button>
 </div>
 <div className="flex gap-2">
 <input
 type="text"
 value={customInput}
 onChange={e => setCustomInput(e.target.value)}
 className="flex-1 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 placeholder="5, 8, 3, 12, 7, 2"
 />
 <button type="submit" className="bg-indigo-500 text-white px-4 py-2 font-medium">
 Применить
 </button>
 </div>
 </form>
 </div>
 </div>
 </div>
);

```

```

 </div>
 <div className="text-[10px] text-slate-500" style={
 Запрос: sum(A[{qL}..{qR}]) = {arr.slice(q
 </div>
 </div>
 </div>
 </div>

```

```

{ /* ---- TABS ---- */
<div className="flex flex-wrap items-center gap-1" style={
 <button onClick={() => setView('build')} className="border-1
 -colors ${view === 'build' ? 'border-indigo-500 text-slate-200'}`}>
 <Hammer className="w-3.5 h-3.5" /> Построить
 </button>
 <button onClick={() => setView('queries')} className="border-1
 on-colors ${view === 'queries' ? 'border-emerald-500 text-slate-200'}`}>
 <Search className="w-3.5 h-3.5" /> Запрос: св
 </button>
 <button onClick={() => setView('lazy')} className="border-1
 colors ${view === 'lazy' ? 'border-amber-500 text-slate-200'}`} title="Массовое обновление на отрез
 Обновление на отрезке (Lazy)
 </button>
 <button onClick={() => setView('theory')} className="border-1
 n-colors ${view === 'theory' ? 'border-blue-500 text-slate-200'}`}>
 <Info className="w-3.5 h-3.5" /> Почему / Сравн
 </button>
</div>

```

```

{ /* ---- TAB CONTENT ---- */
{view === 'build' && (
 <SimPanel
 key={`build-${arr.join('-')}`}
 title="Построение дерева (рекурсия)"
 icon=" "
 >

```

```

 ...
 <input type="number" min={0} max={arr.length} value={e.target.value} className="w-14 bg-slate-500 border-slate-700 outline-none focus:border-amber-500" />
 </div>
 <div className="flex items-center gap-2">
 <label className="text-[10px] uppercase text-slate-500">Add</label>
 <input type="number" value={effAdd} onChange={handleAdd} className="w-14 bg-slate-500 border-slate-700 text-white text-xs font-mono" />
 </div>
 <p className="text-[11px] text-slate-500 margin-top: 10px">
 Ко всем элементам отрезка прибавляем сумму (введенную в поле).

 {liveAdd !== null && +{liveAdd}}

 {liveV !== null && +{liveV}}

 </p>
 </div>
 </SimPanel>
 <div className="flex items-center gap-2">
 <div>
 key={`lazy-${arr.join('-')}-${uL}-${uR}-${effAdd}`}
 title={`update([${uL}..${uR}], +${effAdd})`}
 icon="🔢"
 steps={lazySteps}
 code={LAZY_CODE}
 color="amber"
 arr={arr}
 liveNode={liveV}
 </div>
 </div>
)}

{view === 'theory' && (
 <div className="space-y-5">
 <div>
 <h4>Why it splits this way</h4>
 <div className="bg-slate-950 p-5 rounded-xl border-slate-700">
 <h4>text-base font-bold text-white</h4>
 во разбилось именно так для N={arr.length}?
 </div>
 </div>
 </div>
)
```



```

 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск по темам.."
 className="w-full bg-slate-800/70 border border-bo
 500 focus:outline-none focus:border-indigo-!
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -tran
 aria-label="Очистить поиск"
 >
 <X className="w-3.5 h-3.5" />
 </button>
)}
 </div>
</div>

<div className="flex-1 overflow-y-auto px-3 pb-4
 {groups.length === 0 && <p className="text-sm t

 {groups.map(([category, items]) => (
 <div key={category}>
 <div className="text-[10px] font-bold upperc
 <div className="space-y-1">
 {items.map((chapter) => {
 const isSelected = chapter.id === selec
 const hasViz = Boolean(VIZ_REGISTRY[cha
 return (
 <button
 key={chapter.id}
 onClick={() => {
 setSelectedChapterId(chapter.id);
 onNavigate?.();
 window.scrollTo({ top: 0, behavio
 }}
 className={`w-full text-left px-3 p
 isSelected

```

---

ФАЙЛ 66/94: src/components/SplayPlayground.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from "react";
import { SplayRotationsViz } from "../SplayRotationsViz";
import SplayWalkViz from "../SplayWalkViz";
import { SplayRotationSandbox } from "../SplayRotationSandbox";
```

```
/**
```

```
 * Страница splay – три вкладки по нарастанию сложности
```

```
 * 1. «Механика» – покадровый разбор Zig / Zig-Zig / Zig-Zag
```

```
 * (прицел → поворот, призраки старых позиций, обход)
```

```
 * 2. «Большое дерево» – те же случаи внутри настоящего
```

```
 * поворот – кадры «прицел → отстёгиваем среднее по
```

```
 * 3. «Собери сам» – песочница: подними узел кликами, и
```

```
 */
```

```
type Tab = "cases" | "big" | "play";
```

```
export const SplayPlayground: React.FC<{ defaultTab?: Tab }> = ({ defaultTab }) =>
```

```
 const [tab, setTab] = useState<Tab>(defaultTab);
```

```
 return (
```

```
 <div className="bg-slate-800/90 p-6 rounded-2xl border">
```

```
 <div className="flex flex-wrap items-center justify-between">
```

```
 <div className="flex items-center gap-3">
```

```
 <div className="p-2.5 bg-indigo-600 text-white">
```

```

```

```
 </div>
```

```
 <div>
```

```
 <h3 className="text-2xl font-bold text-white">
```

```
 <p className="text-sm text-indigo-300">Один
```

```
 </div>
```

```
 </div>
```

```
 </div>
```



```
import React, { useCallback, useMemo, useState } from "
import { CheckCircle2, Dices, Eye, EyeOff, RotateCcw, U

/**
 * Песочница поворотов.
 *
 * Здесь ничего не подсказывается: дано настоящее дерев
 * поднять отмеченный узел в корень. Клик по узлу = оди
 * Порядок поворотов выбираешь сам; правильный ли он по
 * только после того, как узел окажется наверху (или по
 */

export interface TNode {
 key: number;
 left: TNode | null;
 right: TNode | null;
}

const leaf = (key: number): TNode => ({ key, left: null

const clone = (t: TNode | null): TNode | null =>
 t ? { key: t.key, left: clone(t.left), right: clone(t

/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
export const PRESETS: { name: string; hintCase: string;
 {
 name: "Линия влево",
 hintCase: "Zig-Zig",
 target: 1,
 build: () => {
 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
 },
},
```

```

 cur = key < cur.key ? cur.left : cur.right;
 }
 return [];
};

/** Поднять узел key на один уровень (поворот с родителем)
export function rotateUp(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return root;

 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const gg = path.length >= 3 ? path[path.length - 3] : null;

 if (p.left === x) {
 p.left = x.right;
 x.right = p;
 } else {
 p.right = x.left;
 x.left = p;
 }

 if (!gg) return x;
 if (gg.left === p) gg.left = x;
 else gg.right = x;
 return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number): TNode {
 const path = findPath(root, key);
 if (path.length < 2) return null;
 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const g = path.length >= 3 ? path[path.length - 3] : null;

 if (!g) return { node: x.key, kind: "Zig" };

```

```

 height: rows * rowH + 20,
 };
}

/* ----- КОМПОНЕНТ -----

export const SplayRotationSandbox: React.FC = () => {
 const [presetIdx, setPresetIdx] = useState(0);
 const preset = PRESETS[presetIdx];

 const [tree, setTree] = useState<TNode>(() => preset.build());
 const [history, setHistory] = useState<TNode[]>([]);
 const [moves, setMoves] = useState<{ node: number; count: number }>([]);
 const [showAnswer, setShowAnswer] = useState(false);

 const target = preset.target;
 const solved = tree.key === target;

 const reset = useCallback(
 (idx = presetIdx) => {
 setPresetIdx(idx);
 setTree(PRESETS[idx].build());
 setHistory([]);
 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

 const { nodes, edges, width, height } = useMemo(() => preset.build(), [preset]);
 const path = useMemo(() => new Set(findPath(tree, target)), [tree, target]);
 const expected = useMemo(() => (solved ? null : canonicalPath(target)), [target, solved]);

 const clickNode = (key: number) => {
 if (solved) return;
 const p = findPath(tree, key);
 if (p.length < 2) return; // корень крутить не с кем
 };
};

```

```

 <button
 key={p.name}
 onClick={() => reset(i)}
 className={`px-3 py-1.5 rounded-lg text-[11pt]
 i === presetIdx ? "bg-indigo-600 text-white" : ""
 }`
 >
 {p.name}
 </button>
)}
 </div>

 <div className="bg-slate-900 rounded-xl border border-slate-800"
 >
 <svg
 viewBox={`0 0 ${width} ${height}`}
 className="h-auto block mx-auto"
 style={{ minWidth: Math.min(width * 1.5, 420) }}
 role="img"
 >
 {edges.map(([a, b], i) => {
 const na = nodes.find((n) => n.key === a);
 const nb = nodes.find((n) => n.key === b);
 if (!na || !nb) return null;
 const onPath = path.has(a) && path.has(b);
 return (
 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}
 stroke={onPath ? "#6366f1" : "#475569"}
 strokeWidth={onPath ? 3 : 1.8}
 style={{ transition: "all 700ms cubic-bezier(0.16, 0.66, 0.16, 0.84)" }}
 />
);
 })}
 </div>
 </div>
 </div>
)
}
}

```



```

import { useVizRuntime, vizNumber, vizString } from "...
import { Gauge, Pause, Play, RotateCcw } from "lucide-r

/**
 * Разбор трёх случаев операции Splay: Zig, Zig-Zig, Zi
 *
 * Что сделано ради понятности:
 * * у узлов настоящие числа (20 / 40 / 60), а не абстр
 * сразу видно, что дерево остаётся деревом поиска;
 * * под деревом печатается обход слева направо: он ОД
 * это и есть доказательство, что поворот ничего не
 * * кадр «прицел» ничего не двигает, только подсвечив
 * * на кадре «результат» остаются призраки – пунктирн
 * местах и стрелки «откуда → куда»;
 * * по умолчанию анимация НЕ крутится сама: пользоват
 * идёт в своём темпе. Автопрокрутка включается кноп
 */

type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;

interface Frame {
 pos: Pos;
 edges: [NodeId, NodeId][];
 kind: "start" | "aim" | "result";
 title: string;
 text: string;
 /** Пара, которую крутим на этом шаге: кадр-«прицел».
 focus?: [NodeId, NodeId];
 /** Кто переехал на этом кадре – рисуем призрак и стр
 moved?: NodeId[];
 ms: number;
}

interface Case {
 key: "zig" | "zig-zig" | "zig-zag";
 label: string;
 when: string;
```

```

 ...ZIG_BEFORE,
 kind: "aim",
 title: "Прицел: крутим ребро 40 – 20",
 text: "Ничего пока не двигается. Смотри на подсвеченное ребро",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [160, 50], A: [80, 130], p: [245, 130],
 edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p", "C"}],
 kind: "result",
 title: "Стало: 20 – корень",
 text: "40 стало правым сыном двадцатки. Поддеревом от 40 – это одно и то же место.",
 moved: ["x", "p", "B"],
 ms: RESULT_MS,
 },
],
};

/* ----- Zig-Zig -----
const ZZ_S0 = {
 pos: { g: [255, 40], D: [325, 112], p: [160, 112], C: [212, 112],
 edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g: [255, 40],
 edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZIG: Case = {
 key: "zig-zig",
 label: "Zig-Zig",
 when: "x и p – дети с ОДНОЙ стороны (оба слева или оба справа)",
 rotations: "2 поворота, первым – верхний",
 keys: { x: 20, p: 40, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
};

```

```

 kind: "result",
 title: "Стало: 20 в корне, бамбук стал кустом",
 text: "Сравни с первым кадром: А и В были на глуб
 дерево, а не только достаёт ключ.",
 moved: ["x", "p", "A", "B"],
 ms: RESULT_MS,
 },
],
};

/* ----- Zig-Zag -----
const ZG_S0 = {
 pos: { g: [258, 40], D: [328, 112], p: [148, 112], A:
 edges: [["g", "p"], ["g", "D"], ["p", "A"], ["p", "x"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZG_S1 = {
 pos: { g: [258, 40], D: [328, 112], x: [148, 112], p:
 edges: [["g", "x"], ["g", "D"], ["x", "p"], ["x", "C"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZAG: Case = {
 key: "zig-zag",
 label: "Zig-Zag",
 when: "x и p — дети с РАЗНЫХ сторон (змейка)",
 rotations: "2 поворота, первым — нижний",
 keys: { x: 40, p: 20, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по
 ",
 ms: RESULT_MS,
 },
],
};

```



```
const COLOR: Record<string, { fill: string; stroke: string }> = {
 x: { fill: "#6366f1", stroke: "#a5b4fc" },
 p: { fill: "#0ea5e9", stroke: "#7dd3fc" },
 g: { fill: "#f59e0b", stroke: "#fcd34d" },
 sub: { fill: "#1e293b", stroke: "#475569" },
};
```

```
const ROLE_RU: Partial<Record<NodeId, string>> = {
 x: "x — ищем его",
 p: "p — родитель",
 g: "g — дед",
};
```

```
const isSubtree = (id: NodeId) => id === "A" || id ===
```

```
const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C> = ({
 const { pos, edges, focus, moved } = frame;
 const dim = (id: NodeId) => (focus ? !focus.includes(id) : true);
 const ghosts = frame.kind === "result" ? (moved ?? []) : [];
```

```
 return (
 <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-full h-full">
 <defs>
 <marker id="splay-arrow" viewBox="0 0 10 10" refX="0" refY="0">
 <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
 </marker>
 </defs>
```

```
 {/* призраки: где узел стоял до поворота */}
```

```
 {ghosts.map((id) => {
 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 const pad = r + 8;
```

```

 {(Object.keys(pos) as NodeId[]).map((id) => {
 const p = pos[id];
 if (!p) return null;
 const sub = isSubtree(id);
 const c = COLOR[sub ? "sub" : id];
 const r = sub ? 15 : 20;
 const hot = Boolean(focus && focus.includes(id))
 return (
 <g
 key={id}
 style={{
 transform: `translate(${p[0]}px, ${p[1]}px)`,
 transition: MOVE,
 opacity: dim(id) ? 0.28 : 1,
 }}
 >
 {hot && <circle r={r + 7} fill="none" stroke=c.stroke}
 <circle r={r} fill={c.fill} stroke={c.stroke} />
 <text
 textAnchor="middle"
 dominantBaseline="central"
 fontSize={sub ? 12 : 15}
 fontWeight="700"
 fill={sub ? "#94a3b8" : "#fff"}
 >
 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font-size=10>
 {id}
 </text>
)}
 </g>
);
 })
 }
</svg>
);

```

```

 return () => clearTimeout(t);
 }, [playing, frame, duration, total]);

return (
 <div className="w-full bg-slate-950 rounded-2xl border-2 border-slate-800">
 <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar">
 {CASES.map((c, i) => (
 <button
 key={c.key}
 onClick={() => setCaseIdx(i)}
 className={`px-3 sm:px-4 py-2 rounded-lg text-sm transition-colors
 ${i === caseIdx ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-300"}
 `}
 >
 {c.label}
 </button>
))}
 </div>

 <div className="mb-3 text-xs sm:text-[13px] text-slate-400">
 Корреляция
 ({active.rotation}°)
 </div>

 {/* шаг + пояснение стоят НАД картинкой: сначала шаг, потом пояснение */}
 <div className="mb-3 rounded-xl border border-slate-800">
 <div className="flex items-center gap-2 mb-1 flex-wrap">
 <span
 className={`text-[10px] font-bold uppercase transition-colors
 ${current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"}
 `}
 >
 {current.kind === "aim" ? "прицел" : current.kind === "start" ? "начало" : "конец"}


```

```

 : { width: "100%", background: "#334155"
 }
 />
 </div>

 <div className="flex flex-wrap items-center gap-2"
 <button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f - 1 + total) % total);
 }}
 disabled={idx === 0}
 className="px-3 py-2 rounded-lg bg-slate-800
 d:hover:text-slate-300 text-xs font-bold tr
 >
 ← Назад
 </button>
 <button
 onClick={() => {
 setPlaying(false);
 setFrame((f) => (f + 1) % total);
 }}
 className="px-4 py-2 rounded-lg bg-indigo-600
 w-indigo-900/40"
 >
 {last ? "⏮ Сначала" : "Дальше →"}
 </button>
 <button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-1
 text-xs font-bold transition-colors"
 >
 {playing ? <Pause className="w-3.5 h-3.5" />
 : {playing ? "Стоп" : "Авто"}}
 </button>
 <button
 onClick={() => setSlow((s) => !s)}
 title="Автопрокрутка ещё медленнее"

```

ФАЙЛ 69/94: src/components/SplayWalkViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useEffect, useMemo, useRef, useState } from 'react';
import { useVizRuntime, vizNumber } from '../data/vizStore';
```

```
/**
 * Splay на БОЛЬШОМ дереве – по урокам treap-прохода.
 *
 * Главный принцип: НИКАКИХ «хоп». Каждый одиночный поворот:
 * aim – прицел на тройку g–p–x, назван случай;
 * cut – отстёгиваем среднее поддереву β (пунктир, поворот);
 * rot – поворот: зелёным подсвечены рёбра, которых не было.
 * Рядом с логом всегда висит врезка «Механика одного поворота».
 * На каждом кадре – живая автопроверка in-order (сортировка).
 *
 * Чистые функции (splaySteps, rotateUp, bstInsert, cloneWithParents).
 */
```

```
export interface SN {
 key: number;
 left: SN | null;
 right: SN | null;
 parent: SN | null;
}
```

```
export function cloneWithParents(t: SN | null): SN | null {
 if (!t) return null;
 const n: SN = { key: t.key, left: null, right: null, parent: null };
 const l = t.left ? cloneWithParents(t.left) : null;
 const r = t.right ? cloneWithParents(t.right) : null;
 if (l) {
 n.left = l;
 l.parent = n;
 }
 if (r) {
```

```

 p.right = child.left;
 if (child.left) child.left.parent = p;
 child.left = p;
 p.parent = child;
 }
 child.parent = g;
 if (!g) return;
 if (g.left === p) g.left = child;
 else g.right = child;
}

/** Рёбра в виде "p→c" — для diff между кадрами. */
export function collectEdges(t: SN | null): string[] {
 const es: string[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push(`${n.key}→${n.left.key}`);
 if (n.right) es.push(`${n.key}→${n.right.key}`);
 walk(n.left);
 walk(n.right);
 };
 walk(t);
 return es.sort();
}

const diffEdges = (before: string[], after: string[]):
 const rm = before.filter((e) => !after.includes(e));
 const ad = after.filter((e) => !before.includes(e));
 const parts: string[] = [];
 for (const a of ad) {
 const [p, c] = a.split("→");
 if (rm.includes(`${c}→${p}`)) parts.push(`ребро ${p}`);
 else parts.push(`поддерево ${c} переехало к ${p}`);
 }
 return parts.join("; ");
};

export type SplayStepKind = "search" | "aim" | "cut" |

```

```

 cur = key < cur.key ? cur.left : cur.right;
 }
 const x = found ? cur : last;
 steps.push({
 kind: "search",
 root: cloneWithParents(root)!,
 pathKeys: [...pathKeys],
 note: isInsert
 ? `Вставили ${key} обычным BST-спуском. Теперь по
 : found
 ? `Спуск по ключу ${key}: путь ${pathKeys.join(
 : `Ключа ${key} нет. Спуск упёрся в ${last.key}
 rotCount: 0,
));

 // --- подъём ---
 let rotCount = 0;
 let target: SN = x as SN;
 const fixRoot = () => {
 while (root && root.parent) root = root.parent;
 };
 /** Один поворот = три кадра: aim → cut → rot. */
 const emitRotation = (child: SN, label: string, aimEd
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge,
 caseLabel: label,
 note: aimNote,
 rotCount,
));
 const p = child.parent!;
 const mid = p.left === child ? child.right : child.
 steps.push({
 kind: "cut",
 root: cloneWithParents(root)!,
 cutEdge: [child.key, mid ? mid.key : child.key],
 cutLabel: mid ? `β = поддерезо ${mid.key}` : "β n

```





```
};
```

```
const CODE_LINES = [
 { line: 1, text: "while x has parent:" },
 { line: 2, text: " p = parent(x); g = parent(p)" },
 { line: 3, text: " if g is None: # Z" },
 { line: 4, text: " rotate(x over p)" },
 { line: 5, text: " elif p,g,x на одной линии: # " },
 { line: 6, text: " rotate(p over g) # B" },
 { line: 7, text: " rotate(x over p)" },
 { line: 8, text: " else: # Z" },
 { line: 9, text: " rotate(x over p) # H" },
 { line: 10, text: " rotate(x over g)" },
];
```

```
/* ----- врезка «механика одного по
```

```
const MechanismInset: React.FC<{ step: SplayStep | null
 const isCut = step?.kind === "cut";
 const isRot = step?.kind === "rot";
 return (
 <div className="bg-slate-950/80 rounded-lg border b
 <h4 className="text-xs font-bold text-slate-300 m
 <p className="text-[10px] text-slate-500 mb-2">
 Как вправление вывиха: одна кость ходит вокруг
 </p>
 <svg viewBox="0 0 340 190" className="w-full">
 <g transform="translate(10,6)">
 <text y="10" fill="#94a3b8" fontSize="11" fon
 <line x1="55" y1="46" x2="30" y2="86" stroke=
 <line x1="55" y1="46" x2="80" y2="86" stroke=
 <line x1="30" y1="86" x2="12" y2="126" stroke=
 <line
 x1="30" y1="86" x2="48" y2="126"
 stroke={isCut ? "#f59e0b" : "#64748b"}
 strokeWidth={isCut ? 3 : 2}
 strokeDasharray={isCut ? "5,3" : "none"}
 />
```

```

 <circle
 cx="63" cy="126" r="13"
 fill={isRot ? "#78350f" : "#0f172a"}
 stroke={isRot ? "#10b981" : "#64748b"}
 strokeWidth={isRot ? 2.5 : 1.5}
 />
 <text x="63" y="130" textAnchor="middle" fill="black">
 <circle cx="95" cy="126" r="13" fill="#0f172a" />
 <text x="95" y="130" textAnchor="middle" fill="black">
 <text x="112" y="92" fill={isRot ? "#10b981" : "#64748b"}
 {isRot ? "↓ β теперь при р" : "β при р"}
 </text>
 <text x="30" y="160" fill="#64748b" fontSize=14>
 </g>
 </svg>
 <p className="text-[10px] text-slate-500 leading-5">
 Обход слева-направо α x β p γ не меняется никог
 числа поворотов. Единственный, кто меняет хозяи
 </p>
 </div>
);
};

```

/\* ----- рендер -----

```

const VIEW_W = 760;
const SLOT_W = 88;
const LEVEL_H = 76;

const layout = (root: SN | null) => {
 const pos = new Map<number, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: SN | null, d: number) => {
 if (!n) return;
 walk(n.left, d + 1);
 pos.set(n.key, { px: 56 + slot * SLOT_W, py: 40 + d * LEVEL_H });
 slot += 1;
 walk(n.right, d + 1);
 };
};

```

```
runKeyRef.current = runKey;
```

```
useEffect(() => {
 if (!playing || !steps) return;
 if (idx >= steps.length - 1) {
 setPlaying(false);
 baseRef.current = steps[steps.length - 1].root ??
 setTreeVers((v) => v + 1);
 return;
 }
 timer.current = setTimeout(() => setIdx((v) => v +
 return () => {
 if (timer.current) clearTimeout(timer.current);
 };
}, [playing, idx, steps]);
```

```
const step = steps ? steps[Math.min(idx, steps.length
const viewRoot = step ? step.root : baseRef.current;
const pos = useMemo(() => layout(viewRoot), [viewRoot
const nodes = useMemo(() => [...pos.entries()].map(({[
const edges = useMemo(() => {
 const es: { a: number; b: number }[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push({ a: n.key, b: n.left.key });
 if (n.right) es.push({ a: n.key, b: n.right.key });
 walk(n.left);
 walk(n.right);
 };
 walk(viewRoot);
 return es;
}, [viewRoot, treeVers, idx]);
```

```
const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.p
const autoH = Math.max(300, ...nodes.map((n) => n.py
```

```
const dBefore = useMemo(() => (steps ? depthOf(steps[
const dNow = steps ? depthOf(viewRoot, steps[steps.le
```

```

const strokeColor = (k: number): string => {
 if (!step) return "#64748b";
 if (step.kind === "search" && step.pathKeys?.includes(k)) return "#f59e0b";
 if ((step.kind === "aim" || step.kind === "cut") &&
 const e = step.aimEdge ?? step.cutEdge!;
 if (e.includes(k)) return "#f59e0b";
 }
 if (step.pathKeys?.includes(k)) return "#38bdf8";
 return "#64748b";
};

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
 {/* Шанка */}
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-indigo-600 text-white" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-indigo-300" >Клики
 </p>
 </div>
 </div>
 <div className="flex flex-wrap items-center gap-3" >
 <input
 value={input}
 onChange={(e) => setInput(e.target.value)}
 onKeyDown={(e) => {
 if (e.key === "Enter") {
 const v = parseInt(input, 10);
 if (!Number.isNaN(v)) {
 const has = inorderKeys(baseRef.current);
 if (has) runKey(v);
 else doInsert();
 setInput("");
 }
 }
 }}
 </input>
 </div>
 </div>
)

```

```

 <button onClick={() => { setIdx(0); setPlaying(
 ld bg-slate-700 text-white disabled:opacity-0.5
 В начало
 </button>
 <button onClick={() => { setPlaying(false); setPlaying(
 -lg text-xs font-bold bg-slate-700 text-white
 ← Шаг
 </button>
 <button onClick={() => setPlaying(!playing)} className="px-3 py-1.5 rounded-lg text-xs font-medium
 er-600" : "bg-emerald-600"}`}>
 {playing ? " Пауза" : "▶ Пуск"}
 </button>
 <button onClick={() => { setPlaying(false); setPlaying(
 className="px-3 py-1.5 rounded-lg text-xs font-medium
 Шаг →
 </button>

 Шаг {idx + 1} из {steps.length} • поворотов

 {step?.caseLabel && (

 00/60">
 {step.caseLabel}

)}
 </div>
)}

 {/* Дерево */}
 <div className="bg-indigo-950/20 rounded-xl p-4 border border-indigo-950/50">
 {!steps && (
 <div className="absolute top-3 left-3 bg-indigo-950/50 text-white font-bold">
 Кликни по любому узлу – splay покадрово
 </div>
)}
 <svg viewBox={`0 0 ${autoW} ${autoH}`} className="w-full h-full">
 {edges.map((e) => {

```

```

 className="cursor-pointer"
 >
 <circle
 r={isAimNode ? 24 : 20}
 fill={stepColor(n.key)}
 stroke={strokeColor(n.key)}
 strokeWidth={isAimNode ? 4 : 2}
 className="transition-all duration-300"
 />
 <text y={4} fill="#ffffff" fontSize="13"
 >
 {n.key}
 </text>
 </g>
);
}
}
</svg>

{/* Лог + механика */}
<div className="w-full flex flex-col lg:flex-row"
 <div className="flex-1 bg-slate-950/80 p-4 rounded"
 <p className="font-semibold text-amber-400"
 {step?.kind === "aim"
 ? " Прицел:"
 : step?.kind === "cut"
 ? "∞ Отстёгиваем среднее поддерево:"
 : step?.kind === "rot"
 ? " Поворот:"
 : step?.kind === "search"
 ? " Спуск:"
 : "Статус:"}
 </p>
 <p className="min-h-[40px] flex items-center"
 {step ? step.note : "Кликни по узлу или в
 ке справа – механика одного поворота."}
 </p>
 {steps && step && step.kind !== "search" &&
 <p className={`text-xs mt-2 ${checkOk ? "

```

```

</div>
<div className="w-full lg:w-1/2 bg-slate-900/60" >
 <h4 className="text-sm font-bold text-slate-200">
 <div className="bg-slate-950/80 rounded-lg p-2">
 {CODE_LINES.map((l) => {
 const zig = step?.caseLabel === "Zig" &&
 const zz1 = step?.caseLabel === "Zig-Zig"
 const zz2 = step?.caseLabel === "Zig-Zig"
 const za1 = step?.caseLabel === "Zig-Zag"
 const za2 = step?.caseLabel === "Zig-Zag"
 const active = zig || zz1 || zz2 || za1 || za2
 return (
 <div key={l.line} className={`py-1 px-2 border-400" : "text-slate-400"}`>

 {l.text}

 </div>
)
 })}
 </div>
 </div>
</div>
</div>
);
};

export default SplayWalkViz;

```

ФАЙЛ 70/94: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizSteps';

```

```

 isDirty: true,
 animState: "idle",
)))
 : dirtyStack;

```

```

const [cleanRack, setCleanRack] = useState<Plate[]>([
const [isWashing, setIsWashing] = useState(false);
const [showSoap, setShowSoap] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>([

```

```

const addLog = (msg: string) => setLog(prev => [msg,

```

```

// PUSH: Положить грязную тарелку сверху стопки

```

```

const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут р
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
}

```

```

const preset = PLATE_PRESETS[counter % PLATE_PRESETS
counter++;

```

```

const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
};

```

```

setDirtyStack(prev => [...prev, newPlate]);
addLog(` PUSH: Положили ${newPlate.emoji} сверху с

```

```

setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id === newP

```



```
const reset = () => {
 counter = 0;
 setDirtyStack([
 { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝',
 State: 'in' },
 { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕',
 imState: 'in' },
 { id: 'r3', name: 'Блюдец от торта', emoji: '🍰',
 mState: 'in' },
]);
 setCleanRack([]);
 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
};
```

```
const topIndex = displayStack.length - 1;
```

```
return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-blue-950/40 border border-blue-950">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именно самую верхнюю.
 Попытаешься вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>

 {/* УПРАВЛЕНИЕ */}
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 >Добавить тарелку</button>
 </div>
 </div>
)
```

```

 r-slate-800/80 text-[10px] font-mono">
 <span className="w-1.5 h-1.5 rounded-full b
 ПАКОВИНА
 </div>

 {/* Мыльные пузыри при мытье */}
 {showSoap && (
 <div className="absolute inset-0 pointer-ev
 <div className="absolute w-6 h-6 rounded-1
 "></div>
 <div className="absolute w-4 h-4 rounded-1
 "></div>
 <div className="absolute w-5 h-5 rounded-1
 "></div>
 <div className="absolute text-4xl anim-cr
 </div>
)}

 {displayStack.length === 0 ? (
 <div className="flex-1 flex flex-col items-
 <span className="text-6xl mb-3 animate-pu
 <p className="text-white font-black text-1
 <p className="text-slate-500 text-xs mt-1
 </div>
) : (
 <div className={`flex-1 flex flex-col justifi

 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-
 style={{ bottom: `${24 + topIndex * 44}
 >
 <span className="text-xs font-mono text
 er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 <span className="text-blue-400 animate-

```

```

 Сушилка для чистой посуды
 </div>

```

```

<div className="absolute top-4 right-4 flex items-center justify-center"
 order-emerald-800/80 text-[9px] font-mono">
 <Sparkles className="w-3.5 h-3.5 animate-pulse" />
 ЧИСТО
</div>

```

```

<div className="flex-1 flex flex-col justify-between"
 {cleanRack.length === 0 ? {
 <div className="flex-1 flex flex-col items-center justify-center"

 <p className="text-xs font-bold font-mono">Пусто!</p>
 <p className="text-[10px] mt-1">Здесь будет сушилка
 </div>
 } : {
 <div className="flex flex-col gap-2 w-full"
 {cleanRack.map(plate => {
 <div
 key={plate.id}
 className="w-full h-8 rounded-full border-1
 items-center justify-center gap-2 opacity-0.5"
 >
 {plate.name}

 {plate.status}
 </div>
 })}
 </div>
)}
</div>

```

```

 {/* Столешница сушилки */}
 <div className="w-full h-4 bg-gradient-to-r from-emerald-500 to-emerald-200" />
 </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}

```

```

 steps.push({ i, j, pi: [...pi], highlight: [i,
 ;

 while (j > 0 && s[i] !== s[j]) {
 steps.push({ i, j, pi: [...pi], highlight:
 яд назад: откат j = pi[${j} - 1] = ${pi[j] -
 j = pi[j] - 1};
 }

 if (s[i] === s[j]) {
 steps.push({ i, j, pi: [...pi], highlight:
 ска увеличивается.` });
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight:
 овпадения нет.` });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записывае

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
}
return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

```

```

}

export function StringAlgorithmsViz({ defaultMode = "kmp"
 const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
 const [pattern, setPattern] = useState("aba");
 const [text, setText] = useState("abacaba");

 // Ensure inputs are valid dynamically
 const safePattern = pattern || " ";
 const safeText = text || " ";

 const { s: demoString, steps, patternLength: demoPatternLength } =
 if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
 else return generateZSteps(safePattern, safeText);
 }, [mode, safePattern, safeText]);
 const runtime = useVizRuntime();
 const runtimeString = vizString(runtime?.variables?.string);
 const runtimePattern = vizString(runtime?.variables?.pattern);
 const s = runtimeString ?? demoString;
 // Длина образца: приоритет – переменная pattern из
 // разделителя "#" в склейке pattern#text; иначе – demoPatternLength
 const patternLength = runtimePattern
 ? runtimePattern.length
 : demoPatternLength
 ? (runtimeString.includes("#") ? runtimeString.length : demoPatternLength)
 : demoPatternLength;

 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);
 const timer = useRef<NodeJS.Timeout | null>(null);

 // Reset when inputs or mode change
 useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
 }, [s, mode]);

 // Timer loop for autoPlay

```

```

return (
 <div className="space-y-4">
 <div className="flex items-center justify-b
 <div className="flex bg-slate-900 borde
 <button onClick={() => setMode("kmp
 className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
 КМП (π-ФУНКЦИЯ)
 </button>
 <button onClick={() => setMode("z")
 className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
 Z-ФУНКЦИЯ
 </button>
 </div>

 <div className="flex items-center gap-2
 <div className="bg-slate-800 p-1 fl
 <span className="text-[10px] te
 <input value={pattern} onChange=
xt-white text-xs font-bold font-mono px-2 p
 </div>
 <div className="bg-slate-800 p-1 fl
 <span className="text-[10px] te
 <input value={text} onChange={e
te text-xs font-bold font-mono px-2 py-1 ou
 </div>
 </div>
 </div>

 <div className="bg-slate-950 rounded-xl bor
px] flex items-center justify-start">
 <div className="flex gap-1.5 px-4 h-ful
 {s.split("").map((char, index) => {
 const isPattern = index < patte
 const isSeparator = index === p

 let borderColor = "border-slate

```

```

 { /* Char Box */
 <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
 {char}
 </div>

 { /* Value array box */
 <div className="text-[1
slate-700/50 font-mono font-bold">
 {(mode === "kmp" ?
 </div>

 { /* KMP fingers */
 {mode === "kmp" && step
 <div className="abs
z-20">
 <span className
 </div>
 })
 {mode === "kmp" && step
 <div className="abs
 <span className
 </div>
 })
 {mode === "kmp" && step
 <div className="abs
20">
 <span className
 </div>
 })

 { /* Z fingers */
 {mode === "z" && step.i
 <div className="abs
e z-20">
 <span className
 </div>
 })
 }

```

```

 r:bg-slate-800 rounded-lg disabled:opacity-0
 <SkipForward strokeWidth={3} size={24} />
 </button>
 <button onClick={reset} className="border border-slate-800"
 <RefreshCw strokeWidth={3} size={24} />
 </button>
 </div>

```

```

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50 overflow-hidden">
 <div className="absolute top-0 right-0"
 <span className="font-mono text-sm text-slate-400"
 {steps.length}</div>
 <div className="text-sm text-slate-400"
 {step.desc}
 </div>
 </div>
 </div>

```

```

{/* Why this matters card */}
<div className="bg-slate-800 p-4 rounded-xl"

 <p className="text-xs text-slate-400"
 Обрати внимание на хитрость с решёткой.
 Мы пишем Шаблон + # + Текст.
 Когда значени {mode === 'kmp' ? "пройди шаги вручную, посмотри, как избежать пропуска работы (в Z)."
 </p>
</div>

```

```

{/* Code Sneak Peek */}
<div className="bg-slate-900 border border-slate-800"

```



```

 children: React.ReactNode;
 /** С какой стороны показывать. По умолчанию – сверху
 side?: "top" | "bottom";
 className?: string;
 }

 /**
 * Всплывающая подсказка (как title="", только красивая
 * Появляется по наведению курсора и по фокусу с клавиатуры
 */
 export const Tooltip: React.FC<TooltipProps> = ({ content, className }) => {
 const [open, setOpen] = useState(false);

 const hiddenShift = side === "top" ? "translate-y-1" : "translate-y-0";

 return (
 <span
 className={`relative inline-flex ${className ?? ""} ${
 side === "top" ? "transform" : "transform"
 }`}
 onMouseEnter={() => setOpen(true)}
 onMouseLeave={() => setOpen(false)}
 onFocus={() => setOpen(true)}
 onBlur={() => setOpen(false)}
 >
 {children}
 <span
 role="tooltip"
 className={`pointer-events-none absolute left-1/2 top-1/2
 bg-slate-900 px-3 py-2 text-left text-[11px]
 duration-150 ${
 side === "top" ? "bottom-full mb-2" : "top-full mt-2"
 } ${open ? "visible translate-y-0 opacity-100" : "invisible"}
 `}
 >
 {content}
 <span
 className={`absolute left-1/2 top-1/2 -translate-x-50% -translate-y-50%
 side === "top" ? "-bottom-[5px] border-b border-slate-900" : ""
 `}
 />

);
 };

```

```

 { id: 4, x: 640, y: 150, label: "4", name: "Машинное"
};
```

```
const dagEdges: TEdge[] = [
 { u: 0, v: 1 },
 { u: 0, v: 2 },
 { u: 1, v: 3 },
 { u: 2, v: 3 },
 { u: 3, v: 4 },
];
```

```
interface TStep {
 desc: string;
 visited: Set<number>;
 fullyProcessed: Set<number>;
 currentNode: number | null;
 topoList: number[];
 dfsStack: number[];
}
```

```
export const TopologicalSortViz: React.FC = () => {
 const [steps, setSteps] = useState<TStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode !== v));
 return candidates.at(-1)?.index ?? null;
});
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const listSteps: TStep[] = [];
 const visited = new Set<number>();
```

```

 `⚠ Внимание: Рёбра в серую вершину ${to} о
 u,
);
 }
}

fullyProcessed.add(u);
dfsStack.pop();
topoList.push(u); // prepend behavior: we write i
recordStep(
 `DFS: закончили обработку '${dagNodes[u].name}'
 ,
 u,
);
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 recordStep(
 `Запускаем новый обход DFS от нетронутой верши
 null,
);
 dfs(i);
 }
}

recordStep(
 "Топологическая сортировка завершена! Полученный
 null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {
 desc: "Запуск...",
 visited: new Set(),

```

```

<div className="flex items-center gap-2 bg-slate-500"
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-3 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 className="flex items-center justify-center gap-2 px-3 py-3 text-white"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

```

```

<div className="grid grid-cols-1 lg:grid-cols-12"
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] relative"
 <svg className="w-full h-full max-w-[650px]"
 <defs>
 <marker
 id="dag-arrow"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"

```

```
}}}
```

```
{/* Nodes */}
```

```
{dagNodes.map({node} => {
 const isCurrent = step.currentNode === node.id
 const isVisited = step.visited.has(node.id)
 const isProcessed = step.fullyProcessed.has(node.id)
```

```
 let fill = "#1e293b";
 let stroke = "#334155";
 let textColor = "text-slate-400";
 void textColor;
```

```
 if (isCurrent) {
 fill = "#312e81";
 stroke = "#818cf8";
 textColor = "text-white font-bold";
 } else if (isProcessed) {
 fill = "#022c22";
 stroke = "#059669";
 textColor = "text-emerald-300";
 } else if (isVisited) {
 fill = "#3730a3";
 stroke = "#6366f1";
 textColor = "text-indigo-200";
 }
}
```

```
return (
 <g key={node.id}>
 <rect
 x={node.x - 55}
 y={node.y - 22}
 width="110"
 height="44"
 rx="8"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
```

```

 {step.desc}
 </p>
 </div>

 <div className="flex-1 space-y-4 overflow-y-auto" >
 {/* Topological List output */}
 <div>

 РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):

 <div className="bg-slate-900/60 p-2.5 rounded-3xl pr-1 text-slate-300">
 {step.topoList.length === 0 ? (

 ожидаем обработку узлов...

) : (
 getTopoListString()
)}
 </div>
 </div>

 {/* Color key */}
 <div className="space-y-1">

 ВЕРШИНЫ В DFS:

 <div className="flex items-center gap-2 text-[10px]" >

 Серые (зашли)

 </div>
 <div className="flex items-center gap-2 text-[10px]" >

 Черные (готово)

 </div>
 </div>
 </div>
 </div>
</div>

```

```
import React, { useEffect, useMemo, useState } from "react";
import { useVizRuntime, vizNumber } from "../data/vizState";

/**
 * Интерактивный Tgear – шесть режимов:
 * build – сборка: ВИДИМАЯ сортировка точек (и чем сортировка
 * координатная плоскость НАВЕРХУ (ассоциативная сортировка)
 * чек-лист инвариантов (автопроверка, как у сортировки)
 * split – разрез по x с двумя корзинами (грамотная сортировка)
 * merge – склейка двух деревьев по y.
 * erase – удаление ПРОИЗВОЛЬНОЙ точки: найти по x, удалить по y.
 * layers – миф «дети в массиве по $2k/2k+1$ »: работает
 * search – «отсортирую бинарным поиском»: поиск не сортировкой
 * сортировка платит $\geq n \cdot \log n$ сравнений, сортировка
 *
 * Канонический набор: все x и все y различны $\Rightarrow$ дерево
 * Узел идентифицируется парой $id = "x:y"$.
 */

export interface Pair {
 key: number;
 grade: number;
}

type NodeId = string;
const pairId = (key: number, grade: number): NodeId =>
 `${key}:${grade}`;

interface TNode {
 id: NodeId;
 key: number;
 grade: number;
 left: TNode | null;
 right: TNode | null;
}

export const POINTS: Pair[] = [
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
];
```

-----  
}

```
export interface SplitResult {
 l: TNode | null;
 r: TNode | null;
 trace: { atId: NodeId; goLeft: boolean; note: string }
}
export function splitTree(root: TNode | null, x: number) {
 const trace: SplitResult["trace"] = [];
 const go = (t: TNode | null): { l: TNode | null; r: TNode | null } {
 if (!t) return { l: null, r: null };
 if (t.key <= x) {
 const sub = go(t.right);
 trace.push({ atId: t.id, goLeft: true, note: `(${t.key} >= x)
 о правое поддерево.` });
 t.right = sub.l;
 return { l: t, r: sub.r };
 }
 const sub = go(t.left);
 trace.push({ atId: t.id, goLeft: false, note: `(${t.key} < x)
 о левое поддерево.` });
 t.left = sub.r;
 return { l: sub.l, r: t };
 };
 const res = go(root);
 return { l: res.l, r: res.r, trace };
}
```

```
export interface MergeResult {
 root: TNode | null;
 trace: { aId: NodeId | null; bId: NodeId | null; chosen: string }
}
export function mergeTree(a: TNode | null, b: TNode | null) {
 const trace: MergeResult["trace"] = [];
 const go = (a: TNode | null, b: TNode | null): TNode | null {
 if (!a || !b) {
 trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, chosen: `
 другое целиком.` });
 return a || b;
 }
 if (a.key < b.key) {
 const sub = go(a.right, b);
 a.right = sub;
 return go(a, b);
 }
 if (a.key > b.key) {
 const sub = go(a, b.right);
 b.right = sub;
 return go(a, b);
 }
 // a.key === b.key
 const sub = go(a.right, b.right);
 a.right = sub;
 b.right = sub;
 return go(a, b);
 };
 const res = go(a, b);
 return { root: res, trace };
}
```



```

const cloneTree = (t: TNode | null): TNode | null =>
 t ? { id: t.id, key: t.key, grade: t.grade, left: cloneTree(t.left), right: cloneTree(t.right) } : null

/* ----- валидация treap (для чек-листа и стресс-теста) ----- */

export function checkTreap(root: TNode | null): { bst: boolean, heap: boolean } {
 let bst = true;
 let heap = true;
 let prev: number | null = null;
 const walk = (n: TNode | null, min: number, max: number, parentGrade: number | null): void => {
 if (!n) return;
 if (n.key <= min || n.key > max) bst = false;
 if (parentGrade !== null && n.grade > parentGrade) heap = false;
 walk(n.left, min, n.key, n.grade);
 if (prev !== null && n.key < prev) bst = false;
 prev = n.key;
 walk(n.right, n.key, max, n.grade);
 };
 walk(root, -Infinity, Infinity, null);
 return { bst, heap };
}

/* ----- раскладка сцены дерева ----- */

const layoutTree = (root: TNode | null) => {
 const pos = new Map<NodeId, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 10 });
 slot += 1;
 walk(n.right, depth + 1);
 };
 walk(root, 0);
 return pos;
};

```

```

 const b = byId(e.to);
 if (!a || !b) return null;
 return (
 <line
 key={`te-${e.from}-${e.to}`}
 x1={a.px} y1={a.py} x2={b.px} y2={b.py}
 stroke={edgeColor ? edgeColor(e.from, e.to) : "#ccc"}
 strokeWidth={2.5}
 strokeDasharray={edgeDash && edgeDash(e.from, e.to)}
 className="transition-all duration-500"
 />
);
 })}
 {nodes.map((n) => {
 const p = pairParts(n.id);
 const pulse = pulseId === n.id;
 return (
 <g key={`tn-${n.id}`} transform={`translate(${n.px}, ${n.py})`}
 <circle r={pulse ? 24 : 20} fill={nodeColor(n)}
 strokeWidth={pulse ? 4 : 2} className="transition-all duration-500" />
 <text y={-2} fill="#fff" fontSize="13" fontWeight="bold">{n.name}</text>
 <text y={13} fill="#cbd5e1" fontSize="10">{n.id}</text>
 </g>
);
 })}
 </svg>
 {subtitle && <p className="text-xs text-slate-400">{subtitle}</p>}
</div>
);
};

const pairParts = (id: NodeId): [number, number] => {
 const [a, b] = id.split(":").map(Number);
 return [a, b];
};

/* ----- плоскость (клетчатая бумага) ----- */

```

```

{[-4, 0, 4, 8].map((g) => {
 <text key={`ty-${g}`} x={PX(1) - 14} y={PY(g) +
 }}}
<text x={PL_W - 14} y={PY(0) - 8} fill="#94a3b8"
<text x={PX(1) - 26} y="14" fill="#94a3b8" fontSi

{drawnEdges.map((e, i) => {
 const a = byPair(e.a);
 const b = byPair(e.b);
 const isBad = badEdge && ((badEdge.a.key === e.a.
 ade === e.b.grade) || (badEdge.a.key === e.b.
 e === e.a.grade));
 return <line key={`pe-${i}`} x1={a.x} y1={a.y} :
 2} strokeDasharray={isBad ? "4,3" : "none"}
 }}}

{/* временная линия игры */}
{interactive && selected && cursor && (
 <line
 x1={PX(selected.key)} y1={PY(selected.grade)}
 stroke="#818cf8" strokeWidth={2.5} strokeDash
 />
)}

{points.map((p) => {
 const done = drawnEdges.some((e) => (e.a.key ===
 const isSel = selected && selected.key === p.key
 return (
 <g
 key={`pp-${p.key}:${p.grade}`}
 className="transition-all duration-500"
 onPointerDown={interactive && onPointDown ?
 onPointerUp={interactive && onPointUp ? () :
 style={interactive ? { cursor: "pointer" }
 >
 <circle cx={PX(p.key)} cy={PY(p.grade)} r={
 troke={isSel ? "#a5b4fc" : done ? "#38bdf8"
 <text x={PX(p.key)} y={PY(p.grade) + 3.5} f

```

```

edges.find((e) => gpid(e.b) === id);
const reaches = (edges: GEdge[], from: Pair, target: Pair): boolean {
 let cur: Pair | undefined = from;
 const seen = new Set<string>();
 while (cur) {
 if (sameP(cur, target)) return true;
 if (seen.has(gpid(cur))) return false;
 seen.add(gpid(cur));
 cur = parentOf(edges, gpid(cur))?.a;
 }
 return false;
};

```

```

/** Проверка BST леса: у узла ≤1 ребёнка слева и ≤1 справа
export function forestIsBst(edges: GEdge[], all: Pair[]): boolean {
 const kids = new Map<string, Pair[]>();
 for (const e of edges) kids.set(gpid(e.a), [...(kids.get(gpid(e.b)) || []), e]);
 const hasParent = new Set(edges.map((e) => gpid(e.b)));
 const roots = all.filter((p) => !hasParent.has(gpid(p.a)));
 const ok = (n: Pair | undefined, lo: number, hi: number): boolean {
 if (!n) return true;
 if (n.key <= lo || n.key > hi) return false;
 const ch = (kids.get(gpid(n.a)) ?? []).slice().sort((a, b) => a.key - b.key);
 const left = ch.filter((c) => c.key < n.key);
 const right = ch.filter((c) => c.key > n.key);
 if (left.length > 1 || right.length > 1) return false;
 return ok(left[0], lo, n.key) && ok(right[0], n.key, hi);
 };
 return roots.every((r) => ok(r, -Infinity, Infinity));
}

```

```

/** Полный вердикт по текущим линиям. */
export function gameVerdict(edges: GEdge[], all: Pair[]): boolean {
 const heap = edges.every((e) => e.a.grade > e.b.grade);
 const oneParent = new Set(edges.map((e) => gpid(e.b)));
 const acyclic = !edges.some((e) => reaches(edges, e.a, e.b));
 const bst = forestIsBst(edges, all);
 const roots = all.filter((p) => !edges.some((e) => gpid(e.b) === gpid(p.a)));

```

```

 }
 const id = `${nums[0]}:${nums[1]}`;
 if (seen.has(id)) {
 errors.push(`точка (${nums[0]}; ${nums[1]}) дважды`);
 continue;
 }
 seen.add(id);
 pairs.push({ key: nums[0], grade: nums[1] });
}
const xs = pairs.map((p) => p.key);
const ys = pairs.map((p) => p.grade);
return { pairs, errors, equalX: new Set(xs).size !== 1 };
};

const DEMO_RAW = "45 8, 3 6, 6 5, 2 4, 9 3, 7 2, 12 0,

/* ----- игра «соедини мышкой»: чистая логика

/** Ребро игры: a = родитель (выше по y), b = ребёнок.
export interface GameEdge {
 a: Pair;
 b: Pair;
}
const pid = (p: Pair) => `${p.key}:${p.grade}`;

export function canConnect(
 edges: GameEdge[],
 x: Pair,
 y: Pair,
): { ok: boolean; edge?: GameEdge; reason?: string } {
 if (x.key === y.key && x.grade === y.grade) return { ok: false, reason: "такая линия уже проведена" };
 for (const e of edges) {
 if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) ||
 (pid(e.a) === pid(y) && pid(e.b) === pid(x)))
 return { ok: false, reason: "такая линия уже проведена" };
 }
 if (x.grade === y.grade) return { ok: false, reason: "ребёнок не может быть старше родителя" };
 const [parent, child] = x.grade > y.grade ? [x, y] : [y, x];

```

```

 x = comp.get(x)!;
 }
 return x;
};
for (const e of edges) {
 const ra = find(pid(e.a));
 const rb = find(pid(e.b));
 if (ra !== rb) comp.set(ra, rb);
}
const comps = new Set(points.map((p) => find(pid(p))))
const connected = comps.size === 1;
const countOk = edges.length === n - 1;
// BST: дети по x, inorder не убывает; корень один; э
const children = new Map<string, Pair[]>();
for (const e of edges) children.set(pid(e.a), [...(ch
const roots = points.filter((p) => !parentCount.has(p
let bstOk = roots.length === 1;
if (bstOk) {
 const seen = new Set<string>();
 const inorder: number[] = [];
 const walk = (p: Pair): void => {
 if (seen.has(pid(p))) {
 bstOk = false; // цикл
 return;
 }
 seen.add(pid(p));
 const kids = children.get(pid(p)) ?? [];
 if (kids.length > 2) {
 bstOk = false;
 return;
 }
 const sorted = [...kids].sort((u, v) => u.key - v
 let left: Pair | null = null;
 let right: Pair | null = null;
 if (sorted.length === 2) {
 if (sorted[0].key === sorted[1].key || sorted[0
 bstOk = false; // двое детей с равным x или с
 return;
 }
 }
}

```

```

 nodes.set(pid(p), node);
 const kids = edges.filter((e) => pid(e.a) === pid(p));
 if (kids.length > 2) return node; // невалидно – по
 kids.sort((u, v) => u.key - v.key);
 const left = kids.filter((k) => k.key <= p.key); //
 const right = kids.filter((k) => k.key > p.key);
 if (left.length > 1 || right.length > 1) return node;
 if (left[0]) node.left = build(left[0]);
 if (right[0]) node.right = build(right[0]);
 return node;
 };
 return build(rootPair);
}

```

```

const BuildMode: React.FC = () => {
 const [raw, setRaw] = useState(DEMO_RAW);
 const [phase, setPhase] = useState<BuildPhase>("sort");
 const [algo, setAlgo] = useState<SortAlgo>("none");
 const [order, setOrder] = useState<ConnectOrder>("y-d");
 const [sortIdx, setSortIdx] = useState(0);
 const [connIdx, setConnIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [sortCost, setSortCost] = useState<string | null>();
 const live = useTreapLive();

```

```

 const parsed = useMemo(() => parsePairs(raw), [raw]);
 const userPairs = parsed.errors.length ? [] : parsed.pairs;

```

```

 const n = userPairs.length;

```

```

 const userTree = useMemo(() => (userPairs.length >= 1 ?
 const canonEdges = useMemo(() => {
 const es: { a: Pair; b: Pair }[] = [];
 const walk = (node: TNode | null) => {
 if (!node) return;
 if (node.left) { es.push({ a: { key: node.key, gr
 left); }
 if (node.right) { es.push({ a: { key: node.key, g

```

```

const connEdgesAll = order === "y-desc" ? canonEdges
const connEdges = connEdgesAll.slice(0, connIdx);
const connDone = phase === "connect" && n >= 1 && conn

// --- игра «соедини сам» ---
const [gameEdges, setGameEdges] = useState<GameEdge[]>();
const [sel, setSel] = useState<Pair | null>(null);
const [cursor, setCursor] = useState<{ x: number; y: number}>();
const [gameMsg, setGameMsg] = useState<string | null>(null);
const [badEdge, setBadEdge] = useState<GameEdge | null>(null);
const [hintUsed, setHintUsed] = useState(false);

const resetGame = () => {
 setGameEdges([]);
 setSel(null);
 setCursor(null);
 setGameMsg(null);
 setBadEdge(null);
 setHintUsed(false);
};

const startGame = () => {
 resetGame();
 setPhase("game");
 setPlaying(false);
};

const verdict = useMemo(() => checkGameState(userPair), [userPair]);
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree(connEdges) : null), [connEdges, gameWon]);

const attemptConnect = (x: Pair, y: Pair) => {
 const r = canConnect(gameEdges, x, y);
 if (r.ok && r.edge) {
 setGameEdges((prev) => [...prev, r.edge!]);
 setGameMsg(r.reason ?? null);
 setBadEdge(null);
 } else {
 setGameMsg(r.reason ?? "так нельзя");
 }
};

```



```
};
```

```
return (
 <div className="space-y-5">
 {/* Произвольный список */}
 <div className="bg-slate-900/60 rounded-xl p-4 border border-slate-800">
 <div className="flex flex-wrap items-center justify-between">
 <h4 className="text-sm font-bold text-slate-300">Список
 <button onClick={() => { setRaw(DEMO_RAW); setRaw(
 py-1 rounded-lg text-xs font-bold bg-slate-800
 демо-набор
 </button>
 </div>
 <textarea
 value={raw}
 onChange={(e) => { setRaw(e.target.value); setRaw(
 rows={2}
 className="w-full bg-slate-950 border border-slate-800
 s:border-emerald-500"
 placeholder="3 5, 45 8, 1 -4 ..."
 />
 {parsed.errors.length > 0 && (
 <div className="mt-2 text-xs text-rose-300 bg-slate-900/60 border border-slate-800">
 {parsed.errors.map((e, i) => <div key={i}>{e.message}
 </div>
)}
 {parsed.errors.length === 0 && (parsed.equalX || parsed.equalY) && (
 <div className="mt-2 text-xs text-amber-300 bg-slate-900/60 border border-slate-800">
 {parsed.equalY && <div>⚡ Есть равные y (сортировка ста
 кто раньше в списке (сортировка ста
 {parsed.equalX && <div>⚡ Есть равные x (сортировка ста
 de.key → left).</div>}
 </div>
)}
 </div>
)}
 </div>

 {/* Координатная плоскость – НАВЕРХУ */}
```

```

 onClick={startGame}
 disabled={n < 2}
 className={`px-3 py-1.5 rounded-lg text-
go-600 text-white" : "bg-indigo-500/80 text
 >
 соединить сам
 </button>
 </>
 })
</div>
</div>
<Plane
 points={userPairs}
 drawnEdges={phase === "game" ? gameEdges : ph
 pending={null}
 interactive={phase === "game"}
 selected={sel}
 cursor={cursor}
 badEdge={badEdge}
 onPointDown={pointDown}
 onPointUp={pointUp}
 onMove={planeMove}
/>
<div className="mt-2 text-xs text-slate-300 bg-
 {phase === "sort" ? (
 algo === "none" ? (
 <>Ввод приходит как попало ({n} па
 ть.</>
) : sortIdx < 3 ? (
 algo === "quicksort" ? (
 <>⚡ quicksort: счётчик сравнений: <b cl
 {comparisons} (n·log2n).</>
) : (
 <> counting: точки в корзины по значени
 , потому что у – маленькие целые.</>
)
) : (
 <>Отсортировано ({sortCost ?? "способ не

```

```

 </>
 })

 {phase === "game" && (
 <div className="flex flex-wrap gap-2 mt-3">
 <button onClick={undoEdge} disabled={!gameE
 white disabled:opacity-30">↵ Убрать линию</
 <button onClick={resetGame} disabled={!gameE
 ate-700 text-white disabled:opacity-30"> С
 <button onClick={showSolution} disabled={ga
 te disabled:opacity-30"> Показать решение<
 </div>
)}
 </div>

 {/* Победное дерево в игре */}
 {phase === "game" && gameTree && (
 <div className="bg-emerald-950/20 rounded-xl p-
 <h4 className="text-sm font-bold text-emerald
 <TreeView root={gameTree} subtitle="Слева-нап
 ществует." />
 </div>
)}

 {/* Дерево */}
 {order === "y-desc" && userTree && (
 <div className="bg-slate-900/40 rounded-xl p-4
 <h4 className="text-sm font-bold text-slate-3
 <TreeView root={userTree} subtitle="Слева-нап
 </div>
)}
</div>
);
};

/* ===== РЕЖИМ SPLIT ===== */

const SplitMode: React.FC = () => {

```

```

 root={fullTree}
 nodeColor={(id) => (inL.has(id) ? "#1e3a8a" :
 nodeStroke={(id) => (inL.has(id) ? "#60a5fa"
 subtitle={done ? "Каждый узел покрашен ровно
 />
 </div>

 <div className="text-xs text-slate-300 bg-slate-900"
 {steps[idx]?.note}
 </div>

 {done && (
 <div className="grid grid-cols-1 md:grid-cols-2"
 <div className="bg-sky-950/20 rounded-xl p-3"
 <h5 className="text-sm font-bold text-sky-300"
 <TreeView root={lTree} width={480} height={
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3"
 <h5 className="text-sm font-bold text-rose-:
 <TreeView root={rTree} width={480} height={
 </div>
 </div>
)}
)}

 <div className="text-xs text-slate-400 bg-slate-900"
 <LiveChip live={live} />{' '}
 Как это работает на самом деле: идём с
 деревом), отдаётся вместе с веткой – д
 </div>
</div>
);
};

/* ===== РЕЖИМ MERGE ===== */

const MergeMode: React.FC = () => {
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();

```

```

 </div>

 {done && (
 <div className="bg-emerald-950/20 rounded-xl p-4">
 <h5 className="text-sm font-bold text-emerald-500">
 <TreeView root={merged.root} subtitle="merge">
 </div>
)}

 <div className="text-xs text-slate-400 bg-slate-900 p-4">
 Грамотно про «слияние»: у корней сравним
 в (то, что сохраняет порядок x) отправляется
 </div>
 </div>
);
};

/* ===== РЕЖИМ ERASE ===== */

const EraseMode: React.FC = () => {
 const [key, setKey] = useState(9);
 const [stepIdx, setStepIdx] = useState(0);
 const fullTree = useMemo(() => buildTree(POINTS), [])
 const res = useMemo(() => eraseNode(buildTree(POINTS), key), [key])
 const steps = res.trace;
 const live = useTreapLive();
 // Компилятор: x – ключ удаляемой вершины, i – шаг слияния
 useEffect(() => { if (live.x !== null && POINTS.some((v) => v.id === x)) {
 useEffect(() => { if (live.i !== null) setStepIdx(Math.min(stepIdx, steps.length - 1));
 const idx = Math.min(stepIdx, steps.length - 1);
 const done = idx >= steps.length - 1;

 const pathIds = useMemo(() => {
 const ids: NodeId[] = [];
 for (const s of steps.slice(0, idx + 1)) if (s.atId === x) ids.push(s.id);
 return ids;
 }, [steps, idx]);
 const pathSet = new Set(pathIds);
 }
 }, [steps, idx]);
 const pathSet = new Set(pathIds);

```

```

 {steps[idx]?.note}
 </div>

 <div className="text-xs text-slate-400 bg-slate-900 p-2">
 Ответ на «убрать можно только самый верхний»:
 Ягивает merge двух детей. Удаляется любая из них.
 </div>
 </div>
);
};

/* ===== РЕЖИМ LAYERS (2k/2k+1) ===== */

const LayersMode: React.FC = () => {
 const [pick, setPick] = useState<"heap" | "treap">("heap");
 // Полное дерево из 7 узлов (куча): индексы 1..7
 const heapArr = [0, 98, 45, 90, 32, 21, 76, 12];
 const full = useMemo(() => buildTree(POINTS), []);
 // Честная слотовая разметка treap: индексы как в куче
 const slots = useMemo(() => {
 const arr: (string | null)[] = Array(16).fill(null);
 const walk = (n: TNode | null, i: number) => {
 if (!n || i >= 16) return;
 arr[i] = n.id;
 walk(n.left, 2 * i);
 walk(n.right, 2 * i + 1);
 };
 walk(full, 1);
 return arr;
 }, [full]);
 const nodeCount = slots.filter(Boolean).length;

 return (
 <div className="space-y-4">
 <div className="flex gap-2">
 <button onClick={() => setPick("heap")} className="text-white bg-slate-700 text-slate-300">heap</button>
 <button onClick={() => setPick("treap")} className="text-white bg-slate-700 text-slate-300">treap</button>
 </div>
 <div>
 Полное дерево из 7 узлов (куча): индексы 1..7
 <pre>
graph TD
 1((1)) --- 2((2))
 1 --- 3((3))
 2 --- 4((4))
 2 --- 5((5))
 3 --- 6((6))
 3 --- 7((7))
 </pre>
 Куча: [0, 98, 45, 90, 32, 21, 76, 12]
 Честная слотовая разметка treap: индексы как в куче
 <pre>
graph LR
 slots[slots] --- arr[arr]
 arr --- walk[walk]
 walk --- n[n]
 walk --- i[i]
 </pre>
 </div>
 </div>
);
};

```

```

 то левая ветка ушла на 4 уровня и заняла два
 казателями (left/right), а не арифметикой.<
 </div>
 })

 <div className="text-xs text-slate-400 bg-slate-900 p-2">
 Грабля «дети в массиве по 2k/2k+1»: формирова
 ивым: слои дырявые, и «адрес» ребёнка посчитан
 </div>
</div>
);
};

/* ===== РЕЖИМ SEARCH (поиск ≠ сортировка) ===== */

const SearchMode: React.FC = () => {
 // Подобрано так, что бинарный поиск ГАРАНТИРОВАННО пр
 // mid (индекс 3) = 9 > 7 → уходит влево, а цель лежи
 const shuffled: Pair[] = [
 { key: 12, grade: 0 },
 { key: 3, grade: 6 },
 { key: 45, grade: 8 },
 { key: 9, grade: 3 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 7, grade: 2 },
 { key: 1, grade: -4 },
];
 const target = 7;
 const sortedByKey = [...shuffled].sort((a, b) => a.key - b.key);
 const [stage, setStage] = useState<0 | 1 | 2>(0); // 0 - начало
 const [bidx, setBidx] = useState(0);

 // шаги бинарного поиска на отсортированном массиве
 const bsSteps = useMemo(() => {
 const steps: { lo: number; hi: number; mid: number; } = {
 lo: 0, hi: sortedByKey.length - 1, mid: 0;
 };
 while (lo <= hi) {

```

```

const isFound = stage === 2 && bidx >= bsSteps.length - 1
return (
 <div key={` ${p.key}:${p.grade}-${i}`} className="position-all ${highlight ? "border-rose-500 border-2 border-solid" : "border-amber-500 border-2 border-solid"} bg-amber-950/30" : inBs ? "border-rose-500 border-2 border-solid" : "border-amber-500 border-2 border-solid">
 <div className="text-white font-bold">{p.key}</div>
 <div className="text-slate-400 text-[10px]>{p.grade}</div>
 </div>
);
}}}
</div>

<div className="text-xs text-slate-300 bg-slate-900/50 p-10">
 {stage === 0 && <>Массив пар не отсортирован и находится в беспорядке.</>}
 {stage === 1 && <> Бинарный поиск посмотрел на первую половину... в которой лежала цель. На следующем шаге он представил: массив как был в беспорядке, так и после сортировки.
 {stage === 2 && bidx < bsSteps.length - 1 && <>
 {stage === 2 && bidx >= bsSteps.length - 1 && <>
 сравнения. Но чтобы поиск вообще заработал, нужно было сделать сравнения на всех. «Отсортирую бинарным поиском».
 </div>

 <div className="text-xs text-slate-400 bg-slate-900/50 p-10">
 Есть ли разница, чем сортировать? Сравнение границ, каждое сравнение даёт 1 бит. Но для маленьких целых у (вкладка «Собрать», кнопка «Сортировать») задать порядок соединения.
 </div>
</div>
);
};

/* ===== РЕЖИМ GAME ===== */

const GameMode: React.FC = () => {
 const [edges, setEdges] = useState<GEdge[]>([]);

```



```

3 py-1.5 rounded-lg text-xs font-bold bg-slate-400
<button onClick={() => { setEdges(CANON_GAME.map(
); }} className="px-3 py-1.5 rounded-lg text-slate-400"
линий:
</div>

```

```

<div className="bg-emerald-950/20 rounded-xl p-4" style={{ width: 1000px, height: 1000px }}>
 <svg viewBox={`0 0 ${PL_W} ${PL_H + 20}`} className="w-full h-full">
 <defs>
 <pattern id="gm-grid" width="14" height="14">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="black" stroke-width="1" />
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height={PL_H} fill="none" stroke="black" stroke-width="1" />
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)} stroke="black" stroke-width="1" />
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PL_H} stroke="black" stroke-width="1" />
 {edges.map((e, i) => {
 <line
 key={`ge-${i}`}
 x1={PX(e.a.key)} y1={PY(e.a.grade)}
 x2={PX(e.b.key)} y2={PY(e.b.grade)}
 stroke={i === edges.length - 1 ? "#34d399" : "black"}
 strokeWidth={i === edges.length - 1 ? 3.5 : 1}
 className="transition-all duration-300"
 />
 })}
 </svg>
</div>

```

```

{POINTS.map((p) => {
 const isSel = sel && sameP(sel, p);
 const isHover = hover && sameP(hover, p) && !isSel;
 return (
 <g key={`gp-${p.key}:${p.grade}`} onClick={() => { setSel(p); }} className="cursor-pointer">
 <circle cx={PX(p.key)} cy={PY(p.grade)} fill="black" stroke="black" stroke-width="1" />
 <circle cx={PX(p.key)} cy={PY(p.grade)} fill={isHover ? "#94a3b8" : "#a5b4fc"} stroke="black" stroke-width="1" />
 {p.key}
 </g>
);
})}

```

```

{ id: "split", label: "Split $\approx$ " },
{ id: "merge", label: "Merge " },
{ id: "erase", label: "Erase " },
{ id: "layers", label: "Миф: $2k/2k+1$ " },
{ id: "search", label: "Поиск $\neq$ сортировка" },
{ id: "game", label: "Игра " },
];

```

/\*\* Значения из Python-компилятора: x — ключ, y — приоритет

```

function useTreapLive(): { x: number | null; y: number | null; i: number | null; linked: boolean } {
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const x = vizNumber(vars?.x);
 const y = vizNumber(vars?.y);
 const i = vizNumber(vars?.i);
 return { x, y, i, linked: x !== null || y !== null || i !== null };
}

```

```

const LiveChip: React.FC<{ live: { x: number | null; y: number | null; i: number | null; linked: boolean } }> = ({ live }) => {
 if (!live || !live.linked) return null;
 return (

 Python: x={live.x ?? "-"} · y={live.y ?? "-"} · i={live.i ?? "-"}

);
};

```

```

export const TreapBuildViz = () => {
 const [tab, setTab] = useState<Tab>("build");

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white rounded-md">

 </div>
 <div></div>
 </div>
 <div></div>
 </div>
 </div>
);
};

```

## Универсальное пособие • полный исходный код

```

import { ArticulationPointsViz } from "./ArticulationPo
import BellmanFordViz from "./BellmanFordViz";
import BfsViz from "./BfsViz";
import { DPVisualizer } from "./DPVisualizer";
import { DfsBridgesSimulator } from "./DfsBridgesSimula
import DijkstraViz from "./DijkstraViz";
import { EulerSimulator } from "./EulerSimulator";
import FloydViz from "./FloydViz";
import { GraphTraversalViz } from "./GraphTraversalViz"
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import MnemonicCards from "./MnemonicCards";
import { PlanarityDemo } from "./PlanarityDemo";
import { ColoringViz } from "./ColoringViz";
import { ComponentsViz } from "./ComponentsViz";
import { ComplexityClassesViz } from "./ComplexityClass
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomaton
import { SegmentTreeVisualizer } from "./SegmentTreeVis
import SplayPlayground from "./SplayPlayground";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithms"
import { TopologicalSortViz } from "./TopologicalSortVi
import { WaterfallAnimationWidget } from "./WaterfallAn
import ChapterImage from "./ChapterImage";
import { PrefixSumViz } from "./visualizers/PrefixSumVi
import { SparseTableViz } from "./visualizers/SparseTab
import { TreapBuildViz } from "./TreapBuildViz";

export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}
```

```

 title: "Treap: плоскость → дерево → Split/Merge/Era
 hint: "7 вкладок: Собрать (сортировка + соединение
 обери дерево мышкой, судья-инварианты объясняет к
 Component: TreapBuildViz,
},
"prefix-sums-2d": {
 title: "Префиксные суммы (1D и 2D)",
 hint: "Наведите на число — увидите, из чего оно сло
 Component: PrefixSumViz,
},
"dynamic-programming": {
 title: "Динамическое программирование",
 hint: "Таблица подзадач заполняется на глазах.",
 Component: DPVisualizer,
},
"splay-tree": {
 title: "Splay: механика → большое дерево → собери с
 hint: "Три вкладки: покадровая механика поворотов в
 песочница «подними узел сам» с проверкой порядка.
 Component: SplayPlayground,
},
"splay-rotations": {
 title: "Splay: механика → большое дерево → собери с
 hint: "Три вкладки: покадровая механика поворотов в
 песочница «подними узел сам» с проверкой порядка.
 Component: SplayPlayground,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component:
"top-sort": { title: "Топологическая сортировка", Component:
"scc-kosaraju": { title: "Компоненты сильной связности", Component:
"graph-components": {
 title: "Компоненты связности: BFS-разметка",
 hint: "Каждый запуск обхода красит один «остров»; ч
 Component: ComponentsViz,
},
"graph-articulation": { title: "Точки сочленения", Component:
"bridges-code": { title: "Мосты: DFS + tin/low", Component:
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл"

```

```

"johnson-algo": { title: "Алгоритм Джонсона", Component: JohnsonAlgo },
"mst-kruskal": { title: "Краскал и DSU", hint: "Сортировка по весу ребра. За фазу компонент становится вдвое меньше.", Component: () => <KruskalSimulator defaultAlgo="kruskal" /> },
"mst-prim": { title: "Прим (плесень растёт от старта. Первое ребро наружу.", Component: () => <KruskalSimulator defaultAlgo="prim" /> },
"mst-boruvka": { title: "Борувка (коллективизация)", hint: "За фазу компонент становится вдвое меньше.", Component: () => <KruskalSimulator defaultAlgo="boruvka" /> },
"string-kmp": { title: "Префикс-функция (КМП)", Component: () => <StringKMP /> },
"string-z-func": { title: "Z-функция", Component: () => <StringZFunc /> },
"aho-corasick": {
 title: "Ахо-Корасик",
 hint: "Бор, суффиксные ссылки и BFS-обход по уровням",
 Component: () => (
 <div className="space-y-10">
 <WaterfallAnimationWidget />
 <SalmonAutomatonWidget />
 </div>
),
},
"complexity-classes": {
 title: "Классы сложности: матрёшка $P \subseteq NP \subseteq PSPACE$ ",
 hint: "Кликни класс или задачу; стрелки $\Leftarrow$ показывают включения",
 Component: ComplexityClassesViz,
},
intro: { title: "Мнемокарточки", Component: MnemonicCards },
};

```

```

export const getViz = (id?: string): VizEntry | undefined

```

ФАЙЛ 76/94: src/components/WaterfallAnimationWidget.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

import React, { useEffect, useState } from 'react';
import { useVizRuntime, vizNumber, vizString } from '../vizRuntime';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

```

```

3: { S: 4 },
4: {},
5: { S: 6 },
6: {},
7: { H: 8 },
8: { E: 9 },
9: {},
};

export const WaterfallAnimationWidget: React.FC = () =>
// Переключатель режимов: 'training' (Полный BFS обход)
const [activeMode, setActiveMode] = useState<'training' | 'search'>('training')

// === СОСТОЯНИЯ РЕЖИМА ПОИСКА (SEARCH) ===
const [textToScan, setTextToScan] = useState<string>('')
const [currentIndex, setCurrentIndex] = useState<number>(0)
const [currentNode, setCurrentNode] = useState<number>(0)
const [isSearchDone, setIsSearchDone] = useState<boolean>(false)
const [searchLog, setSearchLog] = useState<string>('')
 'Режим поиска: Введите текст и жмите «Следующий шаг»'
);
const [jumpPath, setJumpPath] = useState<{ from: number; to: number }>({ from: 0, to: 0 })
const [foundMatches, setFoundMatches] = useState<{ index: number; path: string }[]>([])
const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(0)

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
const [trainStep, setTrainStep] = useState<number>(0)

// — Синхронизация с Python-компилятором: s/текст, i/индекс, v/вершина
const runtime = useVizRuntime();
const liveText = vizString(runtime?.variables?.s) ?? '';
const liveI = vizNumber(runtime?.variables?.i);
const liveNode = vizNumber(runtime?.variables?.v) ?? 0;
useEffect(() => {
 if (liveText) {
 setTextToScan(liveText.toUpperCase().replace(/[^\w\s]/g, ''));
 }
}, [liveText]);

```

```

let jumpType: 'normal' | 'fail' | null = null;
void jumpType;
let originalCurr = curr;

if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];
 logMsg += `У пусла #${curr} (${WATERFALL_NODES[curr].label}). `;
 setJumpPath({ from: curr, to: nextNode, type: 'normal' });
 curr = nextNode;
 setActiveSearchCodeLine(3);
} else {
 setActiveSearchCodeLine(2);
 let jumps = 0;
 while (curr !== 0 && TRIE_TRANSITIONS[curr][char] === undefined) {
 curr = WATERFALL_NODES[curr].fail;
 jumps++;
 }
 const nextNode = TRIE_TRANSITIONS[curr][char] ?? 0;
 if (originalCurr === 0) {
 logMsg += `В корне нет перехода по '${char}'. Логика завершена.`;
 setJumpPath(null);
 } else {
 logMsg += `Поток по '${char}' у вершины #${originalCurr} не достиг. Переходим по fail-ссылке в #${curr} (${WATERFALL_NODES[curr].label}). `;
 setJumpPath({ from: originalCurr, to: nextNode, type: 'fail' });
 curr = nextNode;
 }
}

const currentMatches: { index: number; word: string }[] = [];
const matchedWordsForLog: string[] = [];

let temp = curr;
while (temp !== 0) {
 if (WATERFALL_NODES[temp].is_terminal) {
 currentMatches.push({ index: currentMatches.length, word: WATERFALL_NODES[temp].words.forEach((word) => {

```

```

 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 7,
 title: 'Шаг 2 (Depth 1): Вершина #7 (S)',
 desc: 'Обрабатываем дочернюю вершину #7 (S) на Depth 1. Родитель: H. Разведчик плывет в ROOT и ищет ветку по "S".',
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 2,
 title: 'Шаг 3 (Depth 2): Вершина #2 (HE)',
 desc: 'Переходим на Depth 2 к вершине #2 (HE). Родитель: H. Разведчик плывет в ROOT и ищет ветку по "E".',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'E',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 5,
 title: 'Шаг 4 (Depth 2): Вершина #5 (HI)',
 desc: 'Обрабатываем #5 (HI) на Depth 2. Родитель: H. Разведчик плывет в ROOT и ищет ветку по "I". "H" и "S" не подходят. Ветки нет, fail[HI] = ROOT',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'I',
 checkingBranchesFrom: 0,
 },

```



```

 desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плывет в #1 (H) и ищет ветку "E". У вершины :
codeLine: 4,
scoutPos: 1,
inspectedParentFail: 1,
checkTargetChar: 'E',
checkingBranchesFrom: 1,
},
{
 targetNodeId: 4,
 title: 'Шаг 9 (Depth 4): Вершина #4 (HERS)',
 desc: 'Финальная вершина #4 (HERS) на Depth 4. По
 #7 (S). fail[HERS] = #7 (S). Автомат полностью
codeLine: 4,
scoutPos: 0,
inspectedParentFail: 0,
checkTargetChar: 'S',
checkingBranchesFrom: 0,
},
];

```

```

const currentTrainInfo = TRAINING_STEPS_INFO[trainStep];
const activeFishPosNode = activeMode === 'training' ?
const targetTrainingNode = WATERFALL_NODES[currentTrainInfo.targetNode];

```

```

return (
 <div className="bg-slate-800/95 rounded-2xl p-6 border" >
 /* Шапка и переключатель режимов */
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center space-x-3" >

 <div>
 <h4 className="text-2xl font-bold text-white" >
 Анимация водопада: Полное обучение автомата
 </h4>
 <p className="text-sm text-slate-400" >
 Визуализация пошагового построения всех ф
 </p>

```

```

 Полный обход в ширину (BFS)
 </h5>
 <p className="text-xs text-slate-300 mb-3">
 Показаны все шаги очереди BFS (начиная с шага 8 (SHE → HE), где наглядно видно, как происходит обход)
 </p>
 <div className="space-y-1.5 mb-6 overflow-hidden">
 {TRAINING_STEPS_INFO.map((step, idx) => (
 <button
 key={idx}
 onClick={() => setTrainStep(idx)}
 className={`w-full text-left px-3 py-2 ${
 idx === trainStep
 ? 'bg-indigo-600 text-white font-medium'
 : 'bg-slate-800 text-slate-400'
 }`}
 >
 {step.description}
 {idx === trainStep && Текущий шаг}
 </button>
))}
 </div>
</div>

<div className="flex gap-2">
 <button
 onClick={() => setTrainStep((prev) => Math.max(0, prev - 1))}
 disabled={trainStep === 0}
 className="flex-1 bg-slate-700 hover:bg-slate-600 text-slate-200 font-medium"
 >
 Назад
 </button>
 <button
 onClick={() => setTrainStep((prev) => Math.min(TRAINING_STEPS_INFO.length - 1, prev + 1))}
 disabled={trainStep === TRAINING_STEPS_INFO.length - 1}
 className="flex-1 bg-indigo-600 hover:bg-indigo-500 text-white font-medium"
 >
 Вперед
 </button>
</div>

```

```

 </div>
 <div className={`p-1.5 rounded flex items-center justify-between border-emerald-500 text-emerald-300 font-mono`} >
 4. if (trie[v][char] !== undefined) {
 {currentTrainInfo.codeLine === 4 && <
ан IF!}
 }
 </div>
 <div className={`p-1.5 rounded flex items-center justify-between border-blue-500 text-blue-300 font-bold`} >
 5. else fail[u] = root; // Bero
 {currentTrainInfo.codeLine === 5 && <
root}
 </div>
 </div>
</div>

<div>
 <h5 className="text-slate-200 font-bold mb-2">
 <Lightbulb className="w-4 h-4 text-amber-400"/>
 </h5>
 <div className="bg-slate-800 p-4 rounded-lg w-full min-h-[72px] flex items-center">
 {currentTrainInfo.desc}
 </div>
</div>
</div>
</div>
) : (
 /* ПАНЕЛЬ РЕЖИМА ПОИСКА */
 <div className="grid grid-cols-1 lg:grid-cols-3">
 <div className="bg-slate-900/60 p-5 rounded-xl">
 <div>
 <h5 className="text-white font-bold mb-2">
 ⚡ Текст для сканирования
 </h5>
 <p className="text-xs text-slate-400 mb-4">
 Введи текст, чтобы лосось просканировал

```

```

 disabled={isSearchDone || textToScan.length === 0}
 className={`w-full py-2.5 rounded-lg font-mono text-sm
 ${isSearchDone || textToScan.length === 0
 ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
 : 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'}
 }`}
 >
 {currentIndex === 0 ? 'Начать план' : ''}
 <ArrowRight className="w-4 h-4" />
 </button>
 </div>
</div>

<div className="lg:col-span-2 bg-slate-900/60 p-4 rounded-2xl">
 <div>
 <h5 className="text-white font-bold mb-3">
 ✂ Текущее действие в коде
 </h5>
 <div className="bg-slate-950 p-4 rounded-2xl">
 <div className={`p-1.5 rounded flex items-center justify-between
 border-blue-500 text-blue-300 font-bold`} >
 1. curr = state; char = text[i]
 {activeSearchCodeLine === 1 && инициализация}
 </div>
 <div className={`p-1.5 rounded flex items-center justify-between
 border-rose-500 text-rose-300 font-bold`} >
 2. while (curr !== root && !trie[curr].fail)
 {activeSearchCodeLine === 2 && прыжок по fail!}
 </div>
 <div className={`p-1.5 rounded flex items-center justify-between
 border-blue-400 text-blue-200 font-bold`} >
 3. curr = trie[curr].get(char)
 {activeSearchCodeLine === 3 && ем по течению}
 </div>
 </div>
 </div>

```

```

<div className="flex flex-wrap gap-4 text-xs">

</div>
</div>

```

```

{/* Само SVG дерево водопада */}
<div className="w-full overflow-x-auto custom-scrollbar">
 <svg viewBox="0 0 800 520" className="w-full h-full">

```

```

 {/* Горизонтальные линии и подписи уровней */}
 <g>
 {
 { depth: 0, y: 60, label: 'Depth 0 (Корень)' },
 { depth: 1, y: 160, label: 'Depth 1 (H, S)' },
 { depth: 2, y: 260, label: 'Depth 2 (HE, L)' },
 { depth: 3, y: 360, label: 'Depth 3 (HER, L)' },
 { depth: 4, y: 460, label: 'Depth 4 (HERS, L)' }
 }.map((level) => {
 <g key={`depth-line-${level.depth}`}>
 <line
 x1="20"
 y1={level.y}
 x2="780"
 y2={level.y}
 stroke="#334155"
 strokeWidth="1"
 strokeDasharray="4,4"
 opacity="0.4"
 />
 <rect x="25" y={level.y - 12} width="14" height="24" fill="#94a3b8" />
 <text x="32" y={level.y + 4} fill="#94a3b8">
 {level.label}
 </text>
 </g>
 })
 </g>

```

```

 textAnchor="middle"
 >
 {char}
 </text>

 {/* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В
 {isTrainingExamined && (
 <g transform={`translate(${(fromNode
.y) / 2)}`}>
 <rect x="-15" y="-12" width="30
strokeWidth="2" />
 <text x="0" y="5" fontSize="14"
 {isMatchBranch ? ' ' : ' '}
 </text>
 </g>
)}
 </g>
);
});
}}

```

```

{/* Рендер fail-ссылок (пунктирные водопады)
Object.values(WATERFALL_NODES).map((node) => {
 if (node.id === 0) return null;
 const targetNode = WATERFALL_NODES[node.failId];

 // В режиме обучения показываем fail-ссылку
 // Найдём индекс шага, на котором обработали
 const stepIdxForNode = TRAINING_STEPS_INFO[node.id].stepIdx;
 if (activeMode === 'training' && trainStep === stepIdxForNode) {

 let isHighlighted = activeMode === 'search' || activeMode === 'fail';
 let isJustEstablished = activeMode === 'training' && !node.failLinkEstablished;

 // Вычисляем изогнутую кривую для красоты
 const dx = targetNode.x - node.x;

```

```

 <circle cx={node.x} cy={node.y} r="24"
)}

```

```

 {/* Основной круг вершины */}

```

```

 <circle
 cx={node.x}
 cy={node.y}
 r={isCurrent || isTargetChild ? '24' : '12'}
 fill={isCurrent ? '#2563eb' : isTargetChild ? '#f8d7da' : 'none'}
 stroke={isCurrent ? '#ffffff' : isTargetChild ? '#000000' : 'none'}
 strokeWidth={isCurrent || isTargetChild ? 1 : 0}
 className="transition-all duration-300"
 />

```

```

 {/* Текст внутри вершины */}

```

```

 <text
 x={node.x}
 y={node.y + 5}
 fill={isCurrent || isTargetChild ? 'white' : 'black'}
 fontSize={isCurrent || isTargetChild ? 12 : 10}
 fontWeight="bold"
 textAnchor="middle"
 >
 {node.label}
 </text>

```

```

 {/* Метка искомой вершины в режиме обхода */}

```

```

 {isTargetChild && (
 <g transform={`translate(${node.x}, ${node.y})`}
 <rect x="-35" y="-12" width="70" height="24" fill="#f8d7da"
 <text x="0" y="2" fontSize="10" fill="white">
 ИЩЕМ FAIL
 </text>
 </g>
)}

```

```

 {/* Индикатор словарных слов */}

```

```

 {hasWords && (

```

```

 </div>
 </div>

 /* Список найденных совпадений (только в режиме I
 {activeMode === 'search' && (
 <div className="mt-6 bg-slate-900/50 p-5 rounded
 <h5 className="text-white font-bold mb-3 text
 Улов лосося (Найденные слова
 </h5>
 {foundMatches.length === 0 ? (
 <p className="text-sm text-slate-400 italic
 Пока ничего не поймали. Запустите шаги сн
 </p>
) : (
 <div className="flex flex-wrap gap-2">
 {foundMatches.map((match, idx) => (
 <div
 key={idx}
 className="bg-emerald-950 border bord
 -sm font-bold shadow animate-[scaleUp_0.3s_
 >
 <CheckCircle2 className="w-4 h-4 text
 {match.word}
 <span className="text-[10px] bg-emera
 Индекс {match.index}

 </div>
 </div>
 </div>
)
 </div>
)
};
};
=====

```



```

"w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] te

export const PrefixSumViz: React.FC = () => {
 const [mode, setMode] = useState<"1d" | "2d">("1d");
 const chapterId = useContext(VizChapterContext);

 // Вкладка 2D = своё демо (свой скелет): компилятор п
 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, mode === "2d"
 }, [chapterId, mode]);

 return (
 <div className="w-full bg-slate-950 rounded-2xl bor
 <div className="flex gap-2 mb-4 overflow-x-auto n
 {(
 [
 ["1d", "1. Одномерные суммы"],
 ["2d", "2. Двумерные суммы"],
] as const
).map(([key, label]) => (
 <button
 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm
 mode === key ? "bg-indigo-600 text-white"
 }`}
 >
 {label}
 </button>
)))
 </div>
 {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
 </div>
);
};

/* ----- 1D -----
const Prefix1D: React.FC = () => {

```

```

 <div key={i} className={` ${cellBase} !h-5
 {i}
 </div>
)})
 </div>

 { /* массив a */ }
 <div className="flex gap-1.5 items-center mb-3" >
 <div className="w-[46px] shrink-0 text-right" >
 {A1.map((v, i) => {
 const inCover = covered && i >= covered.f
 const inRange = i >= range.l && i <= range.r
 const isDrivenAdd = shownDriven !== null && i === shownDriven
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind: 'add' })}
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange((r) => (i < r.l ? r.l : i > r.r ? r.r : r))}
 className={` ${cellBase} cursor-pointer` }
 isDrivenAdd={isDrivenAdd}
 ? "bg-amber-500 text-black ring-2 ring-black"
 : inCover
 ? "bg-indigo-500 text-white ring-2 ring-white"
 : inRange
 ? "bg-indigo-900/70 text-indigo-300"
 : "bg-slate-800 text-slate-300"
 >
 {v}
 </div>
)
 })}
 <div
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
)
)})
</div>

 { /* массив P */ }
 <div className="flex gap-1.5 items-center">

```

```
</div>
```

```
{/* Пояснение под курсором */}
```

```
<div className="min-h-[62px] bg-slate-900 border
```

```
 {shownDriven !== null && !hover ? {
```

```
 <p className="text-slate-300">
```

```
 <b className="text-emerald-400">Компилятор
```

```
 i =
```

```
 <span className="font-mono text-emerald-300
```

```
 {hasLiveP && <> = <span className="font-mono
```

```
 Пустой
```

```
 </p>
```

```
) : hover?.kind === "pref" ? {
```

```
 hover.i === 0 ? {
```

```
 <p className="text-slate-300">
```

```
 <b className="text-emerald-400">P[0] = 0<
```

```
 отрезка, начинающегося с нуля.
```

```
 </p>
```

```
) : {
```

```
 <p className="text-slate-300">
```

```
 <b className="text-emerald-400">
```

```
 P[{hover.i}] = {pref[hover.i]}
```

```
 {" "}
```

```
 – это сумма всего, что набежало от начала
```

```
 <span className="font-mono text-indigo-300
```

```
 a[0..{hover.i - 1}] = {A1.slice(0, hove
```

```

```

```
 </p>
```

```
 }
```

```
) : hover?.kind === "a" ? {
```

```
 <p className="text-slate-300">
```

```
 <b className="text-indigo-400">
```

```
 a[{hover.i}] = {A1[hover.i]}
```

```
 {" "}
```

```
 входит во все префиксы, начиная с{" "}
```

```
 <span className="font-mono text-emerald-300
```

```
 подвинуть границу запроса.
```

```
 </p>
```

```
const P = useMemo(() => {
 const p = Array.from({ length: n + 1 }, () => Array(
 for (let i = 1; i <= n; i++) {
 for (let j = 1; j <= m; j++) {
 p[i][j] = A2[i - 1][j - 1] + p[i - 1][j] + p[i]
 }
 }
);
 return p;
}, [n, m]);
```

```
const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });
const [rect, setRect] = useState({ r1: 1, c1: 1, r2: 1, c2: 1 });
```

```
useEffect(() => {
 const r1 = vizNumber(vars?.r1);
 const c1 = vizNumber(vars?.c1);
 const r2 = vizNumber(vars?.r2);
 const c2 = vizNumber(vars?.c2);
 if (r1 === null || c1 === null || r2 === null || c2 === null) return;
 // Python-шаблон (и глава) используют 1-индексные координаты. В JS же
 // клетка (1, 1) – левый верхний элемент матрицы. В Python же (0, 0) –
 // 0-индексные, поэтому переводим: (1,1) → (0,0).
 const to0 = (v: number, hi: number) => Math.max(0, v - 1);
 setRect({
 r1: to0(r1, n - 1),
 c1: to0(c1, m - 1),
 r2: to0(r2, n - 1),
 c2: to0(c2, m - 1),
 });
}, [vars, n, m]);
```

```
const D = P[rect.r1][rect.c1];
const B = P[rect.r1][rect.c2 + 1];
const Cc = P[rect.r2 + 1][rect.c1];
const Aa = P[rect.r2 + 1][rect.c2 + 1];
const total = Aa - B - Cc + D;
```

```
const cornerOf = (i: number, j: number) => {
```



```

 </p>
 </div>
 </div>
);
};

```

---

ФАЙЛ 78/94: src/components/visualizers/SparseTableViz.t  
 КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРӨК: 830 | РАЗМ

---

```

import React, { useContext, useEffect, useState } from
import {
 emitVizDemo,
 useVizRuntime,
 vizArray,
 vizNumber,
 vizRecord,
 VizChapterContext,
} from "../../data/vizStepBus";
import { motion, AnimatePresence } from "motion/react";
import {
 Play,
 RotateCcw,
 ChevronRight,
 ChevronLeft,
 Lock,
 ChevronDown,
} from "lucide-react";

// Data for 1D Array
const arr1D = [2, 3, 5, 62, 3, 21, 1, 4];
const N = arr1D.length;
const LOG = 4;

const st1D: number[][] = Array.from({ length: N }, () =>
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];

```



```

const compilerLinked = liveI !== null || liveJ !== null;
const shownSource = hasLiveSource
 ? liveSource?.length
 ? liveSource
 : Array.from({ length: N }, () => null)
 : arr1D;
const liveTarget =
 liveI !== null && liveJ !== null && Number.isInteger(liveI) && Number.isInteger(liveJ)
 ? { i: liveI, j: liveJ }
 : null;

// Total steps: we animate computing each element for
const computeSteps: { i: number; j: number }[] = [];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 computeSteps.push({ i, j });
 }
}

const maxStep = computeSteps.length;
// Эта таблица читает i/j/st напрямую. Не двигаем её
// строки произвольного Python-кода.
const pointedCell = hover ?? liveTarget;
const covered = pointedCell ? { from: pointedCell.i, to: pointedCell.i + (1 << j) } : null;

return (
 <div className="flex flex-col gap-5">
 <div className="flex items-center justify-between">
 <div>
 <h3 className="text-lg font-bold text-indigo-600">
 Построение Sparse Table (Min)
 </h3>
 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + (1 \ll (j-1))][j-1])$
 </p>
 </div>
 <div className="flex gap-2">
 <button

```



```

 }` }
 >
 {display}
 </div>

 </div>
);
 }}}
</div>
<div className="mt-2 text-xs min-h-[36px]">
 {hover && covered ? (
 <p className="text-slate-300">
 <b className="text-sky-400">
 ST[{hover.i}][{hover.j}] = {stID[hover.i][{hover.j}]}
 {" "}
 – это минимум на отрезке{" "}

 [{covered.from}, {covered.to}]
 {" "}
 длины $2^{\{hover.j\}}$ = {1 << hover.j}:{ " "}

 min({arrID.slice(covered.from, covered.to)[{hover.j}]}

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое число
 </p>
)}
</div>
</div>

<div className="overflow-x-auto pb-2">
 <div className="inline-block min-w-full bg-slate-100">
 <div className="grid grid-cols-[auto_repeat(4,1fr)]">
 {/* Headers */}
 <div className="text-slate-500 font-mono text-sm">
 i \ j
 </div>
 </div>
 </div>
</div>

```

```

let isActive = !!liveTarget && liveTarget.i === i
let isSrc1 = false;
let isSrc2 = false;

if (compilerLinked && liveTarget && liveTarget.i === i) {
 isSrc1 = i === liveTarget.i && j === liveTarget.j
 isSrc2 = i === liveTarget.i + (1 << liveTarget.j) && j === liveTarget.j
} else if (!compilerLinked && step > 0) {
 const currentStep = computeSteps[step]
 isActive = currentStep.i === i && currentStep.j === j
 if (currentStep.j - 1 === j) {
 if (currentStep.i === i) isSrc1 = true
 if (currentStep.i + (1 << (currentStep.j - 1)) === i) isSrc2 = true
 }
}

// Готовые вычисленные уровни демо не вычисляем
// уровень j=0 всегда берём из уже скомпьютеризованного
const shownValue = compilerLinked ? liveTarget : undefined
const displayValue =
 shownValue === undefined || shownValue === null
 ? "."
 : typeof shownValue === "object"
 ? JSON.stringify(shownValue)
 : String(shownValue);

return (
 <div key={j} className="relative flex flex-direction: column align-items: center">
 {!isValid ? (
 <div className="w-[clamp(30px,90px,100px)] h-[clamp(30px,90px,100px)]" />
) : (
 <motion.div
 onMouseEnter={() => (compilerLinked ? setHover(true) : setHover(false))}
 onMouseLeave={() => setHover(false)}
 onClick={() => (compilerLinked ? setHover(true) : setHover(false))}
 style={{
 initial: { opacity: 0, scale: 1.1 },
 animate: {
 opacity: 1,
 scale: 1,
 },
 }}
 />
)}
 </div>
)

```

```

 }}
 >
 <path
 d={`M 0 0 C 30 0, 30 ${isSrc1 ? "0" : "1"} 0 0`}
 fill="none"
 stroke={isSrc1 ? "#10b981" : "#f87171"}
 strokeWidth="3"
 />
 </motion.svg>
)}
 </div>
);
}}}
</React.Fragment>
)}}
</div>
</div>
</div>

{/* Formula helper text */}
<div className="bg-slate-900 border border-slate-800 p-4" >
 {compilerLinked && liveTarget ? (
 <p className="text-base text-slate-300">
 ST
 i =
 – активна ячейка
 {(hasLiveTable || hasLiveSource) && <> =
 ?.[liveTarget.j] ?? {liveTarget.j === 0 ? liveTarget.i : liveTarget.j}

 </p>
) : step > 0 && step <= maxStep ? (
 (() => {
 const c = computeSteps[step - 1];
 const half = 1 << (c.j - 1);
 return (
 <p className="text-lg text-slate-300">

 ST[{c.i}][{c.j}]
 {" "}
 </p>
)
 })
) : null
) : null
}

```

```

const liveC = vizNumber(vars?.c);
const liveTable = vizRecord(vars?.st2);
const hasLiveSource = !!vars && Object.prototype.hasOwnProperty.call(vars, 'source');
const hasLiveTable = !!vars && Object.prototype.hasOwnProperty.call(vars, 'table');
const liveTableEmpty = hasLiveTable && Object.keys(liveTable).length === 0;
const codeCell = liveR !== null && liveC !== null ? {
 code: liveC,
 compilerLinked: codeCell !== null || liveK !== null
} : null;
const shownK = liveK !== null
 ? Math.max(0, Math.min(3, Math.trunc(liveK)))
 : hasLiveSource
 ? 0
 : k;

// Уровень и клетка берутся прямо из k/r/c; номер строки берется из k
useEffect(() => {
 setHover(null);
 setLocked(null);
}, [k]);

const L = 1 << shownK;
const activeCell = codeCell || locked || hover;

return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 shadow">
 <h3 className="text-xl font-bold text-indigo-400">Задача
 <p className="text-sm text-slate-400 mb-6 text-indent-2em">
 Для наглядности показываем только <b className="text-rose-300">клетку

 клетка!
 </p>

 <div className="flex bg-slate-950 p-1 rounded-lg">
 {[0, 1, 2, 3].map(step => (
 <button
 key={step}
 onClick={() => { setK(step); setHover(null); }}
 >

```

```

const isBlank = shownValue === ''
let highlightClass = isBlank
 ? 'bg-slate-950 text-slate-600'
 : isCurr
 ? 'bg-slate-800/80 text-slate-600'
 : 'bg-slate-800/50 text-slate-600'
let zIndex = 'z-0';

const isActive = !!activeCell &&
 (l.c + L) && ((r - activeCell.r) % (1 << strLen))

if (isActive) {
 if (isCurr) {
 highlightClass = `bg-indigo-500 text-white`
 {locked && locked.r === r && locked.c === c}
 zIndex = 'z-10';
 } else {
 const prevL = 1 << (shownK - 1)
 const isTop = r < activeCell.r
 const isLeft = c < activeCell.c

 if (isTop && isLeft) {
 highlightClass = 'bg-rose-500 text-white'
 md:scale-[1.08]';
 } else if (isTop && !isLeft) {
 highlightClass = 'bg-blue-500 text-white'
] md:scale-[1.08]';
 } else if (!isTop && isLeft) {
 highlightClass = 'bg-emerald-500 text-white'
 9,0.3)] md:scale-[1.08]';
 } else {
 highlightClass = 'bg-amber-500 text-white'
 3)] md:scale-[1.08]';
 }
 zIndex = 'z-10';
 }
}
}

```

Матрица `<span className="font-mono`  
аписи базового слоя:

```

 </p>
 <div className="mt-3 rounded-lg bg-[#0a0f1e] p-4">

 </div>
 </div>
) : (
 <>
 <p className="mb-4">При <span classN
драты имеют размер <span className="font-bo
 <div className="bg-[#0a0f1e] rounded
 /

 </div>
 <p className="mt-6 italic text-indig
 <Play size={14} className="fill-in
a.
 </p>
 </>
 </div>
 </div>
) : (
 <div className="flex flex-col gap-4 animat
 <div className="bg-[#0a0f1e] rounded-xl l
ate-300 shadow-inner overflow-x-auto">
 // Te
 int<
="text-amber-300">1);

 // Кв
 ST</
 <span className="text-rose
e="text-amber-300">1], <sp
 <span className="text-blue
ext-amber-300">1][k-<span className=
pan>

 <span className="text-emerg
="text-amber-300">1][k-<span classNa

```

```

 <div className="flex item
 <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
 <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
 </div>
 <div className="flex item
 <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
 <span className="text
[activeCell.r + prevL][activeCell.c + prevL
 </div>
 </>
)
 }}{()}}
</div>

 <div className="pt-3 mt-3 border-t
) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][shownK
 </div>
 <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
 </div>
) : {
 <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
 Наведите курсор на матрицу {L}x{L
 </div>
 }}
</div>
 }}
</div>
</div>
</div>
);
};

```

```

<div className="grid grid-cols-[repeat(
ative shadow-inner w-max max-w-full">
 {Array.from({length: 64}).map((_,
 const r = Math.floor(idx / 8);
 const c = idx % 8;
 const isInQuery = r >= r1 && r
 return (
 <div key={idx} className={`
y-center rounded text-[clamp(9px,2.4vw,12px)
0 border-rose-500/50 border scale-105 z-10'
 {val2D[r][c]}
 </div>
)
)}
)}

```

```

<div
 className="absolute border-[3px
ow-[0_0_15px_rgba(244,63,94,0.3)] border-da
 style={{
 top: `${(r1 / 8) * 100}%`,
 left: `${(c1 / 8) * 100}%`,
 marginTop: '8px',
 marginLeft: '8px',
 height: 'calc(' + (H/8*100) +
 width: 'calc(' + (W/8*100) +
 }}
/>

```

```

{/* Показываем 4 угла в самой матрице */
<div className="absolute pointer-e
shadow-[0_0_10px_#f43f5e]" style={{ top: `
lc(`${(Ly / 8) * 100}% - 16px}`, height: `ca
 <div className="absolute pointer-e
shadow-[0_0_10px_#3b82f6]" style={{ top: `
width: `calc(`${(Ly / 8) * 100}% - 16px}`, l
 <div className="absolute pointer-e
ld-400 shadow-[0 0 10px #10b981]" style={{

```



Чтобы покрыть весь прямоугольник, мы  
 Name="font-mono bg-[#0a0f1e] px-1 rounded b  
 роса. (Цветные прямоугольники слева).

</p>

```
<div className="bg-[#0a0f1e] rounded-xl
text-slate-300 shadow-inner overflow-x-auto
 <div className="absolute right-3 top-3
 // 1.
 int<
ext-amber-300">1); <span className="
 int<
ext-amber-300">1); <span className="
 int<
"text-slate-500">// Выс: {Lx}

 int<
"text-slate-500">// Шир: {Ly}
<b
```

```
 // 2.
 int<
 В запроса)

```

```
 r1][<span className="text-white bg-r
```

```
 во, чтобы правый край уперся в c2)<b
 r1][<span className="text-white bg-b
```

```
 px, чтобы нижний край уперся в r2)<b
```

```
 r2 - Lx + 1][<span className="
```

```
 и влево от c2)

```

```

```

```

 r-slate-700 text-white px-1 font-bold rounded
 </div>
)
 }
 }
</div>
</div>
</div>

 <div className="mt-auto bg-slate-950 font
)]">
 <div className="text-center font-bold
wrap bg-[#0a0f1e] py-3 px-2 rounded-lg border
 <span className="text-slate-400 font

 <span className="text-blue-400 m
n>
 <span className="text-emerald-40
span>
 <span className="text-amber-400 r
 <span className="text-slate-4
 <span className="ml-3 text-emerald-
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly
 </div>
 </div>

 </div>
 </div>
)
}
```

---

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

---

```

 }
 }

function setTheme(theme) {
 const body = document.body;
 body.classList.remove('theme-dark', 'theme-light', 'theme-terminal');
 body.classList.add('theme-' + theme);

 // Just some visual feedback on buttons
 document.querySelectorAll('[id^="theme-"]')
 .forEach(btn => {
 btn.classList.remove('outline', 'outline-dark');
 });

 if (theme === 'dark') document.getElementById('btn-theme-dark').classList.add('text-white transition-colors');
 if (theme === 'light') document.getElementById('btn-theme-light').classList.add('text-white transition-colors');
 if (theme === 'terminal') document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors');

 document.getElementById('btn-theme-dark').classList.add('text-white transition-colors');
 document.getElementById('btn-theme-light').classList.add('text-white transition-colors');
 document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors');
 document.getElementById('btn-theme-notebook').classList.add('text-white transition-colors');
 document.getElementById('btn-theme-terminal').classList.add('text-white transition-colors');
}

// Initialize theme and mode
setTheme('dark');

// Setup intersection observer for TOC highlighting
document.addEventListener('DOMContentLoaded', () => {
 const mobileMenuBtn = document.getElementById('mobile-menu-btn');
 const mobileDrawer = document.getElementById('mobile-drawer');
 const closeDrawerBtn = document.getElementById('close-drawer-btn');
 const drawerOverlay = document.getElementById('drawer-overlay');

```

```
 });

 const observerOptions = {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: 0
 };

 const observer = new IntersectionObserver((
 entries.forEach(entry => {
 if (entry.isIntersecting && !document
 // Highlight active TOC link lo
 const id = entry.target.id;
 document.querySelectorAll('aside
 if (link.getAttribute('href'
 link.classList.add('bg-
 } else {
 link.classList.remove('l
 }
 }));
 }
 }));
 }, observerOptions);

 document.querySelectorAll('section[id^="que
 observer.observe(section);
 });
 });
</script>
</body>
</html>
```

```
/* Global formula scroll settings */
.formula-scroll {
 overflow-x: auto;
 overflow-y: hidden;
 -webkit-overflow-scrolling: touch;
 scrollbar-width: none; /* Firefox */
 -ms-overflow-style: none; /* IE and Edge */
}
.formula-scroll::-webkit-scrollbar {
 display: none;
}

mjax-container {
 max-width: 100% !important;
}
mjax-container[display="true"] {
 overflow-x: auto;
 overflow-y: hidden;
 max-width: 100%;
 scrollbar-width: none;
 -ms-overflow-style: none;
 -webkit-overflow-scrolling: touch;
}
mjax-container[display="true"]::-webkit-scrollbar {
 display: none;
}

@media (max-width: 640px) {
 html { font-size: 13px; }

 .uno-card { width: 40px; height: 60px; font-size: 10px; }
 .uno-card::before, .uno-card::after { font-size: 10px; }
 .uno-card::before { top: 1px; left: 2px; }
 .uno-card::after { bottom: 1px; right: 2px; }

 .uno-card.mini { width: 28px !important; height: 40px; font-size: 4px; }
 .uno-card.mini::before, .uno-card.mini::after { font-size: 4px; }
}
```

```

body.theme-light .bg-slate-800 { background-color: #1e293b; }
body.theme-light .bg-slate-700, body.theme-light .text-slate-800 { color: #f1f3f4; font-weight: bold; }
body.theme-light .text-slate-200, body.theme-light .text-slate-300 { color: #f1f3f4; }
body.theme-light .text-slate-400 { color: #475569; }
body.theme-light .border-slate-600, body.theme-light .border-slate-700 { border-color: #334155; }
body.theme-light .border-slate-500 { border-color: #475569; }
body.theme-light #top-controls { background-color: #f1f3f4; }

body.theme-notebook {
 background-color: #f8f8f8 !important; /* light gray background */
 color: #3b2818 !important; /* brown ink */
 background-image: repeating-linear-gradient(45deg, transparent, transparent 2px, #3b2818 2px, #3b2818 4px);
 background-attachment: local !important;
 font-family: "Comic Sans MS", "Caveat", "Serif";
}
body.theme-notebook .bg-slate-900, body.theme-notebook .bg-slate-700 {
 background-color: rgba(255, 255, 255, 0.7);
 border-color: #cbd5e1 !important;
 box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}
body.theme-notebook .text-slate-200, body.theme-notebook .text-slate-300 { color: #f1f3f4; font-weight: bold; }
body.theme-notebook .text-slate-400 { color: #475569; }
body.theme-notebook .border-slate-600, body.theme-notebook .border-slate-700 { border-color: #334155; }
body.theme-notebook .border-slate-500 { border-color: #475569; }
body.theme-notebook .font-mono { font-family: "Monospace"; }
body.theme-notebook #top-controls { background-color: #f1f3f4; }

/* CREEPY BLINKING */
@keyframes creepy-blink {
 0%, 95%, 98%, 100% { transform: scaleY(1); }
 96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
 transform-origin: center;

```

```

 </div>
 </div>
</div>

<!-- ОПРЕДЕЛЕНИЕ МАРКЕРОВ SVG -->
<svg width="0" height="0" class="absolute">
 <defs>
 <marker id="arrow-yellow" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#fcd34d" />
 </marker>
 <marker id="arrow-orange" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#f97316" />
 </marker>
 <marker id="arrow-pink" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#f472b6" />
 </marker>
 <marker id="arrow-cyan" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#2dd4bf" />
 </marker>
 <marker id="arrow-indigo" markerWidth="6" m
 <path d="M0,0 L6,3 L0,6 Z" fill="#818cf8" />
 </marker>
 <marker id="arrow-lime" markerWidth="6" mar
 <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63f" />
 </marker>
 <marker id="arrow-green" markerWidth="6" ma
 <path d="M0,0 L6,3 L0,6 Z" fill="#4ade80" />
 </marker>

 <!-- ЛЕГО БЛОКИ -->
 <g id="lego-yellow">
 <rect x="0" y="0" width="40" height="20" />
 <rect x="6" y="-5" width="8" height="6" />
 <rect x="26" y="-5" width="8" height="6" />
 <line x1="2" y1="2" x2="38" y2="2" stro
 </g>
 <g id="lego-red">
 <rect x="0" y="0" width="40" height="20" />

```

```
 <svg xmlns="http://www.w3.org/2000/svg" class="lg:w-10 lg:h-10">
 <path stroke-linecap="round" stroke-linejoin="round" stroke="black" stroke-width="2" d="M4 6 6 4" />
 </svg>
 </button>
```

```
<!-- Затемнение фона на мобилках при открытом меню -->
<div id="drawer-overlay" class="lg:hidden fixed inset-0 bg-slate-500 bg-opacity-50 z-40">
```

```
<div class="max-w-7xl mx-auto mt-10 px-4 flex flex-col">
```

```
<!-- Левое меню (Оглавление) -->
```

```
<aside id="mobile-drawer" class="fixed inset-y-0 left-0 w-full lg:w-72 lg:shrink-0 lg:z-auto
 translate-x-full transition-transform duration-300 ease-in-out
 w-none lg:block lg:w-72 lg:shrink-0 lg:z-auto">
 <div class="h-full flex flex-col p-6 overflow-hidden">
 <div class="flex justify-between items-center">
 <h3 class="font-black text-transparent bg-clip-text bg-gradient-to-r from-blue-500 to-blue-300">
 Содержание
 </h3>
 <button id="close-drawer-btn" class="lg:hidden text-slate-500">
 <svg xmlns="http://www.w3.org/2000/svg" class="h-6 w-6">
 <path stroke-linecap="round" stroke-linejoin="round" stroke="black" stroke-width="2" d="M6 6 8 8" />
 </svg>
 </button>
 </div>
 <nav class="space-y-3 text-sm font-medium">
```

```
 <div>
 <details class="group" open>
 <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-300">
 1. Алгебраические структуры
 </summary>
 <div class="pl-6 space-y-2 text-sm">

 1.1. Группы

 1.1.1. Группы

 </div>
 </details>
 </div>
 </nav>
 </div>
</aside>
```



er-amber-500 pl-3 font-bold text-amber-300">

4. Степени и корни

</a>

</summary>

<div class="pl-6 space-y-2 text

<a href="#q4-def" class="block l

<a href="#q4-moivre" class="blo

<a href="#q4-root" class="block

<a href="#q4-count" class="bloc

</div>

</details>

<details class="group">

<summary class="list-none curso

<a href="#question-5" class:

er-fuchsia-500 pl-3 font-bold text-fuchsia-

5. Группа корней из единицы

</a>

</summary>

<div class="pl-6 space-y-2 text

<a href="#q5-def" class="block l

<a href="#q5-group" class="bloc

<a href="#q5-prim" class="block

<a href="#q5-crit" class="block

<a href="#q5-cyclic" class="blo

</div>

</details>

<details class="group">

<summary class="list-none curso

<a href="#question-6" class:

er-lime-500 pl-3 font-bold text-lime-300">

6. Делимость, НОД и Евклид

</a>

</summary>

<div class="pl-6 space-y-2 text

<a href="#q6-1" class="block ho

```


 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-green-500 pl-3 font-bold text-green-400">
 9. Кольцо многочленов

 </summary>
 <div class="pl-6 space-y-2 text-green-400">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-rose-500 pl-3 font-bold text-rose-400">
 10. Делимость и Схема Г

 </summary>
 <div class="pl-6 space-y-2 text-rose-400">

 </div>
</details>

<details class="group">
 <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-400">
 11. Алгебраическая геометрия

 </summary>
 <div class="pl-6 space-y-2 text-blue-400">

 </div>
</details>

```

```
<div class="pl-6 space-y-2 text
 <a href="#q13-1" class="blo
 <a href="#q13-2" class="blo
 <a href="#q13-3" class="blo
</div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-14" clas
der-fuchsia-500 pl-3 font-bold text-fuchsia
 14. Кратность корня. Ко

 </summary>
 <div class="pl-6 mt-2 space-y-2
 <a href="#q14-1" class="blo
 <a href="#q14-2" class="blo
 <a href="#q14-3" class="blo
 deg)
 <a href="#q14-4" class="blo
ition-colors border border-rose-500/30"> K
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-15" clas
der-cyan-500 pl-3 font-bold text-cyan-400">
 15. Производная. Критер

 </summary>
 <div class="pl-6 mt-2 space-y-2
 <a href="#q15-1" class="blo
 <a href="#q15-2" class="blo
 <a href="#q15-3" class="blo
 <a href="#q15-4" class="blo
 </div>
</details>
```

```
 <span class="font-bold text-fuc

 <a href="#proof-algorithms" class="l
-red-500 pl-3 mt-4 text-red-500 font-bold ml
 ▲ Ликбез: Алгоритмы выживания

</nav>
<script>
 document.querySelectorAll('details.
 detail.addEventListener('toggle
 if (detail.open) {
 document.querySelectorA
 if (other !== detail
 other.removeAtt
 }
 });
 }
 });
 });

 // Scroll spy logic
 document.addEventListener('DOMConten
 const sections = document.query
 ="q5-"], div[id^="q6-"], div[id^="q7-"], di
 v[id^="q13-"], div[id^="q14-"], div[id^="q1
 let isScrolling = false;

 document.querySelectorAll('.grou
 link.addEventListener('clie
 isScrolling = true;
 setTimeout(() => isScro
 });
 });

 const observer = new Intersectio
 if (isScrolling) return;
```

```
 });

 function fallbackModal(contentRaw,
 const modal = document.createElement('div');
 modal.style.position = 'fixed';
 modal.style.top = '0';
 modal.style.left = '0';
 modal.style.width = '100vw';
 modal.style.height = '100vh';
 modal.style.backgroundColor = 'black';
 modal.style.zIndex = '99999';
 modal.style.display = 'flex';
 modal.style.flexDirection = 'column';
 modal.style.alignItems = 'center';
 modal.style.justifyContent = 'center';

 const box = document.createElement('div');
 box.style.width = '80%';
 box.style.height = '80%';
 box.style.backgroundColor = '#1a1a1a';
 box.style.padding = '20px';
 box.style.borderRadius = '10px';
 box.style.display = 'flex';
 box.style.flexDirection = 'column';
 box.style.color = '#fff';

 const header = document.createElement('div');
 header.innerText = 'Экспорт в ';
 header.style.marginBottom = '10px';

 const subHeader = document.createElement('div');
 subHeader.innerText = 'Из-за не
ние веб-студии в новой вкладке (через меню)';
 subHeader.style.marginBottom = '10px';
 subHeader.style.color = '#f87171';
 subHeader.style.fontSize = '0.9em';

 const textarea = document.createElement('div');
```

```

 closeBtn.style.borderRadius = '50%';
 closeBtn.style.cursor = 'pointer';
 closeBtn.onclick = () => modal.close();

 box.appendChild(header);
 box.appendChild(subHeader);
 if (title !== 'PDF') {
 box.appendChild(textarea);
 btnContainer.appendChild(closeBtn);
 }
 btnContainer.appendChild(closeBtn);
 box.appendChild(btnContainer);
 modal.appendChild(box);
 document.body.appendChild(modal);
 }
}

```

```

function downloadMD() {
 let clone = document.querySelector('#md').cloneNode(true);

 // Smart markdown parsing based on DOM
 // Smart markdown parsing based on DOM
 clone.querySelectorAll('h2').forEach(el => {
 clone.querySelectorAll('h3').forEach(el => {
 clone.querySelectorAll('.analog').forEach(el => {
 clone.querySelectorAll('li').forEach(el => {
 clone.querySelectorAll('svg').forEach(el => {
 let caption = '';
 if (el.nextElementSibling && el.nextElementSibling.tagName === 'img') {
 caption = el.nextElementSibling.alt;
 }
 el.textContent = '\n\n[Иллюстрация: ' + caption + ']';
 });
 });
 });
 });
 });
 clone.querySelectorAll('.img-caption').forEach(el => {
 let text = clone.innerText;
 text = text.replace(/\n{3,}/g, '\n\n');
 });
}

```



```

</script>

<!-- ФИКСИРОВАННЫЕ БЛОКИ: СМЕНА ТЕМЫ И ...
<div class="mt-8 flex flex-col gap-4 bo
 <!-- Кнопки Конвертации / Экспорта
 <div class="flex flex-col gap-2">
 <div class="text-xs text-slate-
 <button onclick="downloadWord()
ont-bold flex items-center justify-center g
 Word
 </button>
 <a href="algebra.pdf" target="_
-sm font-bold flex items-center justify-cen
 PDF

 <button onclick="downloadMD()"
-b bold flex items-center justify-center gap-
 Markdown
 </button>
 <button onclick="downloadHTML()
t-sm font-bold flex items-center justify-cen
 HTML
 </button>
 </div>

 <!-- Смена темы -->
 <div>
 <div class="text-xs text-slate-
 <div class="flex items-center g
 <button id="theme-dark" onc
justify-center text-amber-300 hover:text-wh
line-amber-500" title="Темная тема"> </butt
 <button id="theme-light" on
r justify-center text-amber-50 hover:text-w
">* </button>
 <button id="theme-terminal"

```



```

 15.
 17.
 18.
 19.
 20.
 21.
 22.

</div>
</header>

<!-- ===== WTF CONTAINER =====>
<div id="wtf-container" class="hidden">
 <h2 class="text-3xl font-black text-cen

 <div class="overflow-x-auto w-full pb-4">
 <div class="grid grid-cols-3 gap-4">
 <!-- Column headers -->
 <div class="bg-yellow-900/40 bo
оля и кольца, целые числа</div>
 <div class="bg-orange-900/40 bo
ольцо полиномов</div>
 <div class="bg-cyan-900/40 bord
сные числа</div>

 <!-- Row 1: База -->
 <a href="#question-1" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tran
 <div class="text-xs text-yellow
 <div class="font-bold text-slat

 <a href="#question-9" onclick="setM
-4 border-orange-500 hover:bg-slate-700 tran
 <div class="text-xs text-orange
 <div class="font-bold text-slat

 <a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans

```

```

 <div class="text-xs text-yellow
 <div class="font-bold text-slate

 <a href="#question-11" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-5" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans.
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 5: Корни / Уравнения -->
 <div class="hidden md:block"></div>
 <a href="#question-13" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans.
 <div class="text-xs text-cyan-5
 <div class="font-bold text-slate

 <!-- Row 6: Кратность -->
 <div class="hidden md:block"></div>
 <a href="#question-14" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <div class="hidden md:block"></div>

 <!-- Row 7: Производная -->
 <div class="hidden md:block"></div>

```

```
border-l-4 border-fuchsia-500 hover:bg-slate-700
 <div class="text-xs text-fuchsia-500">
 <div class="font-bold text-slate-700">

```

```
<!-- КЛАСТЕР 4: АЛГОРИТМЫ ВЫЖИВАНИЯ -->
<div class="col-span-1 md:col-span-3"
border-b border-red-500/30 pb-2">Кластер 4:
```

```
 <a href="#proof-algorithms" onclick="
:col-span-3 bg-red-900/20 p-4 border-l-4 border-r-4
 <div class="text-xs text-red-500">
 <div class="font-bold text-slate-700">

</div>
</div>
```

```
<!-- ЛИКБЕЗ SECTION INSERTED HERE -->
<section id="proof-algorithms" class="mb-6"
 <div class="flex items-center mb-6"
 <div class="bg-red-500/20 p-3 rounded"
 <svg class="w-8 h-8 text-red-500"
rg/2000/svg">
 <path stroke-linecap="round"
110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
 </svg>
 </div>
 <h2 class="text-3xl pr-4 border-r-4"
</div>
```

```
 <div class="prose prose-invert max-w-3xl"
 <p class="text-lg">
 Хватит тупить. Доказательств
 есь для того, чтобы жать на кнопки. Вот жес
 </p>
```

```
 <h2 class="text-2xl font-bold border-b"
(Меметика)</h2>
```

```


 </div>

 <!-- Индукция -->
 <div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
 <h3 class="text-xl text-yellow-400">Индукция
 <p class="text-sm text-slate-400">
 <ul class="list-disc pl-5" style="list-style-type: none;">

 валось.

 Просто перепиши формулу с буквой k .

 усок k , замени его на жареного карася из

 </div>

 <!-- Двойная делимость -->
 <div class="bg-slate-800/80 p-6" style="background-color: #1a202c; color: white;">
 <h3 class="text-xl text-orange-400">Двойная делимость
 <p class="text-sm text-slate-400">
 <ul class="list-disc pl-5" style="list-style-type: none;">

 вверх по уравнениям).

 рху вниз, показывая, что $\Delta \vdots r_i$

 х. Твой d больше.

 </div>

 <h2 class="text-2xl font-bold" style="color: #f1c40f;">
 ритмы экзекуции)</h2>

 <div class="grid grid-cols-1 md:grid-cols-2" style="background-color: #f1c40f;">
 <div class="bg-slate-800 p-6" style="background-color: #1a202c; color: white;">
 <h4 class="text-red-400">

```

```

<div>
 <h4 class="text-cyan">
 <p class="text-sm text-cyan">
 <code class="text-x
 </div>

<li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm text-cyan">
 x двух, потом последних.

 <code class="text-x
 </div>

 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm text-cyan">
 есть сложение – есть ноль. Если есть умноже
 <code class="text-x
 </div>

 <li class="bg-slate-800/50 p-4">

 <div>
 <h4 class="text-cyan">
 <p class="text-sm text-cyan">
 авь их драться. Умножение всегда врывается
 <code class="text-x
 </div>


```

```
* Файл получается автономным: стили защиты внутрь, инт
* его можно открыть с флешки, отправить в мессенджер и
* Живые React-визуализации в статику не переносятся —
* подставляется заметка со ссылкой на онлайн-версию, а
* которые в приложении всплывают по наведению, собираю
* в конце документа.
*/
```

```
/** Собирает CSS страницы: сначала пробуем правила, при
async function collectCss(): Promise<string> {
 const parts: string[] = [];
 for (const sheet of Array.from(document.styleSheets))
 try {
 const rules = (sheet as CSSStyleSheet).cssRules;
 parts.push(Array.from(rules).map((r) => r.cssText));
 } catch {
 const href = (sheet as CSSStyleSheet).href;
 if (!href) continue;
 try {
 const res = await fetch(href);
 if (res.ok) parts.push(await res.text());
 } catch {
 /* чужой домен — пропускаем */
 }
 }
 }
 return parts.join("\n");
}
```

```
const EXTRA_CSS = `
/* — правки для автономного файла — */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; paddi
.export-head { border-bottom: 1px solid #1e293b; paddin
.export-note { background: #0f172a; border: 1px solid #
 border-radius: .5rem; padding: .85rem 1rem; font-size
```

```

 if (!id) return;
 if (!ids.includes(id)) ids.push(id);
 const link = document.createElement("a");
 link.className = "term-hint";
 link.href = `#term-${id}`;
 link.title = GLOSSARY_BY_ID.get(id)?.short ?? "";
 link.textContent = el.textContent ?? "";
 el.replaceWith(link);
 });

 // <details> в файле лучше раскрыть: иначе код и разб
 host.querySelectorAll("details").forEach((d) => d.set

 return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[]): string {
 if (!termIds.length) return "";
 const items = termIds
 .map((id) => {
 const e = GLOSSARY_BY_ID.get(id);
 if (!e) return "";
 return `<div class="export-term" id="term-${escape
<h3>${escapeHtml(e.title)}</h3>
<p>${escapeHtml(e.short)}</p>
${e.formal ? `<p class="formal">${escapeHtml(e.formal
${e.complexity ? `<p class="cx">Сложность: ${escapeHt
</div>`;
 })
 .join("\n");

 return `<section class="export-gloss">
<h2>Словарик терминов из этой темы</h2>
<p style="font-size:.8rem;color:#94a3b8;margin:0 0 1r
 ны сюда.</p>
 ${items}
</section>`;
}

```

```

function triggerDownload(html: string, filename: string) {
 const blob = new Blob([html], { type: "text/html;charset=utf-8" });
 const url = URL.createObjectURL(blob);
 const a = document.createElement("a");
 a.href = url;
 a.download = filename;
 document.body.appendChild(a);
 a.click();
 a.remove();
 setTimeout(() => URL.revokeObjectURL(url), 2000);
}
```

```
/** Имя файла: «04-splay-derevo.html» – без кириллицы и пробелов
function safeName(title: string, id: string): string {
 const num = title.match(/^s*(\d+)/)?.[1];
 const base = id.replace(/[^a-z0-9-]+/gi, "-").toLowerCase();
 return `${num ? String(num).padStart(2, "0") + "-" : ""}${base}.html`;
}
```

```
/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter) {
 const css = await collectCss();
 const { body, termIds } = prepareBody(chapter.content);
 const html = wrap({
 title: chapter.title,
 subtitle: chapter.category || "Универсальное пособие",
 css,
 body,
 gloss: glossarySection(termIds),
 origin: window.location.origin,
 });
 triggerDownload(html, safeName(chapter.title, chapter.id));
}
```

```
/** Выгрузить всё пособие одним файлом со сквозным оглавлением
export async function downloadBookHtml(chapters: Chapter[]) {
 const css = await collectCss();
 const allTerms: string[] = [];
 let body = <div></div>;
 for (const chapter of chapters) {
 const { body, termIds } = prepareBody(chapter.content);
 body = <div><div></div><div></div></div>;
 allTerms.push(...termIds);
 }
 const gloss = glossarySection(allTerms);
 const html = wrap({
 title: "Универсальное пособие",
 subtitle: "Содержание",
 css,
 body,
 gloss,
 origin: window.location.origin,
 });
 triggerDownload(html, "book.html");
}
```



ФАЙЛ 83/94: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

```
/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн
 *
 * node scripts/audit-coverage.mjs # отчёт
 * node scripts/audit-coverage.mjs --md # markd
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
 entryPoints: [path.join(ROOT, "src/data/content.ts")],
 bundle: true,
 format: "esm",
 platform: "node",
 outfile: bundle,
 logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\s{2
```

```

console.log(`Глав в пособии: ${rows.length}\n`);
const dump = (title, list) => {
 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(` * ${r.title} [$
 console.log();
};
dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
dump("НЕТ 2D (аналогия есть)", gapViz);
console.log(`Пропущенные номера билетов 1-24: ${missi
console.log(`Визуализаторы без главы: ${orphanViz.join
}

```

-----  
ФАЙЛ 84/94: scripts/audit-terms.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 68 | РАЗМ

```

/**
 * Проверка покрытия глав всплывающими подсказками:
 * в каждой ли теме находятся термины из глоссария.
 *
 * node scripts/audit-terms.mjs
 */
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
 entryPoints: ["src/data/content.ts", "src/data/glossa
 bundle: true, format: "esm", platform: "node",
 outdir: "tmp/audit-terms", outExtension: { ".js": ".m
 external: ["react", "react-dom", "motion", "lucide-re
});

const { chapters } = await import(path.join(ROOT, "tmp/
const { GLOSSARY LABELS, GLOSSARY } = await import(path

```

```
 if (ids.size === 0) bad.push(c.title);
 console.log(
 String(ids.size).padStart(3), "|", String(highlight
 c.title.slice(0, 46).padEnd(47), [...ids].slice(0,
);
 }
 console.log("\nГлав без единой подсказки:", bad.length,
 console.log("Терминов в глоссарии:", GLOSSARY.length,
 console.log("Не встретились:", GLOSSARY.filter((g) => !
```

---

ФАЙЛ 85/94: scripts/export/build-guide-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРӨК: 28 | РАЗМ

---

```
/**
 * Оркестратор: бандлит TSX-генератор и запускает его.
 * Использование: node scripts/export/build-guide-pdf.m
 */
import { build } from "esbuild";
import { pathToFileURL } from "node:url";
import path from "node:path";
import fs from "node:fs";

const root = path.resolve(import.meta.dirname, "../..");
const outfile = path.join(root, "tmp/guide-pdf-bundle.c

await build({
 entryPoints: [path.join(root, "scripts/export/guide-p
 outfile,
 bundle: true,
 platform: "node",
 format: "cjs",
 target: "node20",
 jsx: "automatic",
 loader: { ".jpg": "dataurl", ".png": "dataurl", ".css
```

```
ensureDir(OUT_DIR);

const doc = new PDFDocument({
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
});

const stream = fs.createWriteStream(PDF_PATH);
doc.pipe(stream);

for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0 });
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
}

doc.end();
await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

console.log("[3/3] png -> pdf");
console.log(`    страниц: ${pages.length}`);
console.log(`    выход:   ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
});
```

```
[/^src\/utils\/$/, "Утилиты"],
[/^src\/$/, "Ядро приложения"],
[/^scripts\/$/, "Скрипты сборки и экспорта"],
[/^\.github\/$/, "CI / автоматизация"],
[/^[^\/]+$/, "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re.test(rel));

/** Порядок разделов в документе. */
const ORDER = [
 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
 "HTML-разметка (layout)",
 "Утилиты",
 "Скрипты сборки и экспорта",
 "CI / автоматизация",
 "Прочее",
];

function listFiles() {
 let files;
 try {
 files = execFileSync("git", ["ls-files", "--cached"], {
 cwd: ROOT,
 encoding: "utf8",
 })
 .split("\n")
 .filter(Boolean);
 } catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
 const abs = path.join(dir, e.name);
 if (e.isDirectory()) {
 walk(abs);
 } else {
 files.push(abs);
 }
 }
 };
 walk(ROOT);
 }
}
```

```

 const nb = parseInt(b.title.match(/(\d+)/)?[0] ??
 return na - nb;
 });
}

function main() {
 const files = listFiles();
 const chapters = extractChapters(files);
 ensureDir(OUT_DIR);

 const out = [];
 const push = (s = "") => out.push(s);

 const stamp = new Date().toLocaleString("ru-RU", { ti
 const totalBytes = files.reduce((s, f) => s + fs.stat

 push(HR);
 push(" УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТ
 push(" ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФА
 push(HR);
 push();
 push(`Дата генерации:                ${stamp}`);
 push(`Файлов исходного кода:         ${files.length}`);
 push(`Суммарный объём кода: ${humanSize(totalByt
 push(`Глав/билетов в пособии:        ${chapters.length}`);
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕ
 push(HR);
 chapters.forEach((c, i) => {
 push(` ${String(i + 1).padStart(3, " ")} ${c.titl
 });
 push();

 push(HR);
 push(" ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДН
 push(HR);

```

```
const txt = out.join("\n") + "\n";
fs.writeFileSync(TXT_PATH, txt, "utf8");

console.log("[1/3] код -> txt");
console.log(`      файлов:  ${files.length}`);
console.log(`      глав:    ${chapters.length}`);
console.log(`      строк:   ${txt.split("\n").length}`);
console.log(`      выход:   ${path.relative(ROOT, TXT_PATH)}`);
}

main();
```

---

ФАЙЛ 88/94: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМЕР: 1.5 КБ

---

```
/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt (collect-code.mjs)
 * txt -> page-XXXX.png (render-pages.mjs)
 * png -> full_code_all_pages.pdf (build-pdf.mjs)
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся браузеру) */
export const OUT_DIR = path.join(ROOT, "public", "export");

/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают в бакет) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");
```

```
 } catch {
 /* пробуем следующий */
 }
 }
 throw new Error(
 "ImageMagick не найден. Установите его: sudo apt-get install imagemagick
);
}
```

---

ФАЙЛ 89/94: scripts/export/guide-pdf-entry.tsx

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 673 | РАЗМЕР: 10.5 КБ

---

```
/**
 * Автосборка PDF-конспекта `guide_all_topics.pdf`.
 *
 * Что внутри:
 * * весь ТЕКСТ страниц пособия (больше текста, чем HTML-версия);
 * * картинки визуализаций – React-компоненты рендерятся с помощью
 * (renderToStaticMarkup), из разметки берутся SVG-снимки;
 * * НИКАКОГО кода компилятора и интерфейсных кнопок.
 *
 * Результат: public/export/guide_all_topics.pdf, ~40–70 МБ.
 */
```

```
import * as fs from "node:fs";
import * as path from "node:path";
import { createElement } from "react";
import { renderToStaticMarkup } from "react-dom/server";
import PDFDocument from "pdfkit";
// @ts-ignore - у пакета нет типов
import SVGtoPDF from "svg-to-pdfkit/source.js";
import { chapters } from "../../src/data/content";
import { VizChapterContext } from "../../src/data/vizState";
import { VIZ_REGISTRY, getViz } from "../../src/components/vizRegistry";
```



```

}

/** Убрать инлайн-теги, оставив чистый текст с пробелами
function stripTags(html: string): string {
 return decodeEntities(
 html
 .replace(/<(script|style)[\s\S]*?</\1>/gi, " ")
 .replace(/<br\s*\n?>/gi, "\n")
 .replace(/<\/(p|div|li|h[1-6]|tr|summary)>/gi, "\n")
 .replace(/<[>]+>/g, " ")
)
 .replace(/[\t]+/g, " ")
 .replace(/ *\n */g, "\n")
 .replace(/\n{2,}/g, "\n")
 .trim();
}

/** Найти закрывающий тег с учётом вложенности однотипных
function findClose(html: string, openEnd: number, tag: string): number {
 const re = new RegExp(`<\/?${tag}([^\s^>]*)?>`, "gi");
 re.lastIndex = openEnd;
 let depth = 1;
 let m: RegExpExecArray | null;
 while ((m = re.exec(html))) {
 depth += m[0][1] === "/" ? -1 : 1;
 if (depth === 0) return m.index;
 }
 return html.length;
}

function findCloseAny(html: string, openEnd: number, tag: string): { end: number; inner: string } {
 const close = html.toLowerCase().indexOf(`</${tag}`);
 if (close < 0) return { end: html.length, inner: html };
 const inner = html.slice(openEnd, close);
 const end = html.indexOf(">", close) + 1;
 return { end, inner };
}

```

```

 continue;
 }
 if (lower.startsWith("<summary", at)) {
 const openEnd = html.indexOf(">", at) + 1;
 const { end, inner } = findCloseAny(html, openEnd);
 const text = stripTags(inner);
 if (text) blocks.push({ kind: "p", text: "▶ " + text });
 // содержимое details выводим тоже: оно идёт сразу
 const detEnd = html.toLowerCase().indexOf("</details>", at);
 if (detEnd >= 0) {
 blocks.push(...htmlToBlocks(html.slice(end, detEnd)));
 i = detEnd + "</details>".length;
 } else i = end;
 continue;
 }
 if (lower.startsWith("<li", at) || (liM && at === 0)) {
 const openEnd = html.indexOf(">", at) + 1;
 const close = html.toLowerCase().indexOf("", at);
 const text = stripTags(html.slice(openEnd, close));
 if (text) blocks.push({ kind: "li", text });
 i = close < 0 ? html.length : close + 5;
 continue;
 }
 if (lower.startsWith("<tr", at)) {
 const openEnd = html.indexOf(">", at) + 1;
 const close = html.toLowerCase().indexOf("</tr>", at);
 const rowHtml = html.slice(openEnd, close < 0 ? html.length : close);
 const cells: string[] = [];
 const tdRe = /<(t[dh])(\s[^\>]*)?>([\s\S]*?)<\/\1>/g;
 let tm: RegExpExecArray | null;
 while ((tm = tdRe.exec(rowHtml))) cells.push(stripTags(tm[3]));
 if (cells.some(Boolean)) blocks.push({ kind: "table", cells });
 i = close < 0 ? html.length : close + 5;
 continue;
 }
 // <p>
 const openEnd = html.indexOf(">", at) + 1;
 const close = html.toLowerCase().indexOf("</p>", at);

```

```

 if (no >= 0 && no < nc) { depth++; pos = html.indexOf("<div", pos); }
 else { depth--; pos = nc + 6; }
 }
 let svg = html.slice(at, pos);
 if (!/xmlns=/.test(svg)) svg = svg.replace("<svg", "<svg xmlns=");
 // выкинуть элементы с tailwind-классами без инлайнов
 svgs.push(svg);
 idx = pos;
}
const imgRe = /<img[^>]*src="(data:image\/(png|jpeg|gif|svg))/;
let im: RegExpExecArray | null;
while ((im = imgRe.exec(html))) images.push(im[1]);
const grids = extractDivGrids(html);
return { svgs, images, grids };
} catch (e) {
 return { svgs: [], images: [], grids: [], error: String(e) };
}
}

/**
 * Визуализации бывают и без SVG: матрицы из div-ячеек
 * (с префиксными суммами). Собираем такие «таблицы» в текстовый формат
 */
function extractDivGrids(html: string): string[][] {
 const grids: string[][] = [];
 const rows: string[] = [];
 const rowMark = /<div class="[^"]*(?:flex gap-1\.5|grid)/;
 let m: RegExpExecArray | null;
 while ((m = rowMark.exec(html))) {
 // сбалансированно находим конец строки-контейнера
 let depth = 1;
 let pos = m.index + m[0].length;
 while (depth > 0 && pos < html.length) {
 const no = html.indexOf("<div", pos);
 const nc = html.indexOf("</div>", pos);
 if (nc < 0) break;
 if (no >= 0 && no < nc) { depth++; pos = no + 4; }
 else { depth--; pos = nc + 6; }
 }
 }
}

```

```

constructor(stream: fs.WriteStream) {
 this.doc = new PDFDocument({ size: "A4", margins: {
 this.doc.pipe(stream);
 this.doc.on("pageAdded", () => this.pages++);
 this.setupFonts();
 this.bodyFont();
}
setupFonts() {
 if (F_REG) this.doc.registerFont("Body", F_REG);
 if (F_BOLD) this.doc.registerFont("BodyBold", F_BOLD);
 if (F_MONO) this.doc.registerFont("Mono", F_MONO);
 if (F_MONO) this.doc.registerFont("MonoBold", F_MONO);
}
bodyFont() { this.doc.font(F_REG ? "Body" : "Helvetica");
boldFont() { this.doc.font(F_BOLD ? "BodyBold" : "Helvetica-Bold");
monoFont() { this.doc.font(F_MONO ? "Mono" : "Courier");

get y() { return this.doc.y; }
set y(v: number) { this.doc.y = v; }
get pageBottom() { return A4[1] - M.bottom; }
ensureSpace(need: number) {
 if (this.y + need > this.pageBottom) this.doc.addPage();
}
footerAll(skipCover = true) {
 const range = this.doc.bufferedPageRange();
 const oldBottom = this.doc.page.margins.bottom;
 for (let p = range.start; p < range.start + range.count; p++) {
 if (skipCover && p === range.start) continue;
 this.doc.switchToPage(p);
 this.doc.page.margins.bottom = 0;
 this.doc.font(F_REG ? "Body" : "Helvetica").fontSize(12)
 .text(String(p + 1), 0, A4[1] - 30, { width: A4[1] });
 this.doc.page.margins.bottom = oldBottom;
 }
}
}
}

```

const CH LINE = 13.4; // межстрочный в теле

```

let from = 0;
let yTop = top;
while (from < lines.length) {
 const avail = d.pageBottom - yTop;
 let fit = Math.floor((avail - 8) / lead);
 if (fit <= 0 && from === 0 && lines.length > 1) { d
 fit = Math.max(fit, 1);
 const to = Math.min(lines.length, from + fit);
 drawPortion(from, to, yTop);
 from = to;
 if (from < lines.length) { d.doc.addPage(); yTop = l
}
d.y = yTop + Math.min(lines.length, Math.floor((d.pag
d.doc.moveDown(3);
}

```

```

function drawTableBlock(d: Doc, rows: string[]) {
 const size = 8.4;
 d.ensureSpace(size * 3);
 const text = rows.map((c) => c.replace(/\s*\n\s*/g, "
 d.doc.fontSize(size).fillColor("#374151").text(text, l
 d.doc.moveDown(2.2);
}

```

```

function drawGrid(d: Doc, rows: string[]) {
 const size = 8;
 const lead = size * 1.6;
 d.ensureSpace(rows.length * lead + 16);
 const top = d.y;
 d.monoFont().fontSize(size).fillColor("#111827");
 const cellW = Math.min(
 34,
 (CONTENT_W - 16) / Math.max(...rows.map((r) => r.sp
);
 rows.forEach((row, ri) => {
 const cells = row.split("\u2009");
 cells.forEach((cval, ci) => {
 const x = M.left + 6 + ci * cellW;

```

```

}

function drawImage(d: Doc, dataUrl: string) {
 const b64 = dataUrl.split(",")[1];
 if (!b64) return;
 const buf = Buffer.from(b64, "base64");
 const maxW = CONTENT_W * 0.82;
 const avail = d.pageBottom - d.y;
 if (avail < 200) d.doc.addPage();
 try {
 d.doc.image(buf, M.left + (CONTENT_W - maxW) / 2, d
 } catch { return; }
 d.y = d.y + 10 + maxW * 0.6; // примерная высота фото
 d.doc.moveDown(2);
}

// ----- сборка -----

const FRONT_TOC_LINES = 44; // строк содержания на стра

function tocEntries() {
 return chapters.map((c) => ({ id: c.id, title: c.titl
})

function tocPagesNeeded(): number {
 return Math.max(1, Math.ceil(tocEntries().length / FR
})

interface BuildOpts {
 outPath: string;
 record: boolean;
 /** id → стартовая страница (для второго прохода) */
 pageMap?: Map<string, number>;
 frontPages: number;
}

function buildOne(opts: BuildOpts): Stats {
 return new Promise<Stats>((resolve, reject) => {

```

```

 d.y = doc.y;
 };
 drawToHeader();
 entries.forEach((ch, i) => {
 if (line > 0 && line % FRONT_TOC_LINES === 0) {
 doc.addPage();
 drawToHeader();
 }
 const pageNo2 = (opts.pageMap?.get(ch.id) ?? 0);
 const isTopic = /^\\d/.test(ch.title);
 doc.fontSize(isTopic ? 10.5 : 10).font(isTopic ?
 .fillColor(isTopic ? "#1f2937" : "#6b7280"));
 // заголовок + точечная отбивка + номер страниц
 const title = ch.title.length > 86 ? ch.title.s
 const w = CONTENT_W - 40;
 const tw = doc.widthOfString(title);
 const pw = doc.widthOfString(String(pageNo2));
 doc.text(title, M.left, d.y, { width: w, lineBr
 const dots = ".".repeat(Math.max(3, Math.floor(
 doc.fillColor("#d1d5db").text(dots, M.left + tw
 doc.fillColor("#111827").text(String(pageNo2), M
 d.y += 16.2;
 line++;
 }));
 // добиваем до расчётного числа страниц содержания
 const used = Math.ceil(entries.length / FRONT_TOC
 for (let p = used; p < opts.frontPages - 1; p++)
 doc.addPage();
 if (!opts.record) console.log(`TOC: ${entries.leng
}

// — главы —
const stats: Stats = { chapterStart: new Map(), tota

chapters.forEach((ch, ci) => {
 const startPageOf = () => {
 // текущая страница = pages (0-based) + 1
 return doc.bufferedPageRange().start + doc.buff

```

```

 }

 // визуализация главы
 const vizEntry = VIZ_REGISTRY[ch.id];
 if (vizEntry) {
 const art = renderVizArt(ch.id);
 if (opts.record) {
 const n = (art?.svgs.length ?? 0) + (art?.images.length ?? 0);
 console.log(`viz ${ch.id}: svg=${art?.svgs.length}, images=${art?.images.length} or ? " ОШИБКА: " + art.error : ""`);
 if (n === 0) console.log(` ⚠ ${ch.id}: визуализация не выполнена`);
 }
 if (art && (art.svgs.length || art.images.length)) {
 d.ensureSpace(60);
 doc.moveDown(0.6);
 doc.fontSize(10).font(F_BOLD ? "BodyBold" : "Body");
 d.text("■ Визуализация: " + vizEntry.title, 0, 0);
 d.y = doc.y + 6;
 let drawn = 0;
 for (const img of art.images) {
 if (drawn >= 4) break;
 drawImage(d, img);
 drawn++;
 }
 for (const svg of art.svgs) {
 if (drawn >= 5) break;
 drawSvg(d, svg);
 drawn++;
 }
 for (const grid of art.grids) {
 if (drawn >= 7) break;
 drawGrid(d, grid);
 drawn++;
 }
 } else if (art?.error) {
 // SSR упал – рисуем текстовую заглушку, PDF не будет
 }
 }
}

```



```
main().catch((e) => {
 console.error(e);
 process.exit(1);
});
```

---

ФАЙЛ 90/94: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМ: 1.5 КБ

---

```
/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu)
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002.png, ...
 */
```

```
import fs from "node:fs";
import path from "node:path";
import os from "node:os";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import {
 ROOT, PAGES_DIR, TXT_PATH, PAGE, ensureDir, humanSize
} from "../common.mjs";
```

```
const execFileAsync = promisify(execFile);
```

```
/** Раскрываем табы и жёстко переносим длинные строки, чтобы
```

```
function wrap(line) {
 const expanded = line.replace(/\t/g, " ").replace(/ {4}/g, "\t");
 if (expanded.length <= PAGE.cols) return [expanded];
 const parts = [];
 const indent = " ".repeat(Math.min(expanded.match(/^ +/).length, 3));
 let rest = expanded;
```

```

 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
 });

const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : null,
 "-pointsize", String(PAGE.pointsized),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

const concurrency = Math.max(2, Math.min(os.cpus().length, 4));
let done = 0;
let cursor = 0;

async function worker() {
 while (cursor < jobs.length) {
 const job = jobs[cursor++];
 await execFileAsync(im, args(job));
 fs.rmSync(job.txtFile, { force: true });
 done += 1;
 if (done % 25 === 0 || done === total) {
 process.stdout.write(`отрендерено ${done} страниц\n`);
 }
 }
}

await Promise.all(Array.from({ length: concurrency }, () => worker()));
process.stdout.write("\n");

```

```

}
ok(!seg.includes("def build"), "segment-trees init не с

// 2. splay: числовой key + кадр i
const sp = init("splay-tree");
ok(sp.includes("key = 32") && sp.includes("keys = [50")

// 3. КМП/Z: pattern/text/s попадают в init
const kmp = init("string-kmp");
ok(kmp.includes('pattern = "aba"') && kmp.includes('s =

// 4. Форду-Беллману — рёбра реального графа (S→A = 4,
const bf = PAGE_SYNC["bellman-ford"].code ?? "";
ok(bf.includes('{"S", "A", 4}')) && bf.includes('{"D", "
ok(!bf.includes('{"S", "A", 5}'), "bellman-ford: заглуш
ok(!/break/.test(bf), "bellman-ford: без break");

// 5. Джонсон: h(B) = -2, ребро -2, w' = 0
const jh = PAGE_SYNC["johnson-algo"].code ?? "";
ok(jh.includes('"B": -2') && jh.includes("w_new = w + h

// 6. Краскал/Прим/Борувка: 9 вершин и вес 51
for (const id of ["mst-kruskal", "mst-boruvka"]) {
 const c = PAGE_SYNC[id].code ?? "";
 ok(/ABCDEFGHI/.test(c), `${id}: 9 вершин A..I`);
}
ok((PAGE_SYNC["mst-prima"].code ?? "").match(/"I":/g)?
ok((PAGE_SYNC["mst-kruskal"].code ?? "").includes('{13,

// 7. prefix-sums-2d#2d: пометка 1-индексных клеток
const p2 = PAGE_SYNC["prefix-sums-2d#2d"].code ?? "";
ok(p2.includes("клетки 1-индексные"), "prefix#2d: комме

// 8. графы top-sort/scc/graph-dfs-bfs/graph-components
ok((PAGE_SYNC["top-sort"].code ?? "").includes("0: [1,
ok((PAGE_SYNC["scc-kosaraju"].code ?? "").includes("2:
 емо");
ok((PAGE_SYNC["graph-dfs-bfs"].code ?? "").includes('ad

```

```

 return q(step, 2 * v, l, m, ql, qr, d) + q(step, 2 * v, l, m, qr, ql, d)
}

let fails = 0;
for (const [uL, uR, add] of cases) {
 const steps = genLazySteps(base, uL, uR, add);
 const last = steps[steps.length - 1];
 const wantRoot = sum(0, 5) + add * (uR - uL + 1);
 const wantQ = sum(uL, uR) + add * (uR - uL + 1);
 const gotRoot = last.treeState[1];
 const gotQ = q(last, 1, 0, 5, uL, uR, 0);
 const okR = gotRoot === wantRoot, okQ = gotQ === wantQ;
 console.log(`(uL,uR,add)=(${uL},${uR},${add}) root=${gotRoot} q=${gotQ}
 "FAIL" | шагов=${steps.length}`);
 if (!okR || !okQ) fails++;
}
// query без обновлений = обычная сумма
const q0 = q({ treeState: buildTreeFull(base), lazyState: {} }, 1, 0, 5, 1, 3, 0);
console.log("query(1,3) без обновлений =", q0, q0 === 23);
if (q0 !== 23) fails++;
console.log(fails === 0 ? "LAZY SELFTEST: ALL OK" : `LAZY SELFTEST: FAILS=${fails}`);
process.exit(fails === 0 ? 0 : 1);

```

## =====

### РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

## =====

-----

ФАЙЛ 93/94: [.github/workflows/deploy-pages.yml](#)

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 83 | РАЗМЕР: 2.9 КБ

-----

name: Deploy app to GitHub Pages

```
ДВЕ настройки репозитория нужно сделать один раз ВРУЧНУЮ
(у бота-интеграции нет admin-прав на них – API отвечает за них)
#
```

```
run: npm ci --no-audit --no-fund

- name: Build for the repository subpath
 env:
 VITE_BASE: /algv0/
 run: npm run build

- name: Make GitHub Pages fallback
 run: cp dist/index.html dist/404.html

- name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
 path: ./dist

deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 # Ранняя и понятная ошибка вместо невнятного «Fail»
 # когда в репозитории Pages не переключён на сбор
 - name: Preflight – Pages собирается через GitHub
 env:
 GH_TOKEN: ${ secrets.GITHUB_TOKEN }
 run: |
 bt=$(gh api "repos/${ github.repository }/pages" | jq -r .build_type)
 echo "Pages build_type: ${bt}"
 if ["${bt}" != "workflow"]; then
 echo "::error title=Pages не настроен::Сетите этот запуск. См. комментарий в начале деплоя"
 exit 1
 fi

 - name: Deploy
```

```
 default: "150"

concurrency:
 group: rebuild-pdf-${{ github.ref }}
 cancel-in-progress: true

permissions:
 contents: write

jobs:
 rebuild-pdf:
 name: код → txt → png → pdf
 runs-on: ubuntu-latest
 # не реагируем на собственный коммит бота
 if: "!contains(github.event.head_commit.message, '[bot: ')"

 steps:
 - name: Checkout
 uses: actions/checkout@v6
 with:
 fetch-depth: 0
 persist-credentials: true

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: "24"
 cache: npm

 - name: Install ImageMagick + DejaVu fonts
 run: |
 sudo apt-get update -qq
 sudo apt-get install -y --no-install-recommends imagemagick
 # На Ubuntu политика ImageMagick иногда запрещает конвертировать текст в картинку
 # разрешаем то, что нужно пайплайну (text -> png)
 for policy in /etc/ImageMagick-6/policy.xml /usr/share/ImageMagick-6/policy.xml; do
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{P
```

- name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v7  
with:  
  name: full-code-export  
  path: |  
    public/export/full\_code\_all\_pages.pdf  
    public/export/full\_code\_all\_pages.txt  
    public/export/guide\_all\_topics.pdf  
  if-no-files-found: error  
  retention-days: 30
- name: Upload artifacts (PNG-страницы)  
uses: actions/upload-artifact@v7  
with:  
  name: full-code-pages-png  
  path: tmp/export-pages/\*.png  
  if-no-files-found: error  
  retention-days: 7
- name: Commit rebuilt export back to the branch  
run: |  
  git config user.name "github-actions[bot]"  
  git config user.email "41898282+github-action@users.noreply.github.com"  
  git add -- public/export/full\_code\_all\_pages.pdf  
  if git diff --cached --quiet; then  
    echo "Экспорт не изменился — коммит не нужен"  
    exit 0  
  fi  
  git commit -m "chore(export): пересборка full-code-export"  
  git push origin "HEAD:\${GITHUB\_REF\_NAME}"